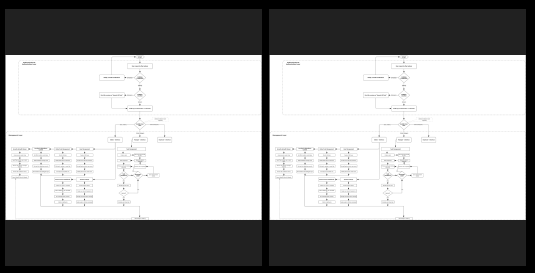
Sun, Sep 6 at 10:18 PM



gemini-chattings.pdf
PDF

Based on the pdf chattings and uploaded images you can assess what I am working on right.. the complex version of the Admin-interface in the image is my design, and so i am stuck at task management under manager role right now, so based on the resources that I uploaded... tell me are you getting my work right now?

Worked for 9s

Yes, mate. **I'm getting your work now — and I understand where you are stuck.** I went through the uploaded chat history and the two diagrams, so I'm not looking at this as a generic "task management flow."

What I understand you are building

You are designing a **multi-role management system** with three roles:

Admin → Manager → Employee

After authentication, the system determines the user's role and routes them into the appropriate interface. Your overall diagram is divided into layers, with **Authentication & Authorization** already designed and the **Management Layer** currently being developed. gemini-chattings

Your **Admin side is largely worked out** using multiple managerial functions such as User Management, Location & Attendance, Global Task Management, System Settings, Announcements, and Security/Audit. gemini-chattings

But **you are currently working specifically on:**

> **Management Layer → Manager Interface → Task Management**

And this is where your two images matter.

The important distinction between your two diagrams

1. The second/reference diagram

The simpler diagram is essentially the **reference architecture/style** you're taking inspiration from.

Its Manager lane is basically:

```
MANAGER DASHBOARD
       |
       v
TASK MANAGEMENT
       |
       v
Create Task
       |
       v
Set Priority & Due Date
       |
       v
Assign Task to Employee
       |
       v
Notify Employee
       |
       v
Employee Works
       |
       v
Task Completed?
       |
       v
Manager Reviews
       |
       v
Approved?
       |
       v
Complete / Close
```

with reassignment as a side loop.

That diagram has the **visual simplicity** you like: relatively few boxes, obvious decisions, clean loops.

The previous discussion explicitly recognized that this reference diagram stands out largely because of its **visual symmetry and readability**, rather than because its

underlying task logic is particularly sophisticated. gemini-chattings

---

2. YOUR diagram

Your large diagram is different.

You aren't simply copying that task flow.

You're trying to build a **more realistic management system while retaining the reference diagram's readability**.

And you've arrived at this architectural idea:



That hexagon is the part you're currently stuck on.

And I understand why you're putting it there.

The key idea you've developed

The first four operations are **not the complicated part**.

They are the deterministic creation/dispatch pipeline:

```
Create Task
     ↓
Set Priority & Due Date
     ↓
Assign to Employee
     ↓
Notify Employee
```

The system has now created a live task.

Only from this point does the task become dynamic.

That's why you told me the hexagon should sit **immediately after those four linear steps**.

gemini-chattings +1

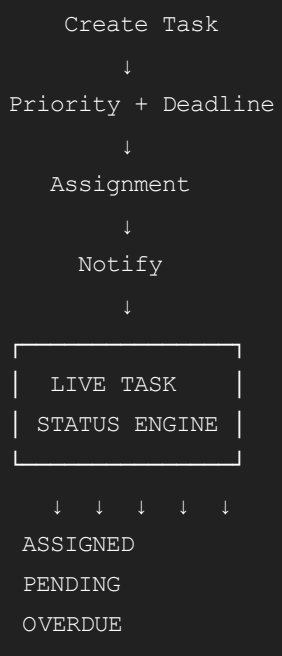
And that's the part I now understand clearly.

The hexagon isn't really "another task step"

This is the important architectural distinction.

You're envisioning it as a **status/state control point**.

Something conceptually like:



```
REASSIGNED
INCOMPLETED
```

Then each status has its own appropriate managerial consequence.

That idea came directly out of your later redesign discussion: you wanted the central control mechanism to produce the same meaningful task categories that the reference diagram's **Automated Reporting** ultimately consumes. gemini-chattings

---

And there's another thing I understand now

You **don't want the Manager to look like a glorified task secretary.**

That was one of the problems identified in the earlier version:

> Create task → priority → assign → employee works

That makes the Manager look like someone who merely distributes work.

Your real-world interpretation of the Manager is broader:

- allocate work/data
- consider employee workload
- establish priority/deadline
- monitor progress
- deal with blockers
- intervene when necessary
- reassign work
- review output
- provide feedback
- approve completion
- maintain accountability/audit history

The uploaded discussion specifically developed **Lead Allocation & Operations** around assigning daily targets/data batches, setting priorities/SLA deadlines, monitoring team bandwidth, and reassigning work when someone is unavailable. gemini-chattings

So the Manager flow needs to communicate **management**, not merely CRUD.

Where you are stuck RIGHT NOW

You're essentially asking:

> **"After Notify Employee, what exactly should my hexagonal Task Status controller do, and how should I branch the rest of the Manager Task Management flow without turning the diagram into spaghetti?"**

And **yes — I understand that as the exact problem you're solving.**

You're trying to achieve this balance:

Reference diagram

Simple + visually clean

but

Your diagram

More technically realistic + more managerial + state-aware

without ending up with:



😂 — the dreaded Draw.io spaghetti.

The previous discussion even identified the **spiderweb effect** as the main danger of putting too many incoming/outgoing arrows around one central state hub. gemini-chattings

---

One correction I want us to keep straight

There are actually **two concepts that got mixed together in the previous conversation**:

Internal operational states

Earlier we discussed:

```
ASSIGNED
   ↓
IN PROGRESS
   ↓
ON HOLD / BLOCKED
   ↓
IN REVIEW
```

```
    ↓
CLOSED
```

with rejection/feedback looping back into the work. gemini-chattings

Reporting/management statuses

Then you deliberately pivoted toward the statuses represented in the reference design:

```
ASSIGNED
REASSIGNED
PENDING
INCOMPLETED
OVERDUE
```

because those are what your reporting layer needs to capture. gemini-chattings

That distinction is extremely important.

We shouldn't blindly throw both sets into the diagram.

We need to decide whether your hexagon represents:

> **internal task states**

or

> **management/reporting states**

—or, most likely, design it so the **internal workflow generates the reporting statuses without visually dumping every microscopic state onto the chairperson-facing diagram.**

That is the architectural problem we're actually solving now.

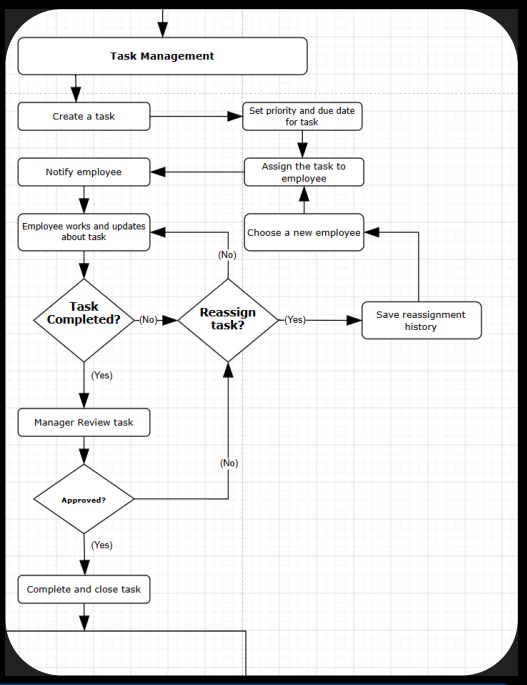
So, in one sentence:

Yes mate — I have the picture.

You're not asking me to invent a generic task-management flow. You're building **your own Management Layer**, inspired visually by the reference diagram, and you're currently engineering the **Manager → Task Management** branch where the first four steps are linear and the **hexagonal Task Status mechanism becomes the central decision/routing point for everything that happens to the task afterward.**

And honestly, **this is the point where we should stop adding random boxes and design the state logic properly before touching Mermaid/Draw.io again.**

Sources



So can you guide the task management things right? I came up with this primarily, which is much likely to the ref.version, but it lack the status review and changes things

Yes. **Now I can see exactly what you are trying to preserve.** Your current flow is actually a good base because it closely follows the reference diagram's visual language, but the weakness is exactly what you identified:

> **It describes actions and decisions, but it doesn't explicitly show the task's changing status/state.**

Your uploaded design currently does this:

```
Create
  ↓
Priority + Due Date
  ↓
Assign
  ↓
Notify
  ↓
Employee Works
```

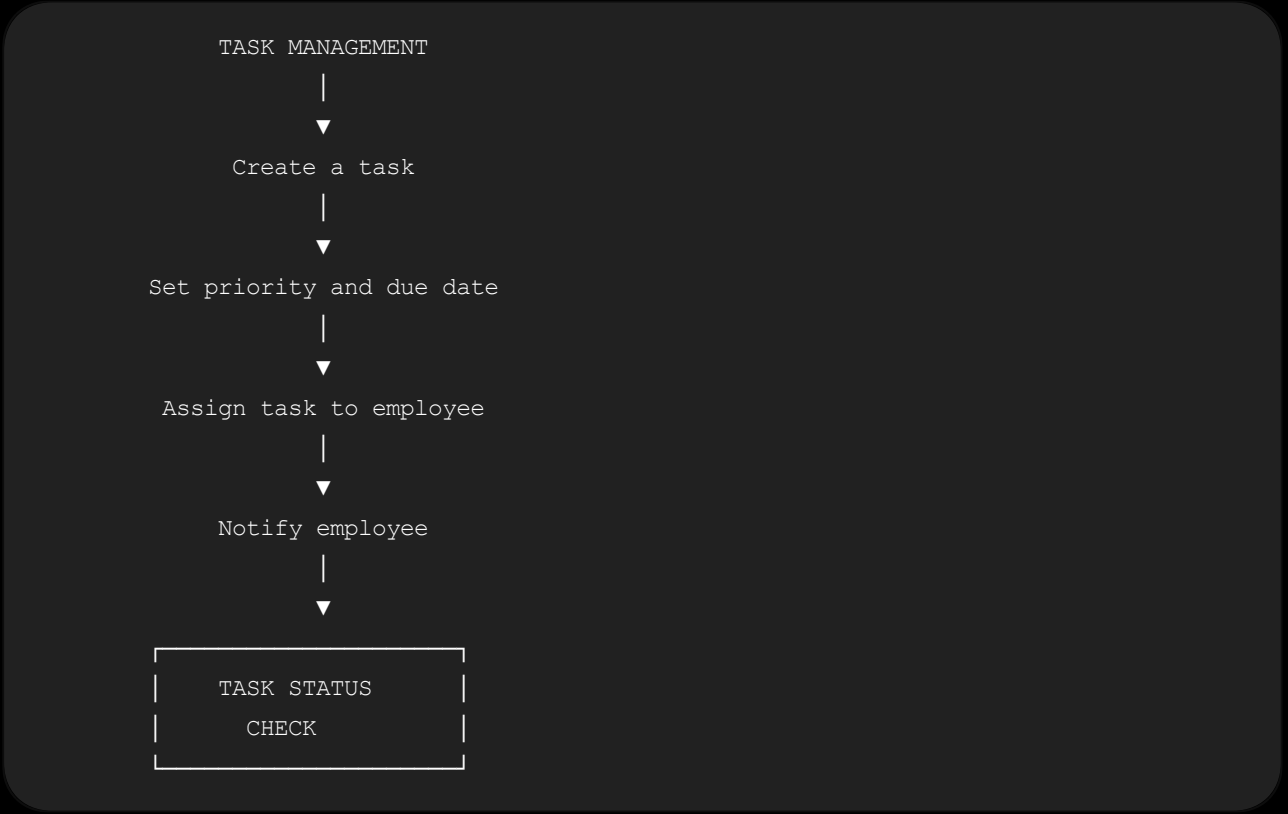
```
  ↓
Task Completed?
  ├── NO → Reassign?
  │          ├── YES → Save history → Choose new employee → Assign
  │          └── NO → Employee works
  │
  └── YES → Manager Review
               ↓
             Approved?
             ├── NO → Reassign?
             └── YES → Close
```

That is **structurally understandable**, but I would *not* throw it away.

The uploaded discussion already established that your task system needs to account for statuses such as **Assigned, Reassigned, Pending, Overdue and Completed**, because those statuses feed the reporting layer. gemini-chattings

What I would change

Keep your first four boxes **exactly as they are**:

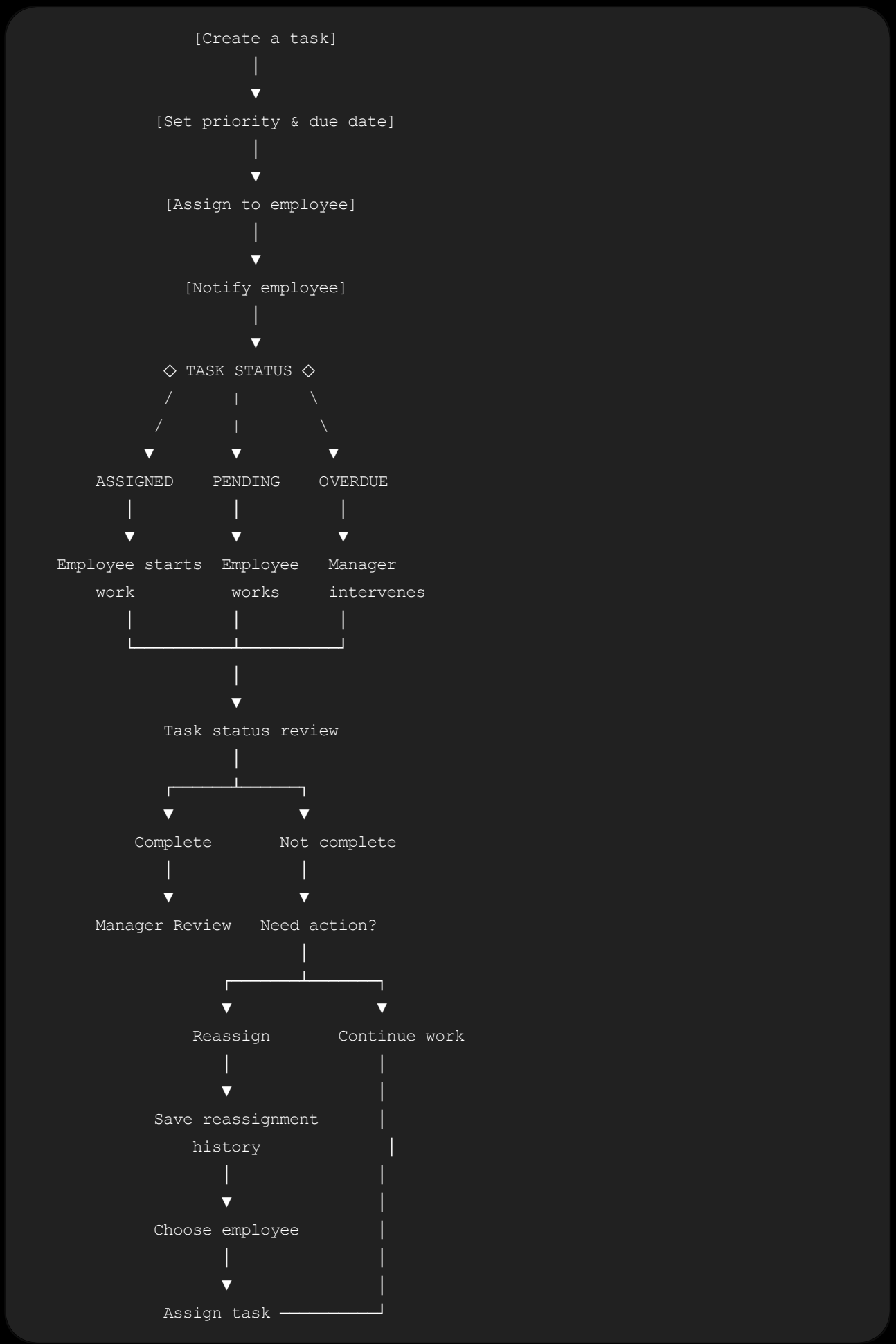


But **don't make the status check simply another "decision" like "Task Completed?"**

Make it the point where the system determines **what state the task is currently in and what happens next.**

The flow I recommend for YOUR diagram

I would make it:



But there's one more important piece.

Don't use "Reassign?" as the answer to every NO

This is the biggest weakness in your current diagram.

Right now:

```
Task Completed?
      |
     NO
      ↓
Reassign task?
```

That implies:

> **If the task isn't completed → maybe reassign it.**

That's not necessarily true.

A task can be incomplete because:

- employee is still working;
- deadline hasn't arrived;
- employee is blocked;
- manager needs to intervene;
- task became overdue;
- employee needs feedback;
- employee is unavailable and therefore reassignment is required.

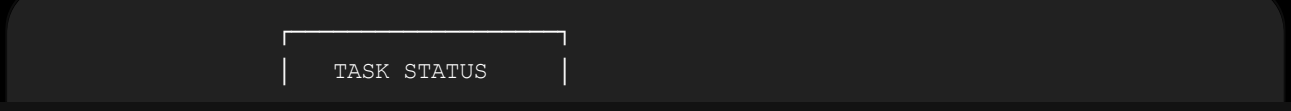
Your previous discussion actually identified this problem: a rejected task should normally return to the **same employee with feedback**, rather than automatically becoming a reassignment. gemini-chattings

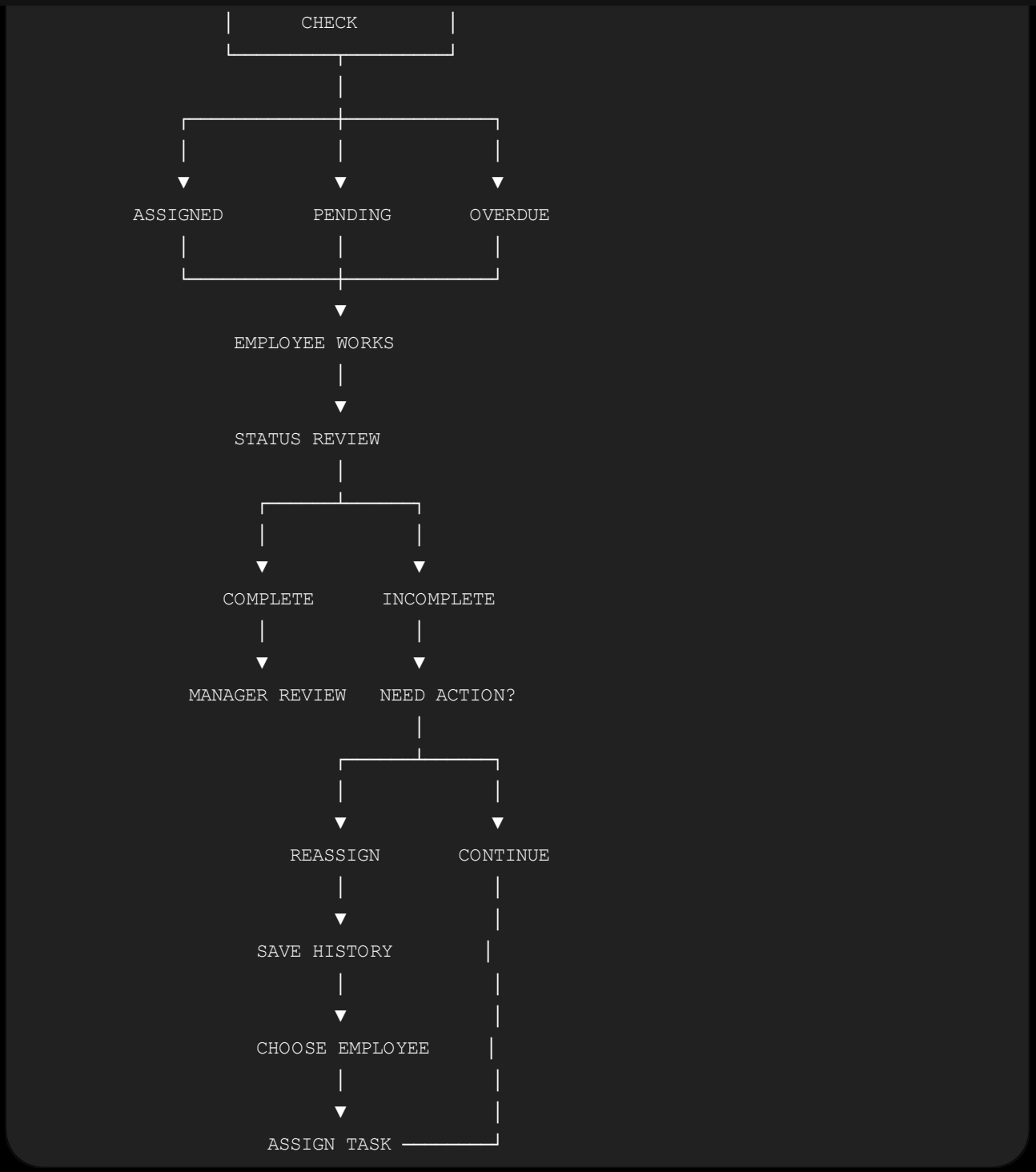
So your diagram needs to distinguish:

Incomplete ≠ Reassigned

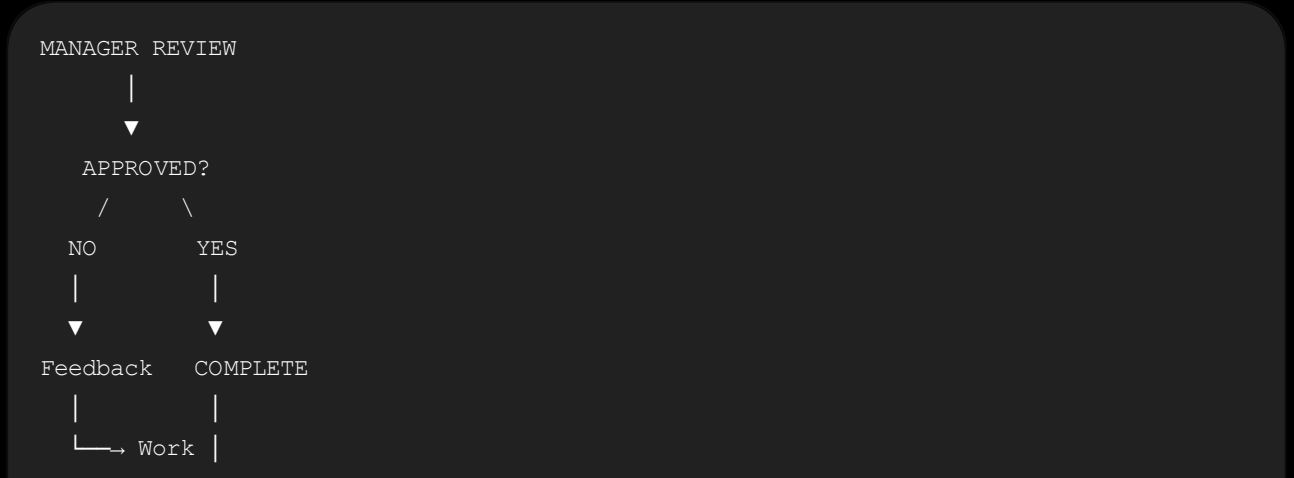
I would therefore make your central status mechanism like this

After "Notify Employee":




CHECK
ASSIGNED
PENDING
OVERDUE
EMPLOYEE WORKS
STATUS REVIEW
COMPLETE
INCOMPLETE
MANAGER REVIEW
NEED ACTION?
REASSIGN
CONTINUE
SAVE HISTORY
CHOOSE EMPLOYEE
ASSIGN TASK

Then:

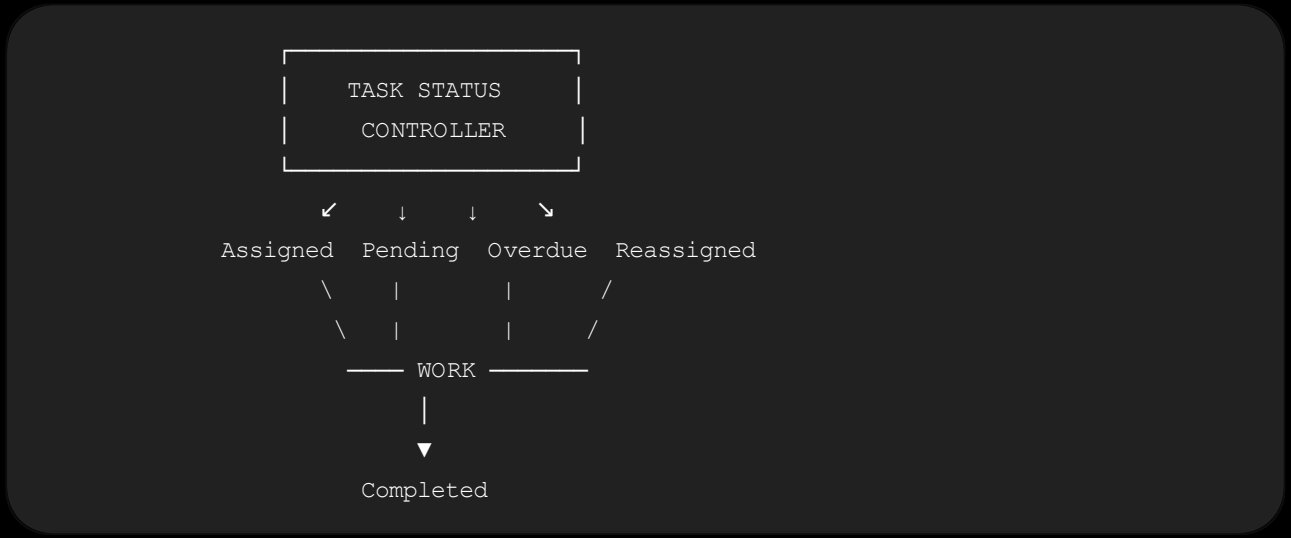

MANAGER REVIEW
APPROVED?
NO
YES
Feedback
COMPLETE
Work

That is much more logically sound.

And here's where your "status changes" idea becomes powerful

You don't necessarily need five huge branches occupying half the page.

Instead, think of the hexagon as a **state controller**:



This is essentially the **State Machine Router** concept you had arrived at in the previous discussion: the central mechanism routes the task according to its current condition rather than forcing everything through one linear waterfall. gemini-chattings

And importantly, your previous source specifically suggested that the five reporting states could be generated by this central router. gemini-chattings

---

But I would make one terminology change

You previously had:

> **INCOMPLETED**

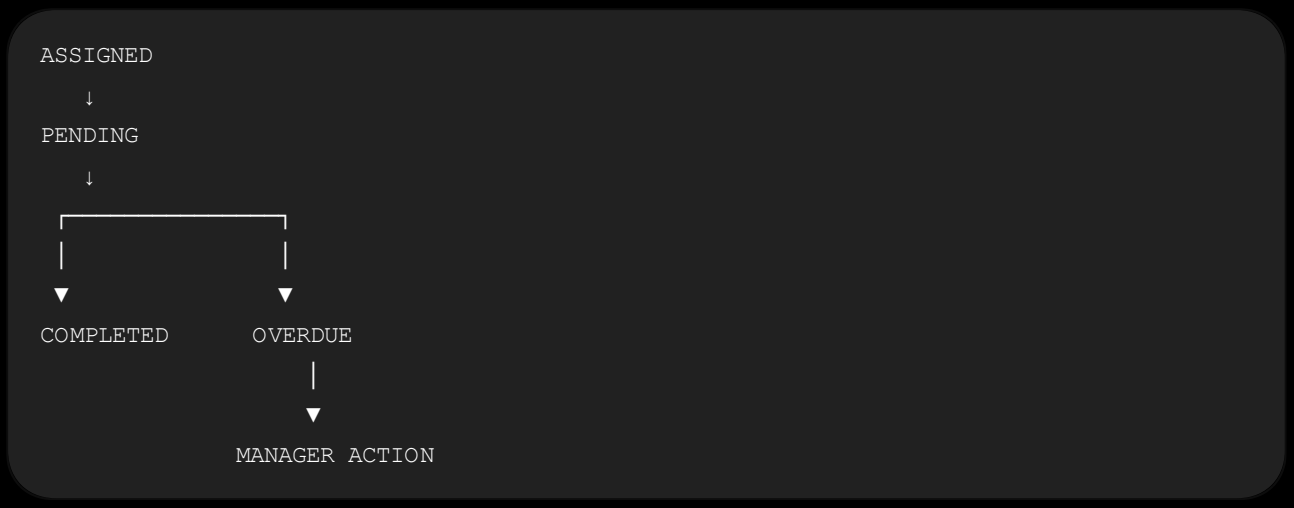
I would use:

> **PENDING**

for the active/non-finished task state.

Then use **OVERDUE** when the due date has passed.

So conceptually:



And:



This makes **REASSIGNED** an event/state transition rather than a completely separate endpoint.

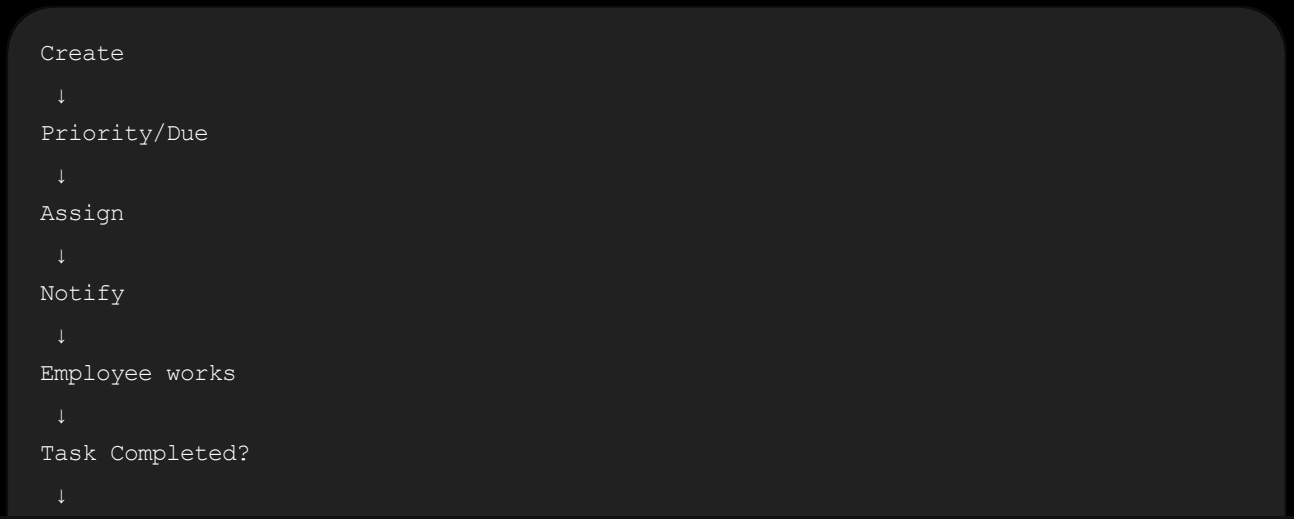
The uploaded discussion itself described reassignment as the manager taking a task from one employee and giving it to another, while overdue represents an SLA/deadline breach. gemini-chattings

---

So your current diagram doesn't need a complete redesign

I'd evolve it like this:

Your current



```
Manager Review
 ↓
Approved?
 ↓
Close
```

Your improved version

```
Create
 ↓
Priority + Due Date
 ↓
Assign
 ↓
Notify
 ↓
╔═══════════════════════╗
║    TASK STATUS        ║
║      CHECK            ║
╚═══════════════════════╝
 ↓
Assigned / Pending / Overdue
 ↓
Employee Works
 ↓
Status Review
 ↓
┌─────────────────┐
│ Task complete?  │
└─────────────────┘
     NO  │ YES
         │    ↓
         │ Manager Review
         │      ↓
         │  Approved?
         │   /     \
         │ NO       YES
         │ ↓         ↓
         │ Feedback  Complete
         │ ↓         ↓
         └→Work    Close
```

And **only when intervention is actually necessary**:

```
Status Review
      ↓
Need reassignment?
      ↓ YES
Choose new employee
      ↓
```

```
Save reassignment history
      ↓
Assign task
      ↓
Task Status Check
```

This is the version I'd pursue

Because it gives you **three layers of intelligence without making the diagram visually insane**:

1. Task creation
→ Create → Priority → Assign → Notify

2. Task state
→ Assigned → Pending → Overdue → Reassigned

3. Manager control
→ Status Review → Reassignment / Feedback / QA → Complete → Audit/Reporting

That is much closer to what you were trying to achieve with the complex version, while retaining the **clean geometry of the reference version**.

And your final **Complete/Close** can feed directly into the Automated Reporting layer, which is exactly where your earlier architecture was heading. gemini-chattings

So I wouldn't replace your diagram. I'd insert the status-control layer into it and fix the `Task Completed? → Reassign?` logic. That's the main surgery it needs.

Can you give me graphical code for: mermaideditor.io

Absolutely. Based on **your current Task Management design**, I'd use a Mermaid flowchart that keeps your original structure but adds the **Task Status Controller** after notification and makes reassignment/feedback logically separate.

Mermaid supports standard flowcharts, decision diamonds, and hexagonal nodes, so this structure is directly suitable for the editor. Mermaid

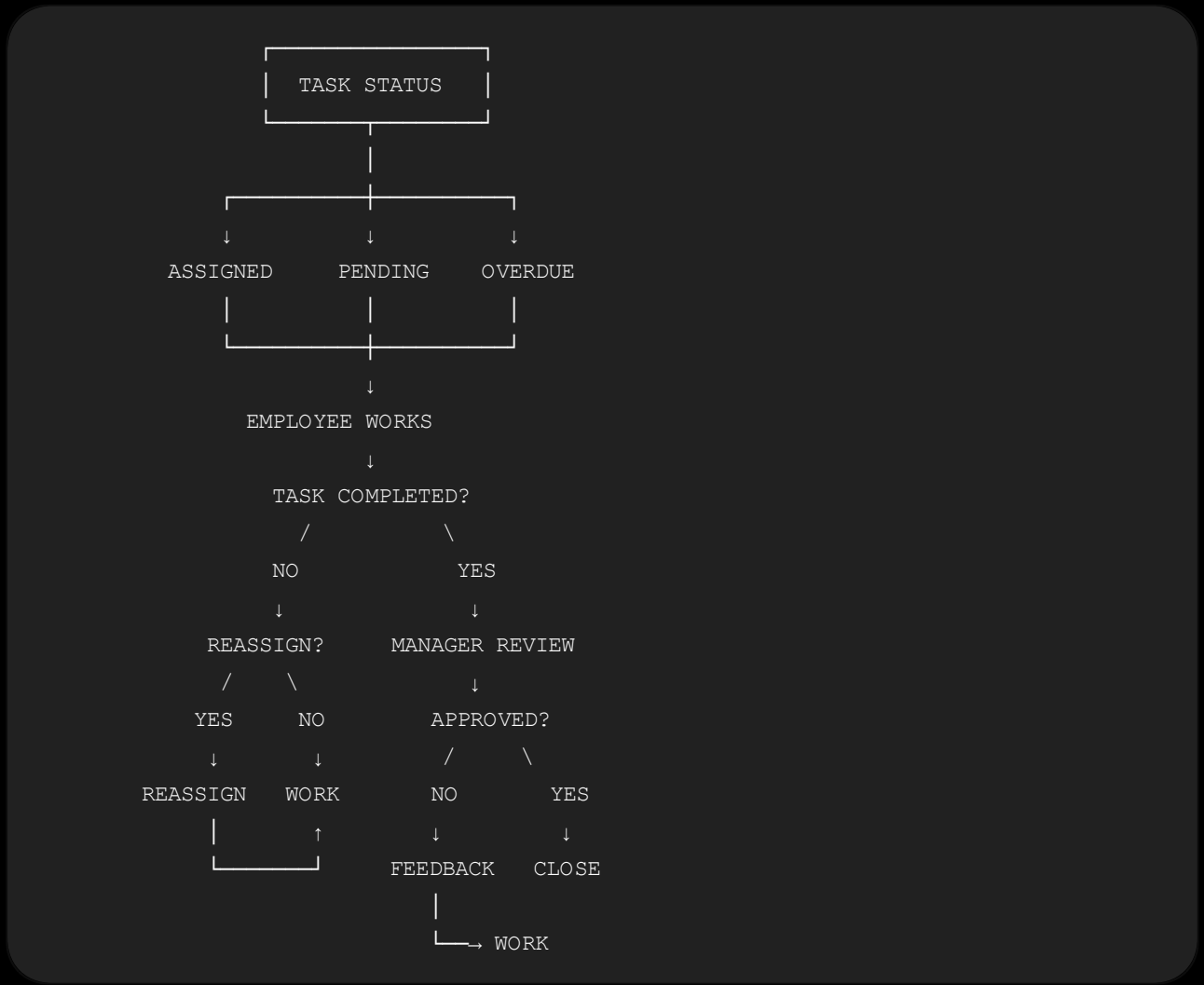
Mermaid code

Loading diagram...

One thing I'd change from this first version

I **wouldn't actually connect** `RA["Reassigned"]` **after** `Complete and close task`. That was only to show the status concept. Reassigned isn't a final state.

The cleaner architecture is:



So `Reassigned` **should be treated as a status transition**, not as the final destination of the task.

That fits the logic we established from your uploaded material: reassignment means the Manager moves the task to another employee, while completion occurs only after Manager review/approval. gemini-chattings

If you want the **next version**, I can give you a **proper polished Mermaid version that visually resembles your reference diagram**—same clean symmetry, Manager flow in the center, status hexagon in the middle, and minimal crossing arrows.

Why did you wrote reassign right after closing the task?

You're right — **that was my mistake.** 😅

I placed:

`Complete and close task → Reassigned`

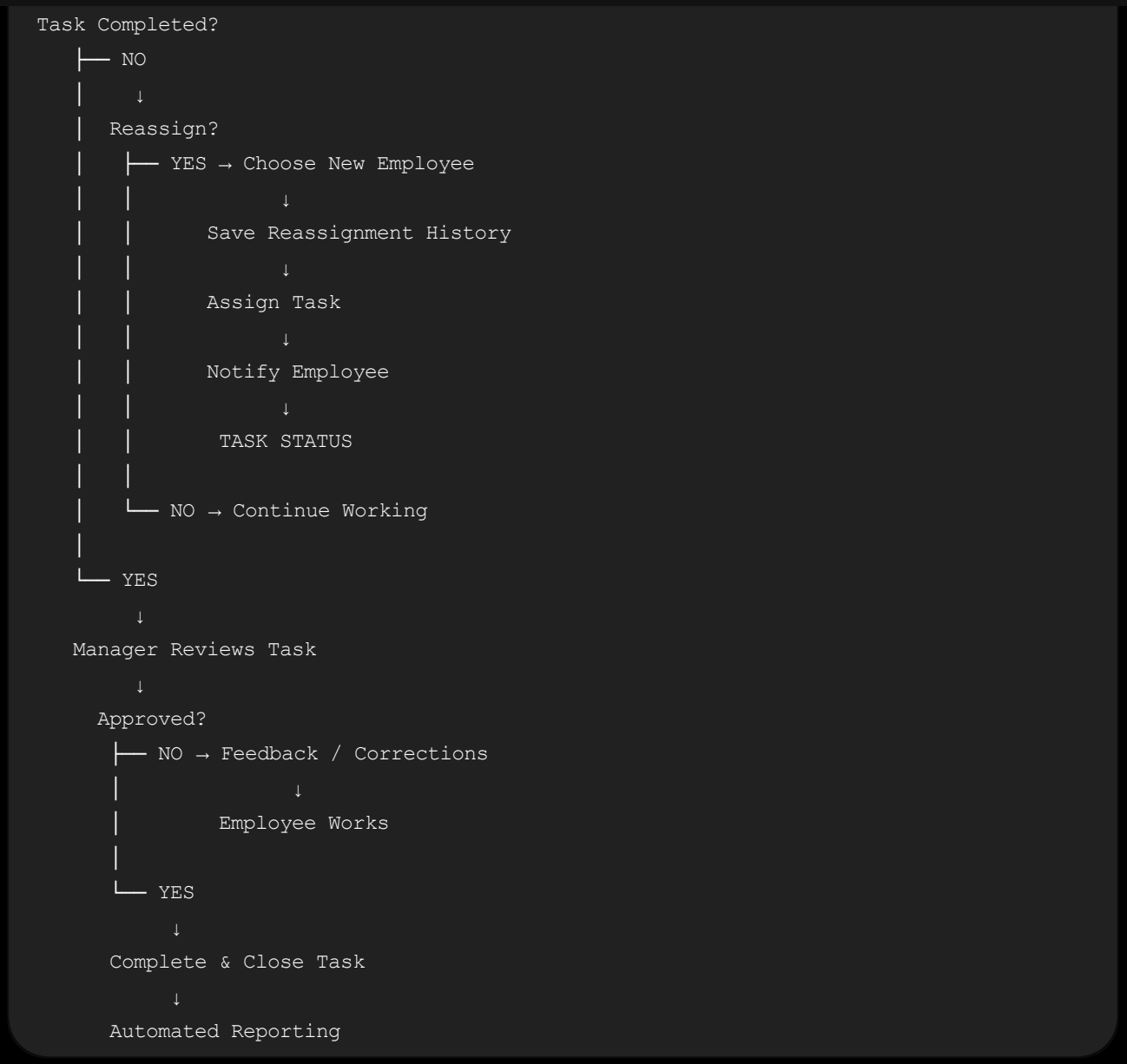
because I incorrectly treated **"Reassigned" as a status that could be reported after a task lifecycle event**. But logically, that makes no sense.

Correct logic

Reassigned is not something that happens after completion.
It happens **during the task lifecycle**, when the manager moves the task from one employee to another.

So the flow should be:

```
Create Task
     ↓
Set Priority + Due Date
     ↓
Assign Employee
     ↓
Notify Employee
     ↓
   TASK STATUS
     ↓
Assigned / Pending / Overdue
     ↓
Employee Works & Updates
     ↓
```



One important distinction

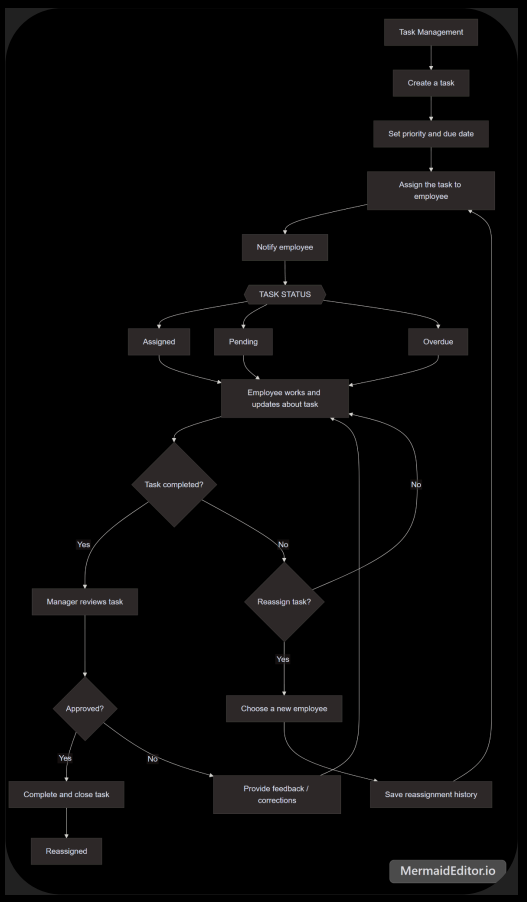
Think of the statuses like this:

Status	Meaning
Assigned	Task has been given to an employee
Pending	Task is active but not completed
Overdue	Due date has passed
Reassigned	Task has been transferred to another employee
Completed	Task has been approved and closed

So **Reassigned should redirect back into the workflow**, not come after `Complete & Close`.

And once a task reaches **Complete & Close**, its lifecycle is over. ✅

The Mermaid code I gave you earlier should therefore be corrected.



I think for the overdue part we need to add a bit logic here what do you say?

Yes — **absolutely.** Looking at your current diagram, the **Overdue branch is the one part that feels too passive**.

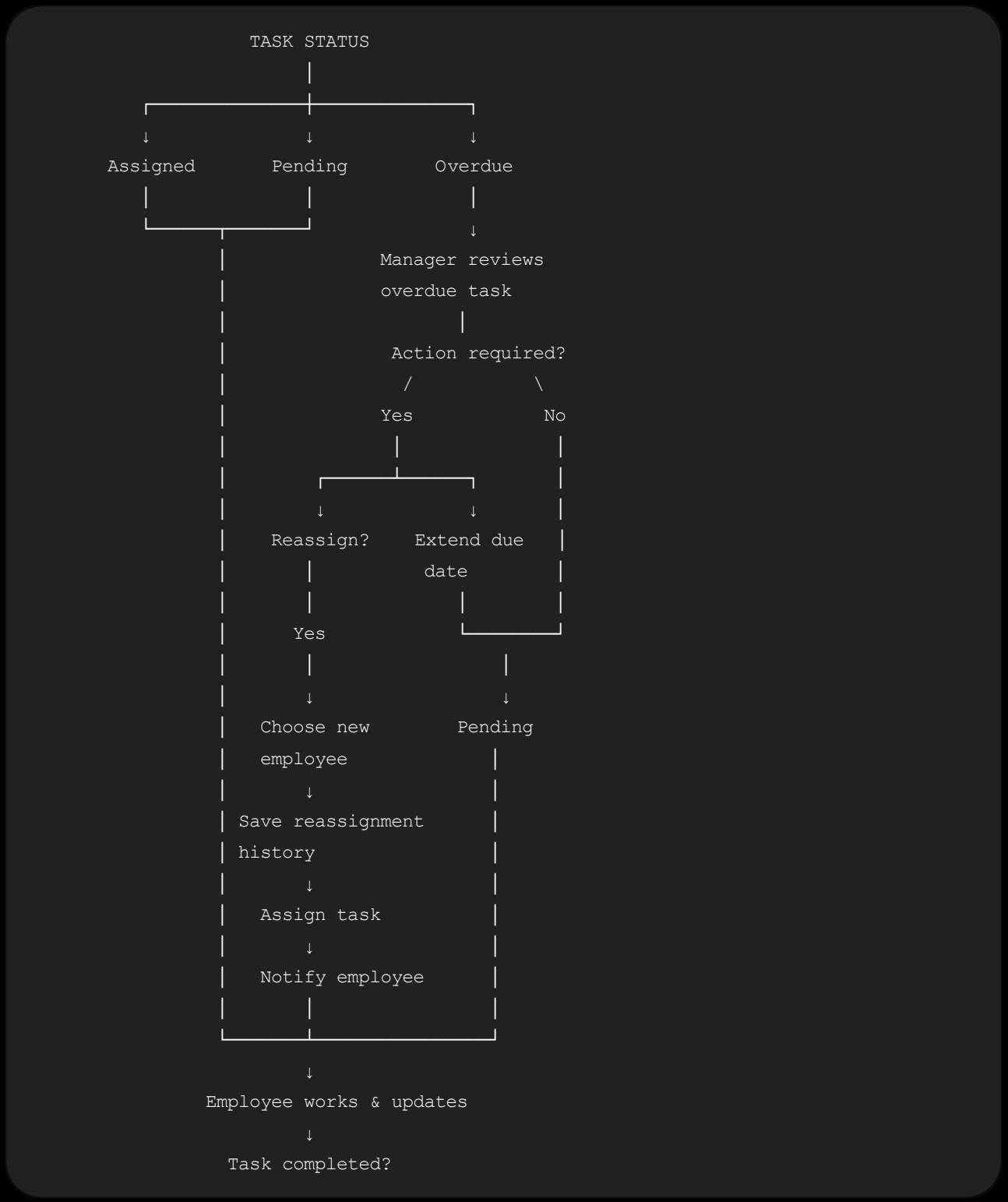
Right now you have:

```
TASK STATUS
   ↓
Overdue
   ↓
Employee works and updates
```

That implies that when a task becomes overdue, the system simply says *“okay, keep working.”* 😅

For a management system, **Overdue should trigger manager intervention/decision logic**.

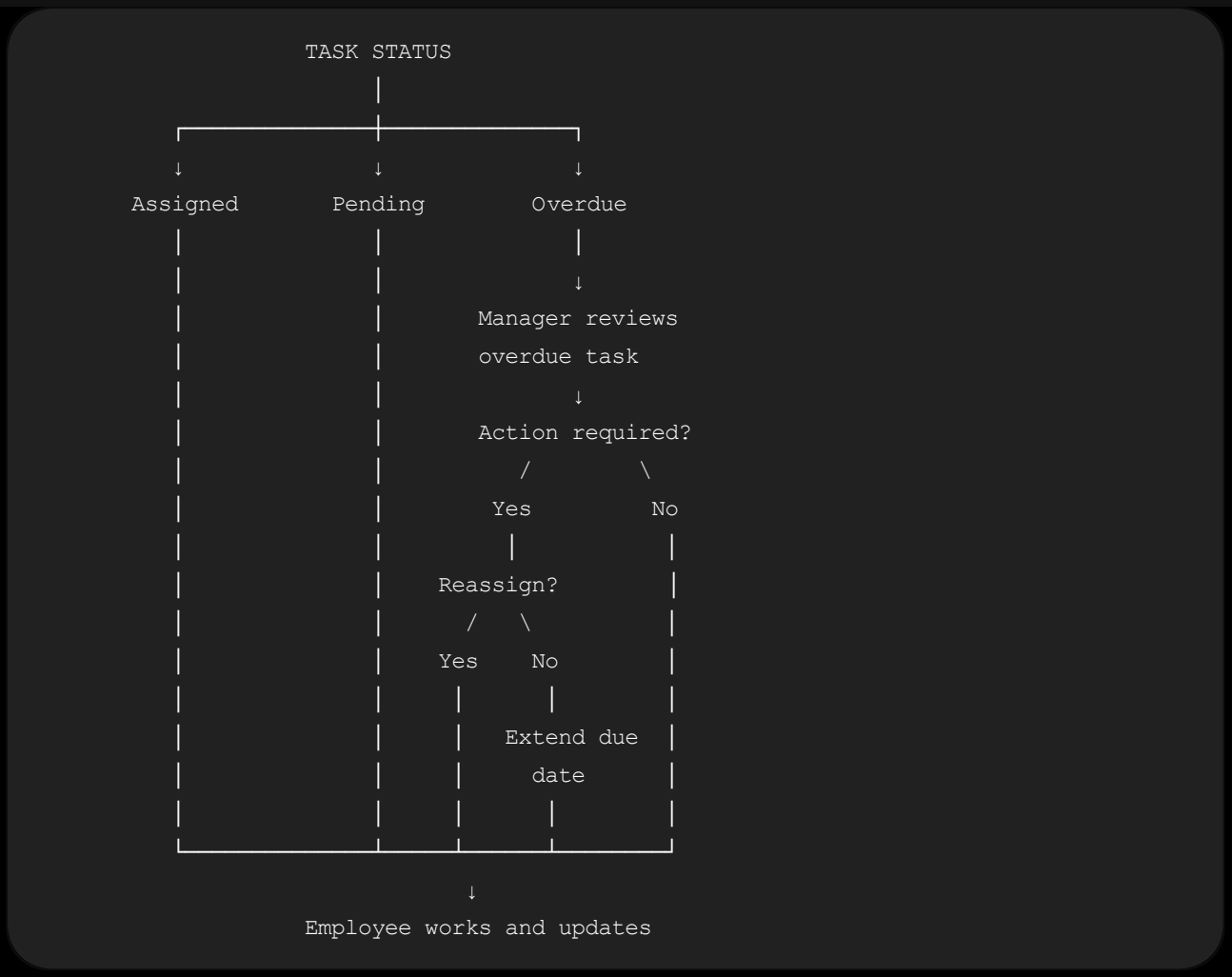
I would change it to this



But I **wouldn't actually put all of that into your diagram** because it will become spaghetti again.

Better for *your* existing diagram

Keep your current structure and modify **only the Overdue branch**:



Then your existing:



can remain exactly where it is.

One thing I'd change in the terminology

Instead of:

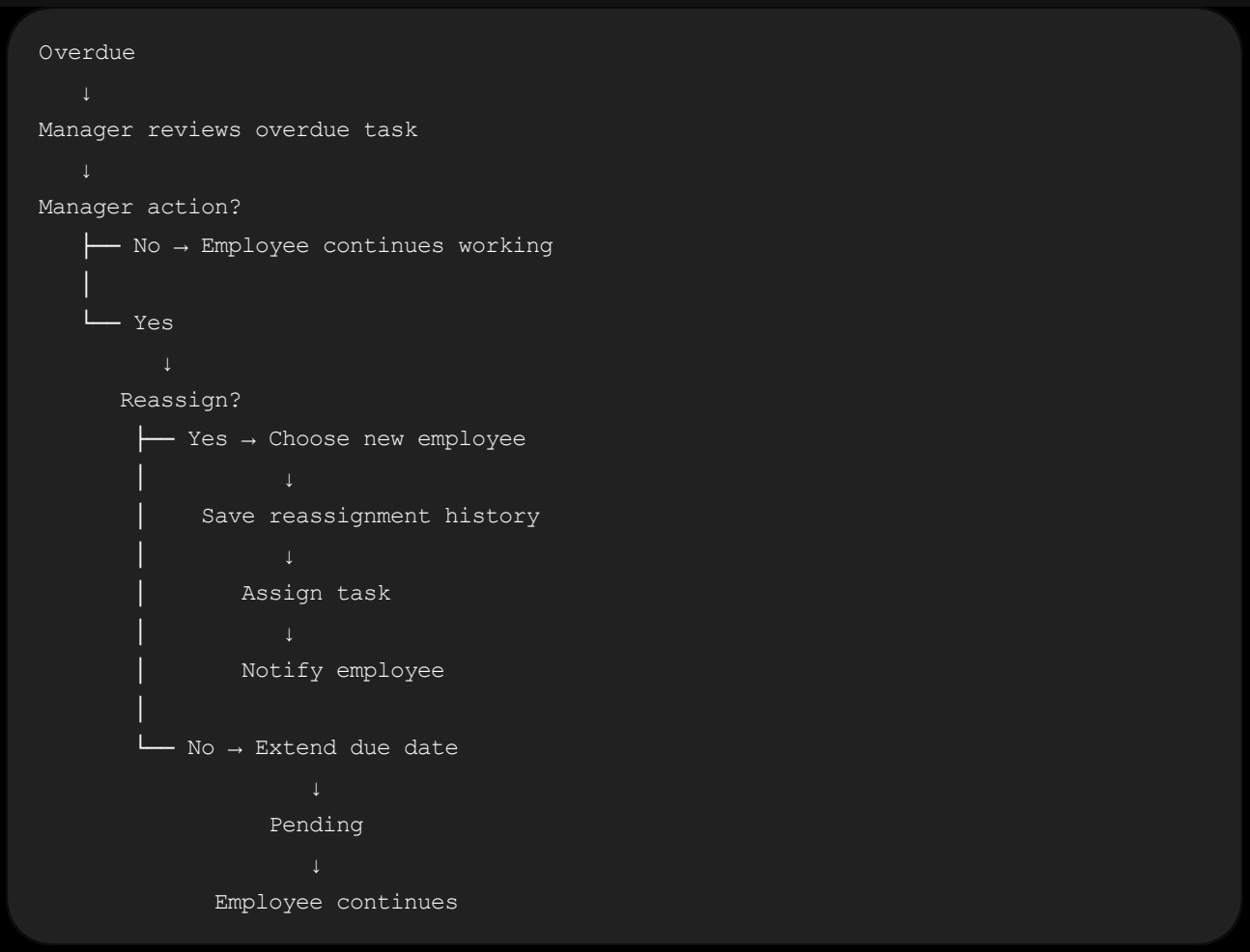
`Action required?`

I'd actually use:

`Manager action?`

because the question is specifically whether the manager needs to intervene.

Then:



This gives your statuses much more meaning

Status	What causes it	What happens
Assigned	Task assigned	Employee receives notification
Pending	Task is active/not finished	Employee continues work
Overdue	Due date is exceeded	**Manager intervention is triggered**
Reassigned	Manager transfers ownership	New employee is assigned
Completed	Manager approves	Task is closed

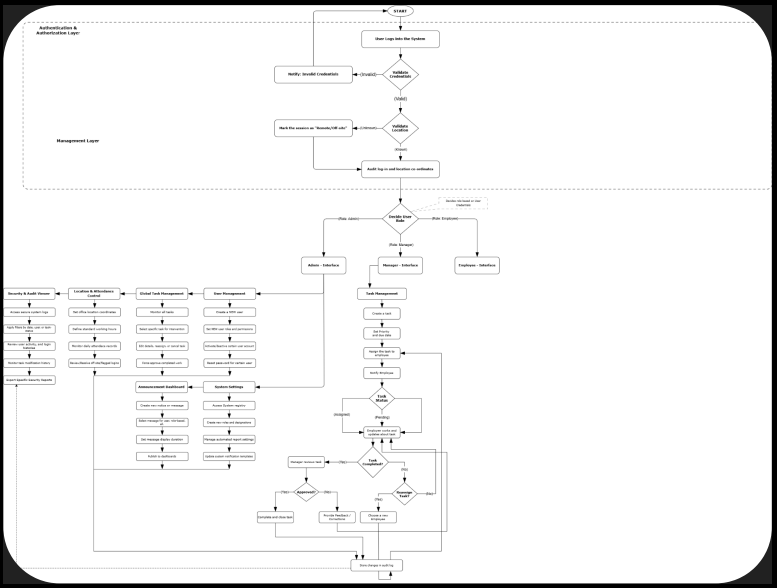
And importantly:

Overdue ≠ Reassigned.

An overdue task **might** be reassigned, but it doesn't have to be. The manager could simply extend the deadline or allow the employee to continue.

Also, I would **remove the** `Reassigned` **box after** `Complete and close task` from your current image. That was the mistake we discussed earlier. `Reassigned` belongs inside the task lifecycle, not after closure.

Your overall diagram is actually getting quite solid now. The **Overdue → Manager intervention** logic is the missing managerial intelligence.



Update the overdue section for this image, do you suggest a seperate branch?

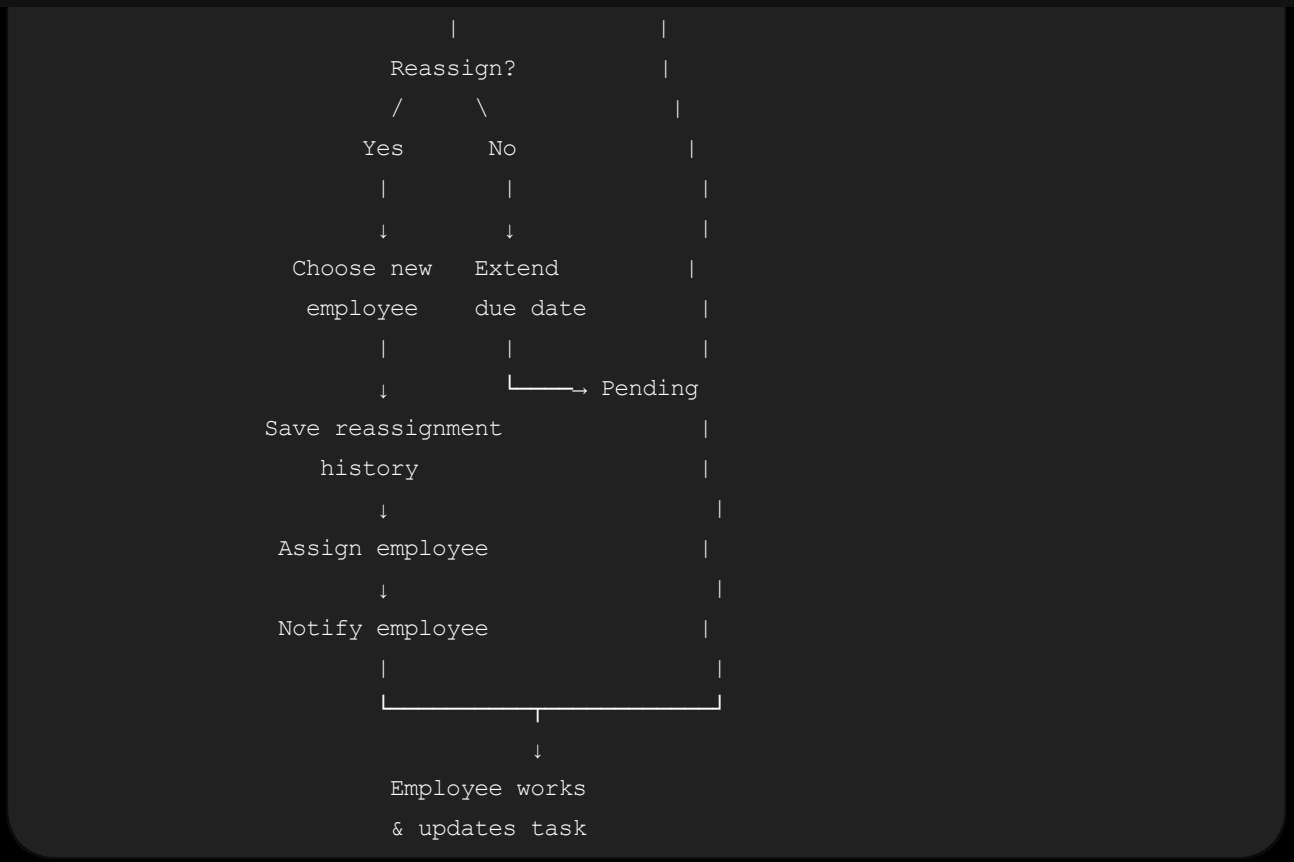
Yes. Looking at **this complete diagram**, I would definitely make **Overdue a separate branch**.

The reason is that your current `Assigned → Employee works` and `Pending → Employee works` are **normal workflow paths**. **Overdue is an exception/escalation condition**, so it deserves its own managerial branch rather than simply joining `Employee works and updates about task`.

What I recommend

Keep your central structure:

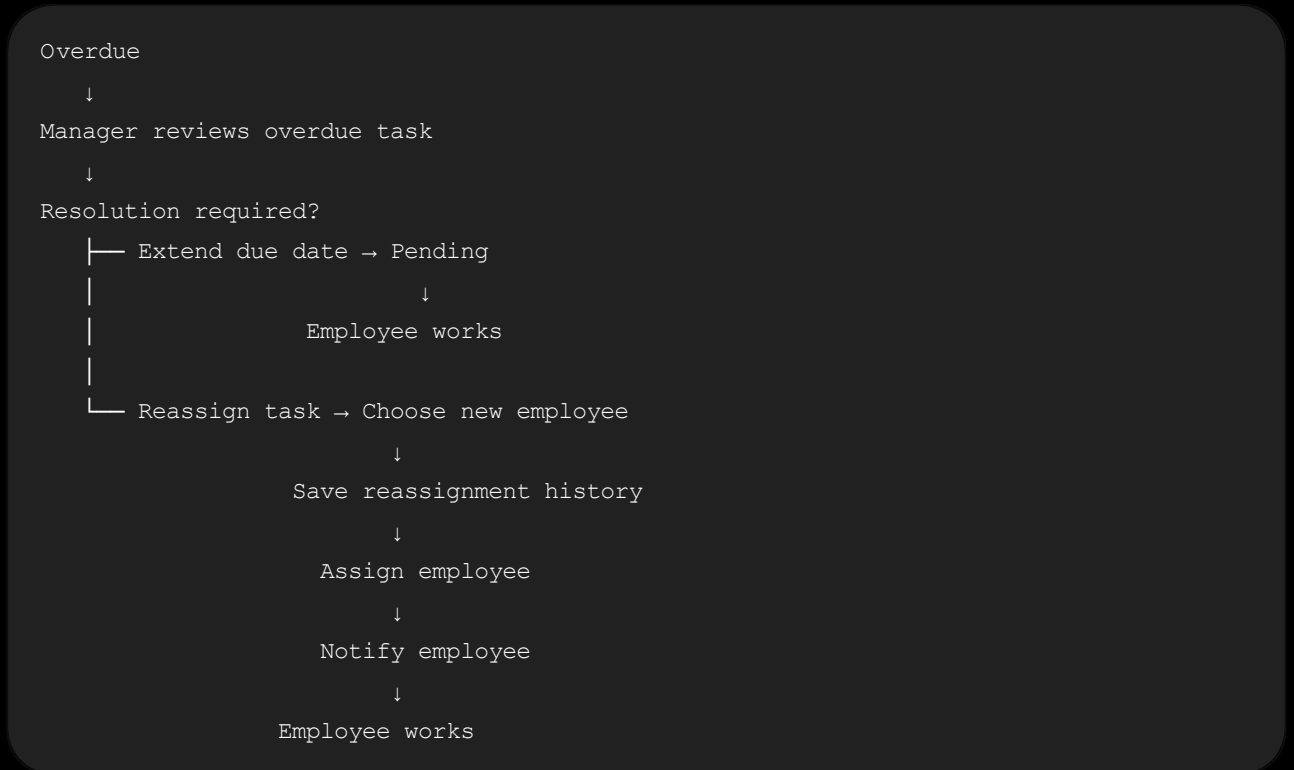
```
                TASK STATUS
              /      |      \
             ↓       ↓       ↓
      Assigned   Pending   Overdue
          \        /         |
           \      /          ↓
            \    /     Manager reviews
             ↓           overdue task
    Employee works           |
    & updates          Manager action?
                          /        \
                        Yes         No
```



But I'd make one small architectural change

I **wouldn't use** `Manager action? → Reassign?` if you want the diagram to look sophisticated but clean.

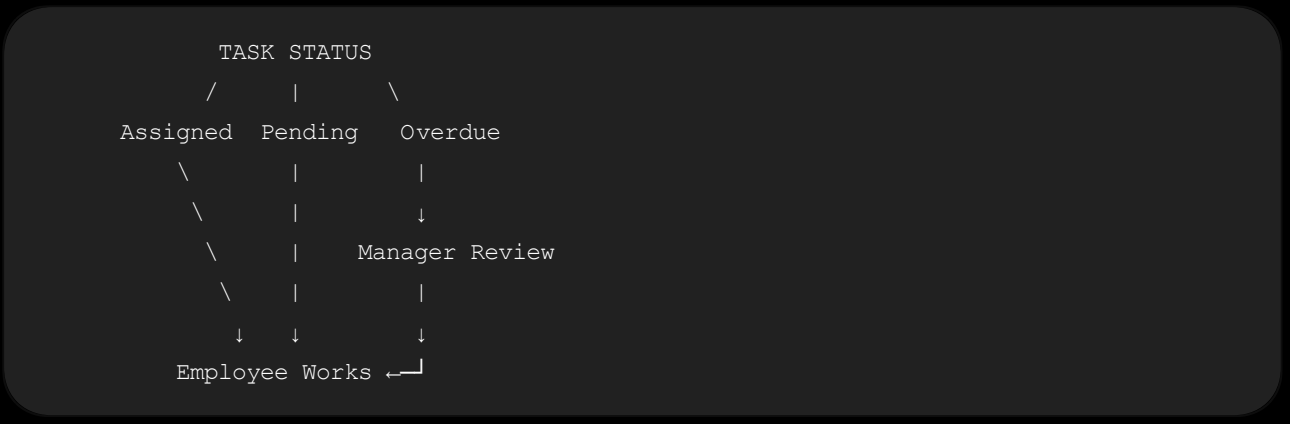
Instead:



This is cleaner because the manager's two meaningful actions are immediately visible.

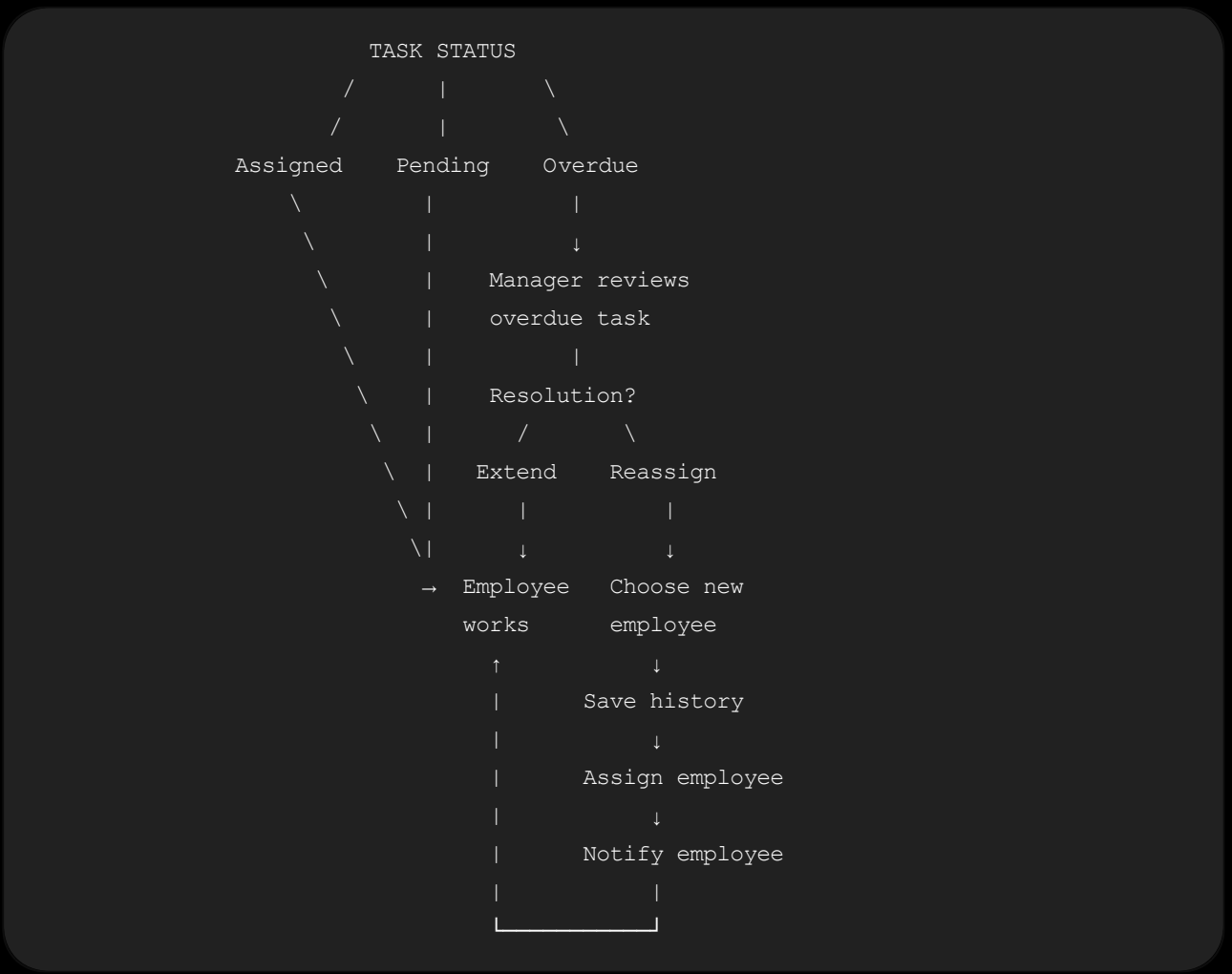
Where I'd put it in YOUR image

Your current `TASK STATUS` is here:



I'd place **Overdue on the right**, then let its branch travel downward on the right-hand side.

Your diagram already has the **reassignment branch on the lower-right**, so this actually fits nicely:



One more important thing

I would **not make** `Overdue → Employee works` **directly**.

That's the part that currently makes the workflow look like the system doesn't care about deadlines.

Instead:

Overdue → Manager reviews overdue task → resolution → back into workflow.

That gives your Manager interface an actual **management/intervention role**, rather than making it look like the manager only creates, assigns, and approves tasks.

And **don't add** `Reassigned` **after** `Complete and close task` in this version. Keep reassignment inside this exception path as an event/transition.

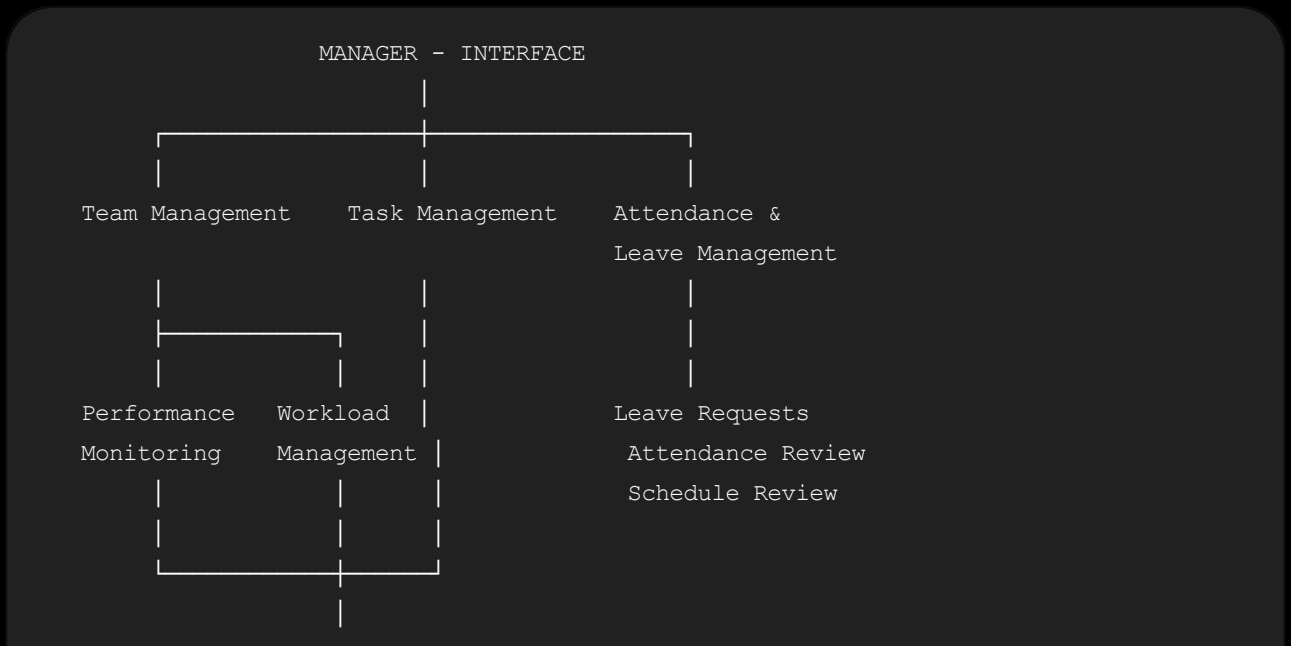
Apart from these task managements by the manager, what the other things he/she can do list such managemental aspects okay?

Yes. If we look at the **Manager as a real managerial role**, Task Management should be only **one part** of the Manager Interface.

The Manager shouldn't have Admin-level controls like creating users or changing system settings. Instead, the Manager should primarily **manage the team, monitor work, make decisions, approve things, and handle exceptions**.

For your flowchart, I would suggest these major Manager-side management aspects:

Manager Interface



```
Approval Management
        |
Reports & Analytics
        |
Communication
        |
Escalation Management
```

But for your actual diagram, I'd make it more structured.

---

1. 👥 Team / Employee Management

This is different from **Admin → User Management**.

The Admin manages the **accounts**.

The Manager manages the **people working under them**.

Manager can:

```
Team Management
      ↓
View team members
      ↓
View employee profiles
      ↓
View assigned responsibilities
      ↓
Monitor individual workload
      ↓
Track employee work progress
```

You could include:

- View team members
- View employee role/designation
- View current assignments
- View workload
- View availability
- View employee work history

Don't give the Manager "Create User / Delete User" because that belongs to Admin.

---

2. 📊 Performance Monitoring

This is a very good managerial component.

```
Performance Monitoring
        ↓
Monitor task completion
        ↓
Review work quality
        ↓
Track deadlines
        ↓
Identify performance issues
        ↓
Provide feedback
```

You can make it more sophisticated:

- Task completion rate
- On-time completion
- Overdue tasks
- Reassignment frequency
- Work quality
- Employee progress
- Manager feedback

This also connects nicely to your **Automated Reporting** section.

---

3. ⚖️ Workload Management

This is one I'd **definitely include**.

The Manager should be able to see whether one employee has too much work while another has very little.

```
Workload Management
        ↓
View employee workload
        ↓
Identify overloaded employees
        ↓
Check employee availability
        ↓
Balance task distribution
        ↓
Reassign tasks if required
```

This actually gives your **Task Management → Reassign Task** decision a reason.

The Manager isn't randomly reassigning tasks; they can do it because of:

- workload imbalance
- employee unavailability
- skill mismatch
- deadline risk

---

4. 🕘 Attendance & Leave Management

Since your Admin already has **Location & Attendance Control**, don't duplicate the technical attendance control.

Admin:

> Controls attendance system/configuration.

Manager:

> Reviews attendance of **their team** and acts on it.

```
Attendance & Leave
        ↓
View team attendance
        ↓
Review late / absent records
        ↓
View leave requests
        ↓
Approve / Reject leave
        ↓
Adjust workload if required
```

This is a very natural Manager responsibility.

---

5. ✅ Approval Management

This is another major managerial function.

Instead of making approvals scattered everywhere, create a central branch:

```
Approval Management
        ↓
Review pending requests
        ↓
     Request Type?
      /    |     \
```

```
   Leave   Task    Other
     |       |       |
  Approve   Review  Review
  / Reject /Reject /Reject
```

Depending on your system, "Other" could include:

- schedule changes
- task deadline extensions
- work submissions
- resource requests

---

6. 📅 Schedule / Resource Management

If your management system involves employees working according to schedules, this is useful.

```
Schedule Management
        ↓
View team schedule
        ↓
Identify availability
        ↓
Allocate work
        ↓
Handle schedule conflicts
        ↓
Update assignments
```

If your project doesn't have scheduling, **don't force this into the diagram**.

---

7. 📈 Reports & Analytics

This should be one of the Manager's strongest sections.

```
Reports & Analytics
        ↓
Task Performance
        ↓
Employee Performance
        ↓
Overdue Tasks
        ↓
Workload Analysis
        ↓
```

```
Attendance Summary
        ↓
Generate Manager Report
```

And this can feed your existing:

Automated Reporting

system.

You could have:

```
Manager Reports
      ↓
Generate / View Reports
      ↓
Automated Reporting
```

---

8. 📢 Team Communication

A Manager should also be able to communicate with their team.

```
Team Communication
        ↓
Send team announcement
        ↓
Send employee-specific message
        ↓
Share task instructions
        ↓
Send deadline reminders
        ↓
Receive employee updates
```

You already have **Announcement Dashboard** under Admin.

So again, distinguish them:

Admin → Organization-wide announcements

Manager → Team-specific communication

---

9. 🚨 Escalation Management

This one would make your system look **much more mature**.

Especially because you've already introduced **Overdue**.

```
Escalation Management
        ↓
Identify issue
        ↓
Assess severity
        ↓
Manager intervention
        ↓
Resolve / Reassign / Escalate
        ↓
Record action
```

For example:

```
Overdue Task
     ↓
Manager Review
     ↓
Can manager resolve?
    /       \
  Yes        No
   ↓          ↓
Resolve    Escalate
              ↓
         Higher Authority
```

This creates a natural relationship between:

Task Management → Overdue → Manager Intervention → Escalation Management

---

10. 📝 Feedback & Evaluation

You already have:

> `Provide feedback / corrections`

inside Task Management.

But at the broader Manager level, you could have:

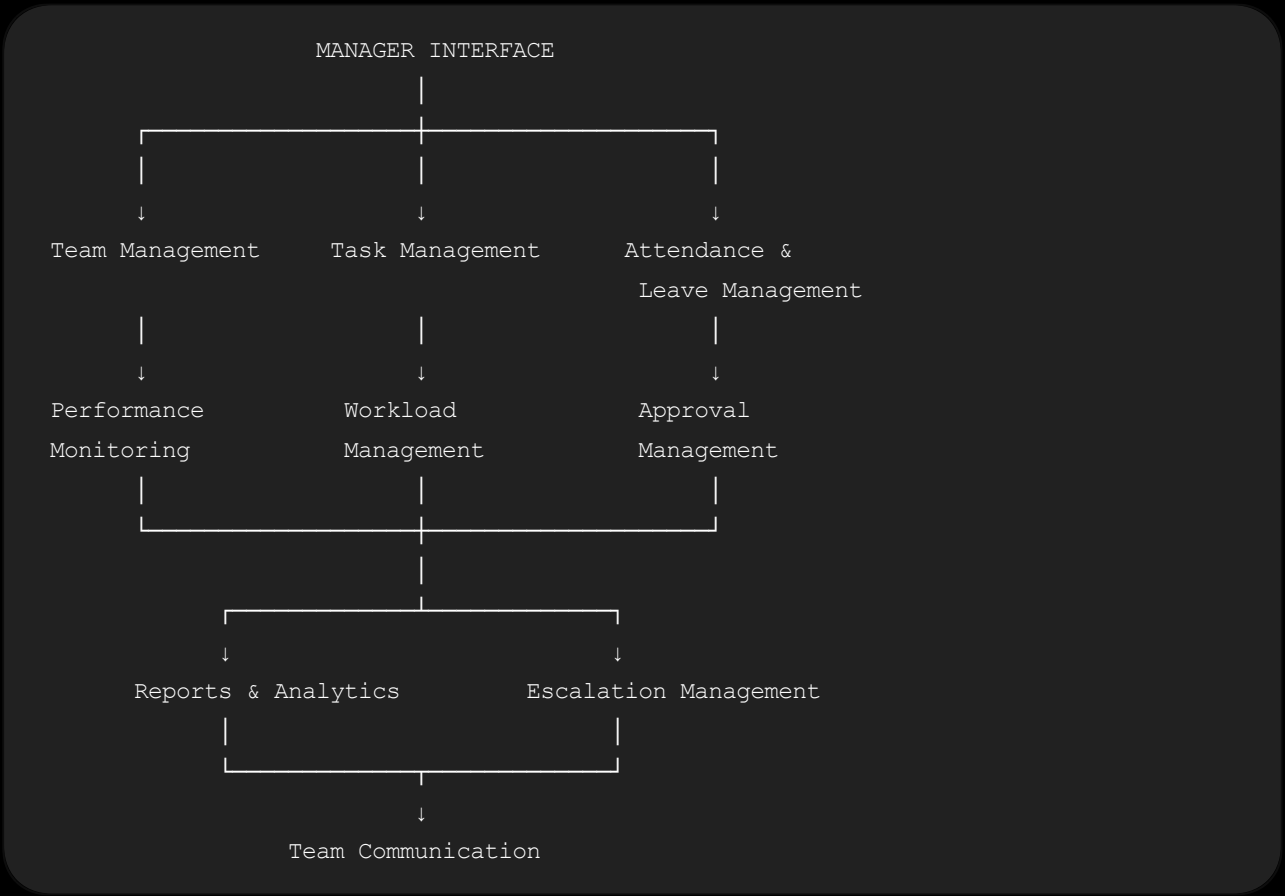
```
Employee Feedback
       ↓
Review employee work
       ↓
Provide feedback
```

```
        ↓
Record evaluation
        ↓
Monitor improvement
```

This is different from simply rejecting a task.

So I'd recommend these Manager branches

For your **final chairperson-facing diagram**, I'd use around **7 major boxes**, rather than trying to show every possible function:



My priority ranking for your project

If you don't want to make the Manager section enormous, I'd prioritize:

Priority	Manager aspect	Why
★★★★★	**Task Management**	Core managerial responsibility
★★★★★	**Team Management**	Defines who the Manager supervises
★★★★★	**Performance Monitoring**	Measures team effectiveness
★★★★★	**Workload Management**	Supports assignment/reassignment decisions

Priority	Manager aspect	Why
★★★★	**Attendance & Leave**	Common managerial responsibility
★★★★	**Approval Management**	Shows decision-making authority
★★★★	**Reports & Analytics**	Supports managerial decisions
★★★★	**Escalation Management**	Handles overdue/problem cases
★★★	**Team Communication**	Supports coordination
★★★	**Schedule Management**	Useful if scheduling exists in your system

One architectural point: don't make the Manager interface look like a smaller Admin interface. The distinction should be obvious:

> **Admin = controls the system and organization.**
> **Manager = controls team operations and decisions.**
> **Employee = performs the assigned work.**

That separation will make your whole three-role architecture much stronger when you present it to the chairperson.

Based on the configurations above mentioned lets work on the employee dashboard now okay?

Absolutely. Let's build the **Employee Dashboard** using the same architecture we've established for Admin and Manager.

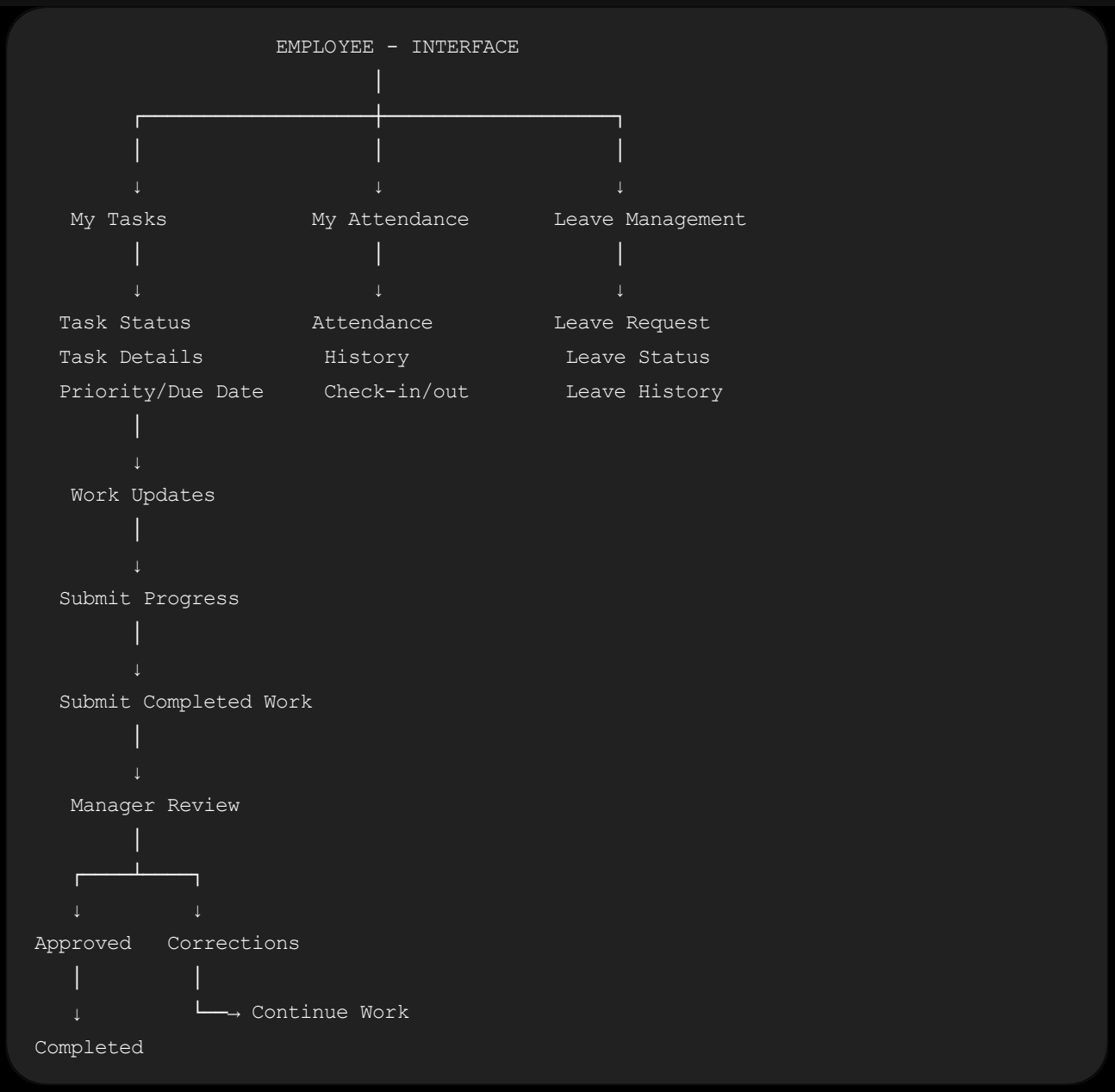
The key is to maintain the hierarchy:

> **Admin → controls the system**
> **Manager → manages the team/work**
> **Employee → performs and reports the work**

So the Employee dashboard should **not** contain management functions such as creating users, assigning tasks to others, or changing system settings.

Employee Interface — major areas

I suggest we start with these:



But I think we can make this **much more sophisticated** while still keeping it clean.

Employee-specific management aspects

1. **My Tasks**
   - View assigned tasks
   - View priority
   - View due date
   - View task status
   - View task instructions
2. **Task Progress / Work Updates**
   - Start task
   - Update progress
   - Add work remarks
   - Submit completed work
   - Respond to manager feedback
3. **Task Status**

- Assigned
- Pending
- Overdue
- Completed

4. **Attendance**
    - Check-in / check-out
    - View attendance
    - View attendance history
5. **Leave Management**
    - Apply for leave
    - View pending request
    - View approved/rejected status
    - View leave history
6. **Notifications**
    - New task assigned
    - Task reassigned
    - Deadline reminder
    - Manager feedback
    - Approval/rejection notifications
7. **Profile / Personal Information**
    - View own profile
    - View designation/department
    - Contact information
    - Account information
8. **Performance / Work Summary**
    - Completed tasks
    - Pending tasks
    - Overdue tasks
    - Work history
    - Manager feedback

Most importantly

The Employee's task flow should connect naturally with the **Manager's Task Management** that we just designed:

```
MANAGER
Create Task
     ↓
Set Priority + Due Date
     ↓
Assign Employee
```

```
      ↓
Notify Employee
      ↓
_____________________________

       EMPLOYEE
_____________________________

      ↓
View Assigned Task
      ↓
Acknowledge / Start Task
      ↓
Work on Task
      ↓
Update Progress
      ↓
Task Completed?
   ├── No → Continue Work
   │
   └── Yes
         ↓
   Submit Completed Work
         ↓
   Manager Review
         ↓
     Approved?
      ├── No → Feedback / Corrections
      │                ↓
      │          Continue Work
      │
      └── Yes → Completed
```

This gives us a **single continuous workflow across Manager and Employee**, rather than designing the Employee dashboard independently.

Since you want the final diagram to be **complex enough technically but simple enough for the chairperson**, I'd build the Employee section in the same style as your current full architecture, starting with **Employee Interface → major branches → detailed flow**.

We can work through **Employee Dashboard → Task Management first**, then Attendance, Leave, Notifications, Profile, and Performance one by one.

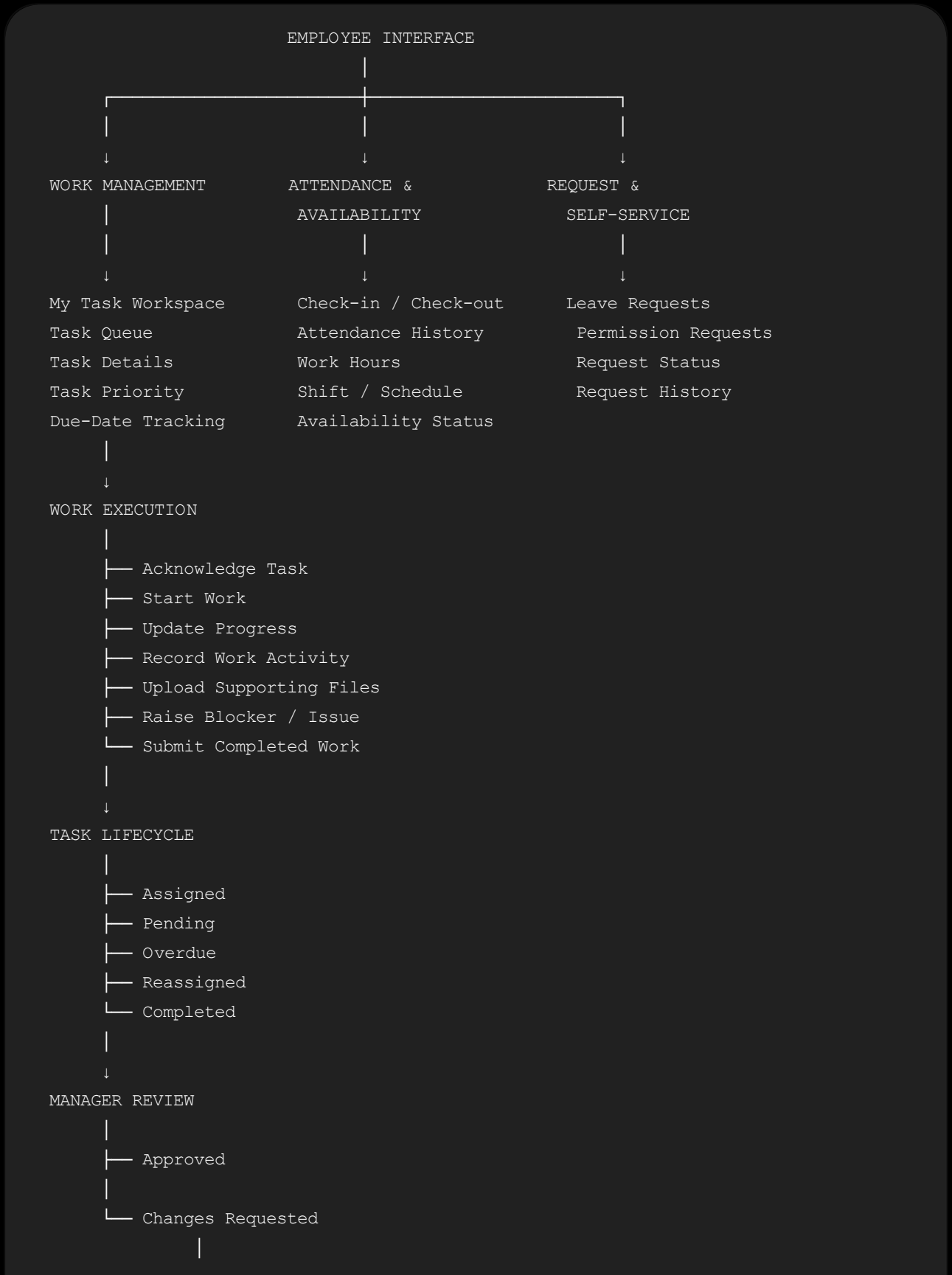
Sounding too basic mate, can you advance it for the industrial standards

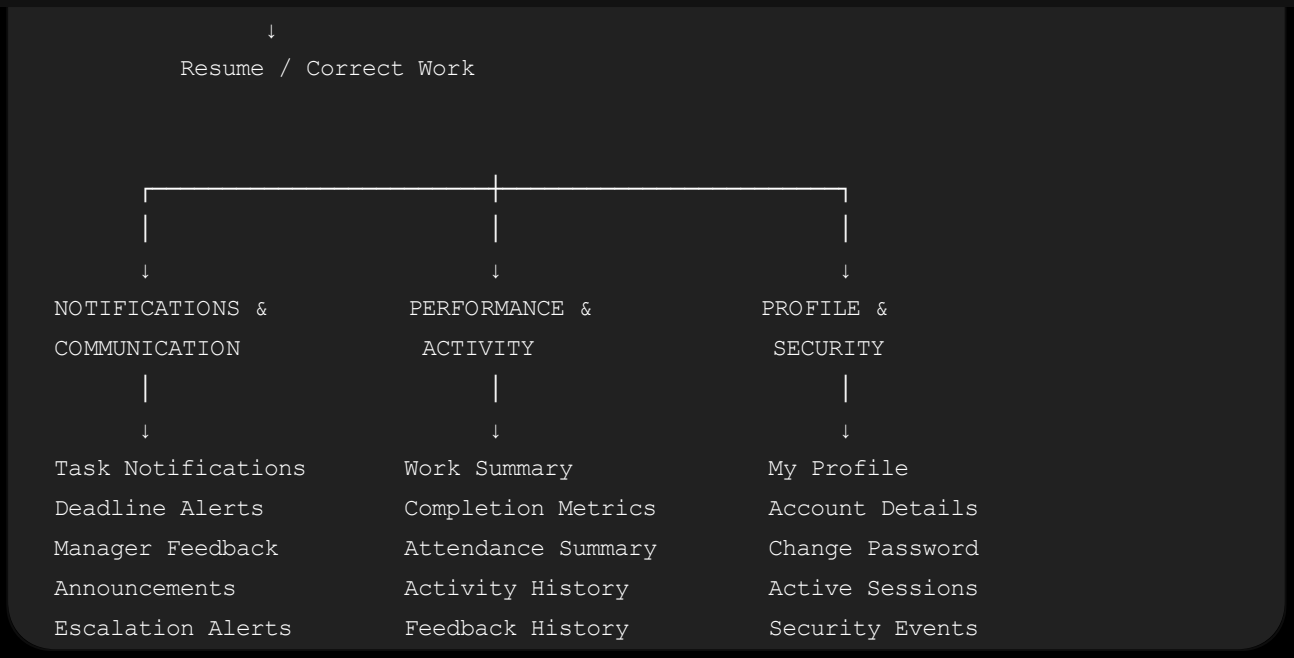
Exactly, mate. The earlier version is more like a **college-level CRUD dashboard**. For an industrial-standard system, the Employee interface should represent **work execution +**

self-service + compliance + exception handling + traceability, while still respecting the boundary that the Manager manages people and tasks.

Your existing Admin and Manager architecture gives us a good foundation, so I would upgrade the Employee side like this.

Employee Interface — Industrial-Level Architecture



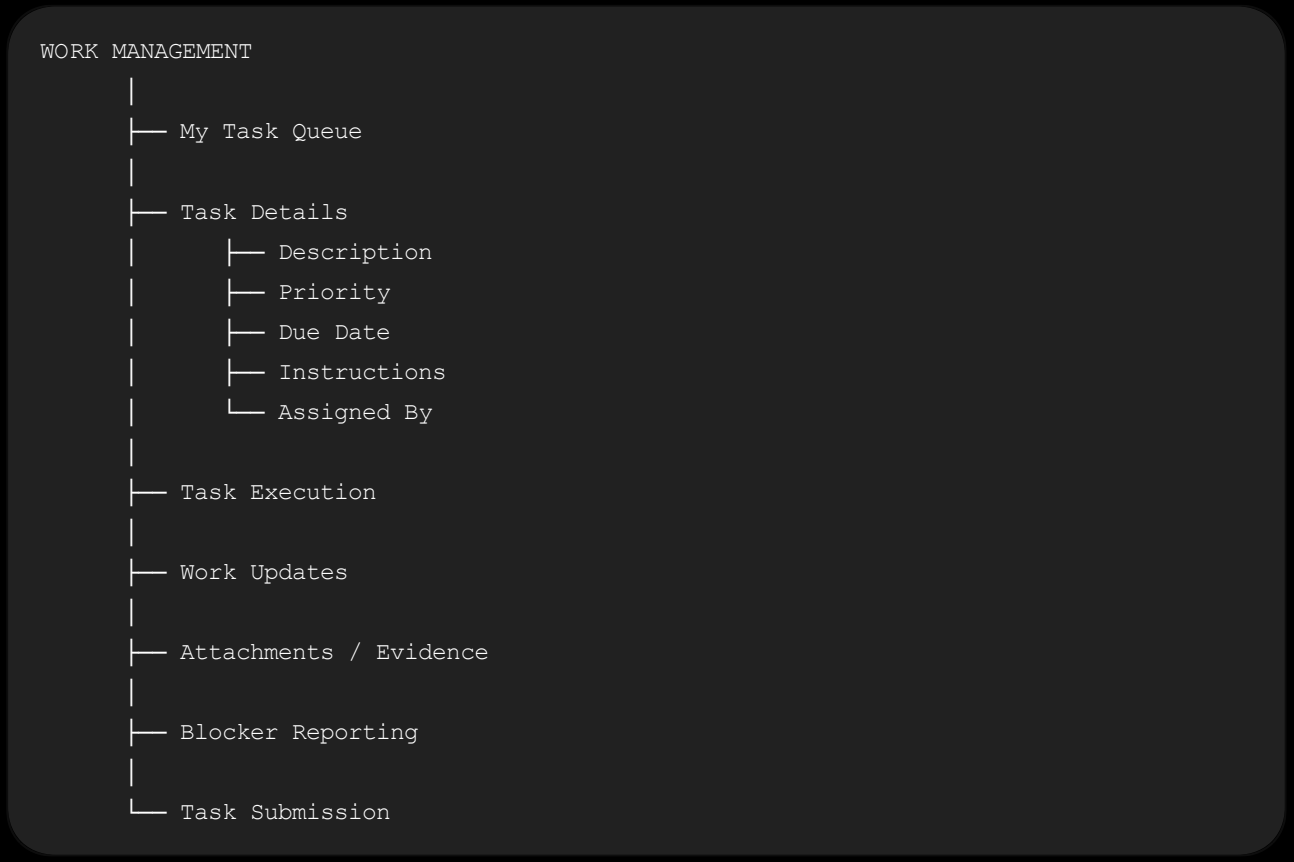


This is much closer to an actual **enterprise employee portal**.

---

1. Work Management should be the CORE

This should be the biggest branch because the employee's primary purpose is to execute assigned work.



Notice the difference from a basic system:

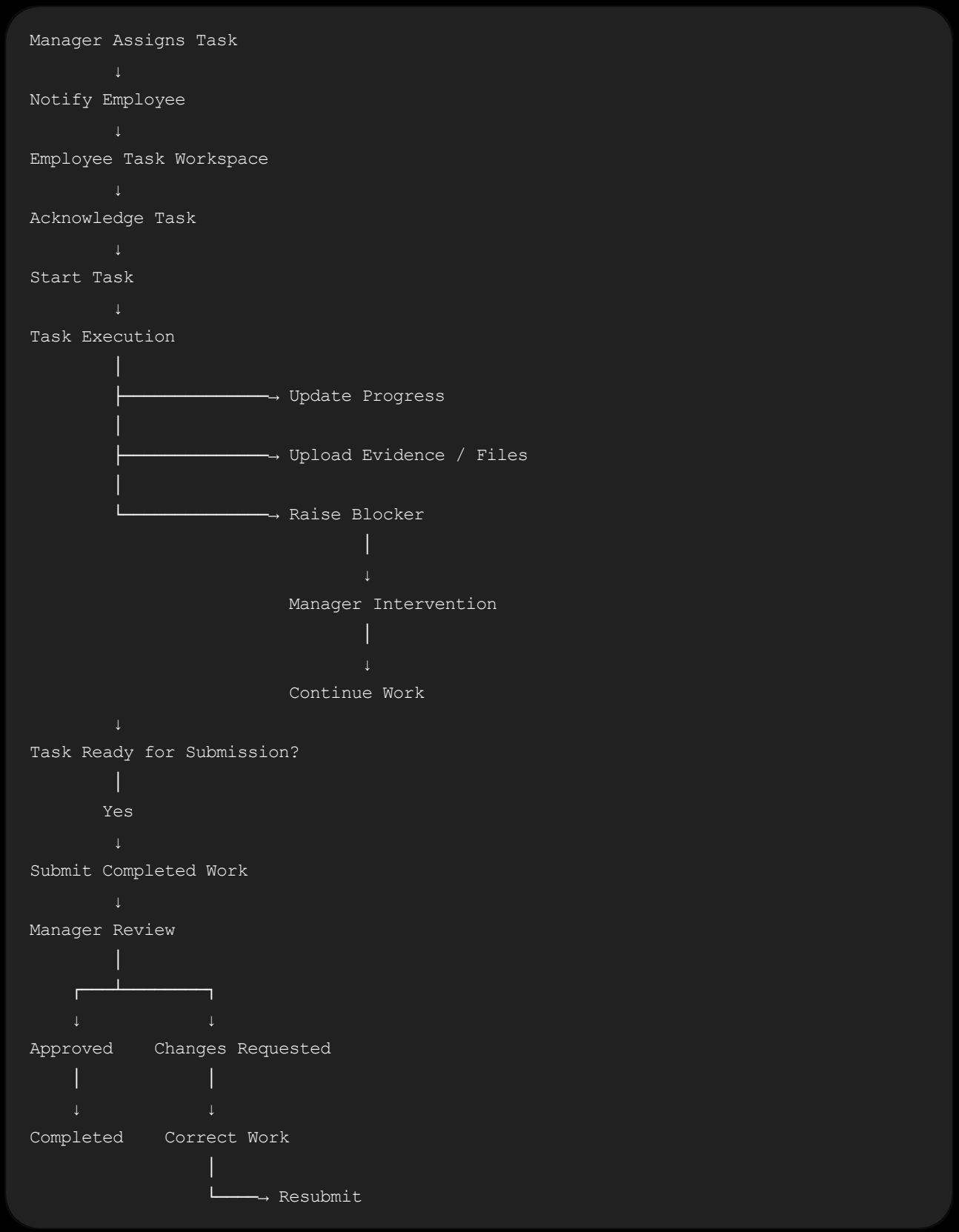
Employee doesn't simply "view task → complete task".

They have a **work execution workspace**.

---

2. Make the Employee Task Workflow much more realistic

This is where your Manager workflow connects directly to Employee.



That **Blocker / Issue** branch is particularly important.

Industrial systems don't assume employees can always finish a task normally.

An employee may encounter:

- dependency not available
- required resource unavailable
- technical issue
- clarification required
- another team dependency
- access problem

So:

> **Blocked ≠ Incomplete ≠ Reassigned**

That's an important architectural distinction.

---

3. Add a proper "Work Activity" layer

Instead of merely:

> "Employee works and updates about task"

I'd use:

Record Work Activity

```
Record Work Activity
        |
        ├── Progress Update
        ├── Work Notes
        ├── Time / Effort Entry
        ├── Supporting Document
        └── Evidence / Attachment
```

This gives management a traceable record of **what the employee actually did**.

It also feeds:

Manager → Performance Monitoring

and

Admin → Security & Audit Viewer

without mixing those responsibilities into the Employee interface.

---

4. Attendance should become "Attendance & Availability"

A simple:

> Check-in → Check-out

is too basic.

Use:

```
ATTENDANCE & AVAILABILITY
          |
          ├── Check-in / Check-out
          ├── Attendance Status
          ├── Work Hours
          ├── Attendance History
          ├── Shift / Schedule
          └── Availability Status
```

And because your authentication layer already validates location, you have a nice connection:

```
Login
 ↓
Location Validation
 ↓
Attendance / Session Audit
 ↓
Employee Dashboard
```

That makes the architecture internally consistent.

---

5. Requests & Self-Service

This is an important enterprise concept.

Employee shouldn't have to contact the Manager manually for everything.

```
REQUEST & SELF-SERVICE
          |
          ├── Leave Request
```

```
        ├── Permission Request
        ├── Schedule Request
        ├── Task Clarification
        ├── Resource Request
        └── Request History
                ↓
          Manager Approval
```

This creates another clean connection:

Employee → Request → Manager → Approve/Reject

rather than giving Employee management authority.

---

6. Notifications should be event-driven

Don't make it just a notification list.

```
NOTIFICATIONS
      |
      ├── New Task Assigned
      ├── Task Reassigned
      ├── Due-Date Reminder
      ├── Task Overdue
      ├── Manager Feedback
      ├── Submission Approved
      ├── Changes Requested
      ├── Leave Decision
      └── System Announcement
```

And architecturally:

```
SYSTEM EVENT
     ↓
Notification Engine
     ↓
Employee Notification
```

That's much more realistic than hard-coding notifications inside every workflow.

---

7. Performance & Activity

This should **not become an employee HR-control module**.

It should be a **self-view**.

```
PERFORMANCE & ACTIVITY
          |
          ├── Completed Tasks
          ├── Pending Tasks
          ├── Overdue Tasks
          ├── On-Time Completion
          ├── Work Activity
          ├── Manager Feedback
          └── Work Summary
```

The employee can **see their own metrics**, while the Manager can see the broader team metrics.

Same underlying data, different authorization scope.

That's an important enterprise design principle:

```
Same Data
   ↓
Role-Based Access
   ↓
Employee → Own Data
Manager  → Team Data
Admin    → Organization Data
```

---

8. Profile & Security

I'd also make this more advanced than just "My Profile".

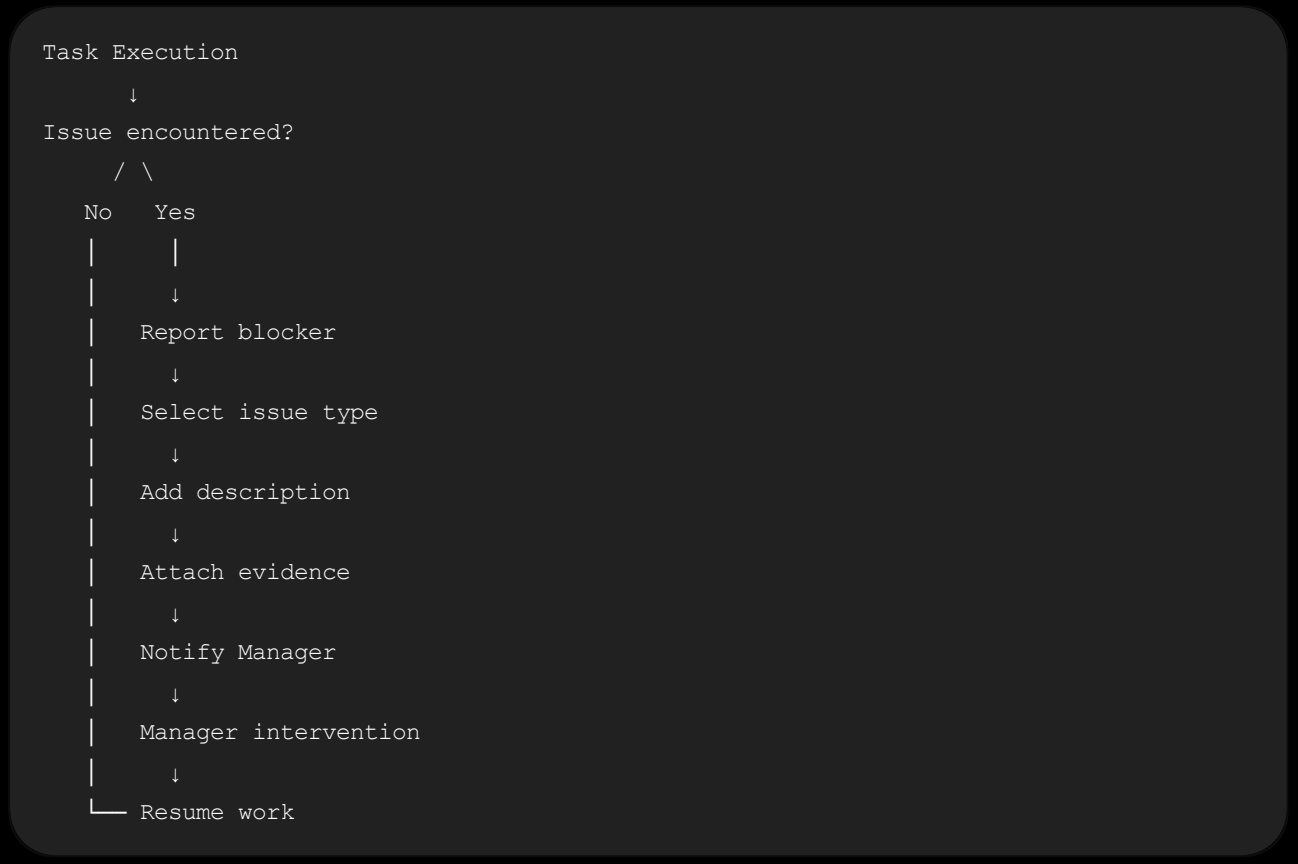
```
PROFILE & SECURITY
          |
          ├── Personal Information
          ├── Employment Details
          ├── Account Information
          ├── Change Password
          ├── Active Sessions
          └── Security Activity
```

Since your authentication layer already has authentication, authorization, location validation and audit logging, this fits naturally into the same architecture.

---

9. One branch I strongly recommend: "Issue / Blocker Management"

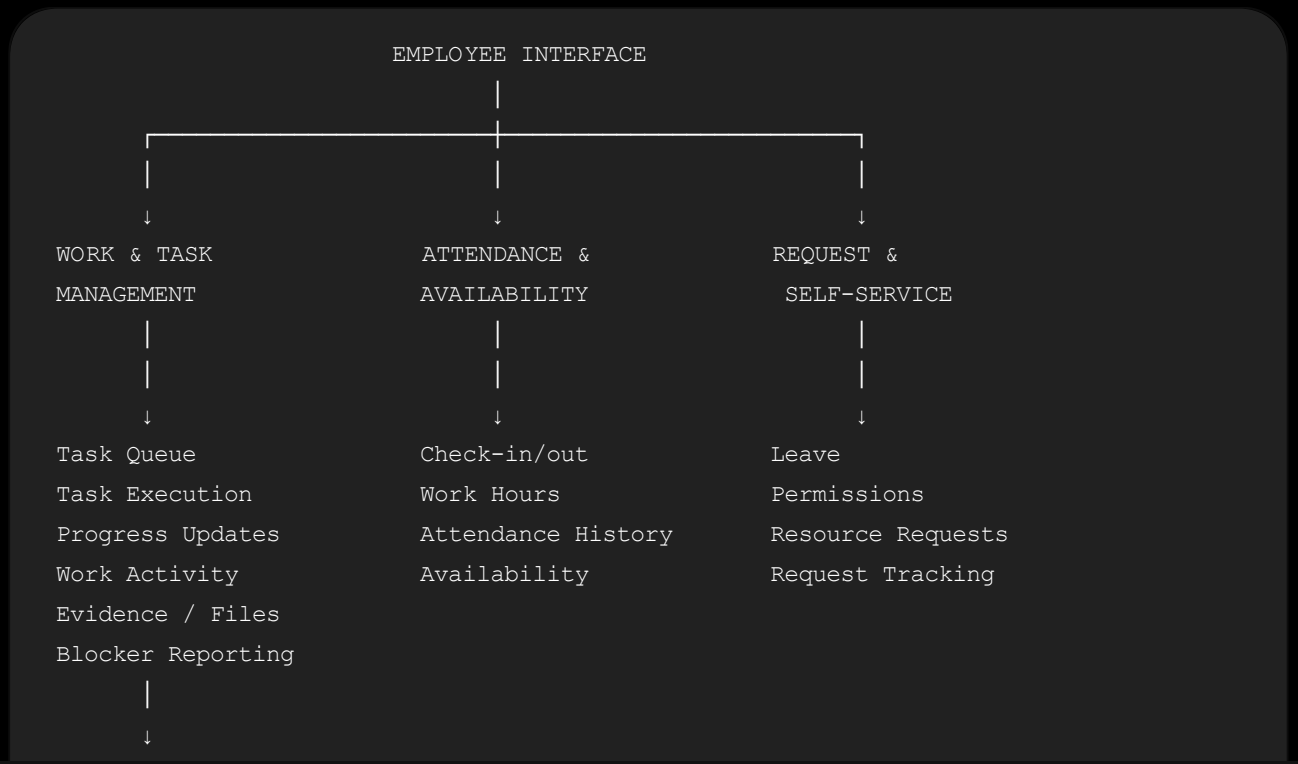
This is the piece that will make your Employee workflow feel **less artificial**.

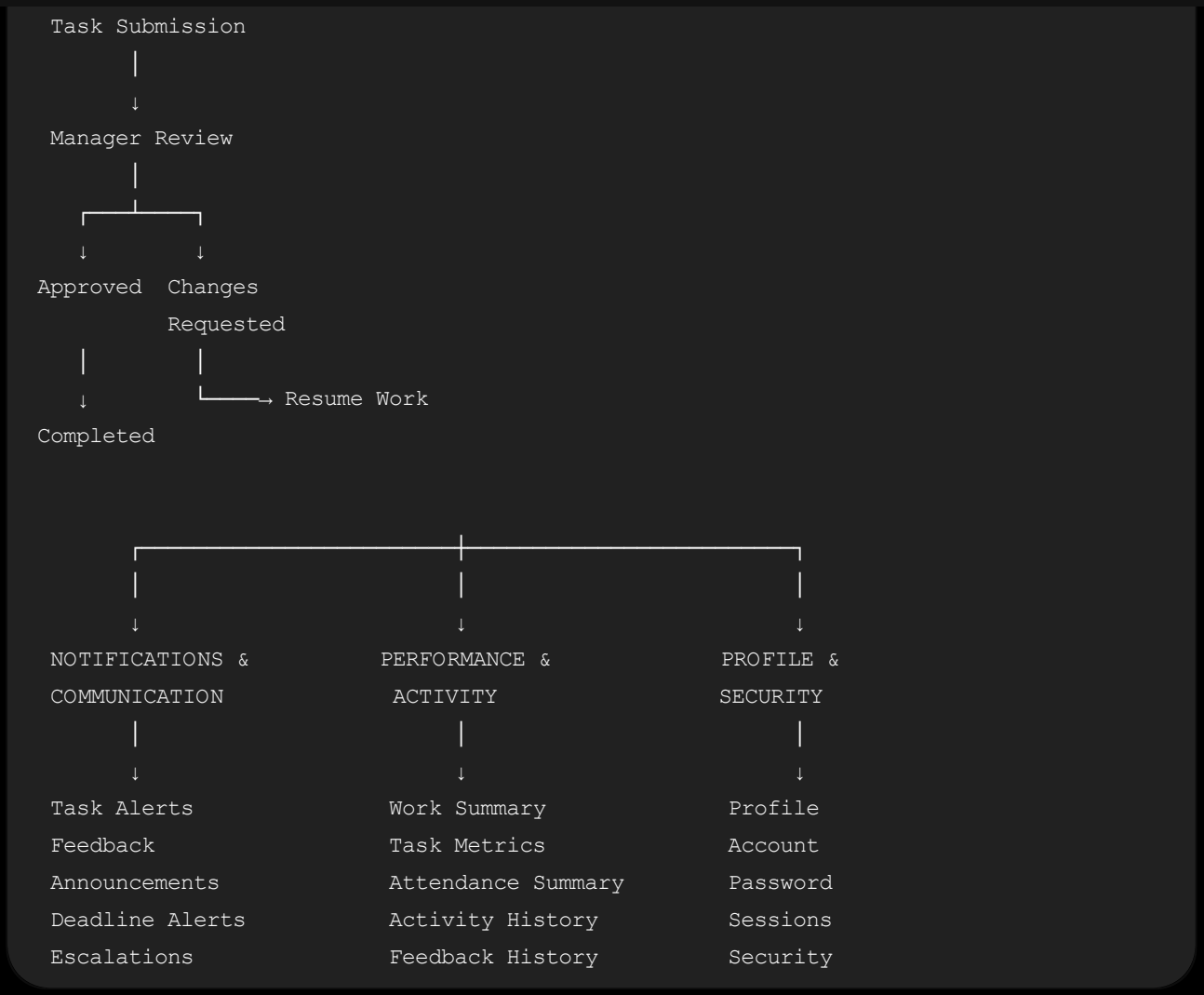


Now the Manager's **Escalation Management** has an actual input from the Employee.

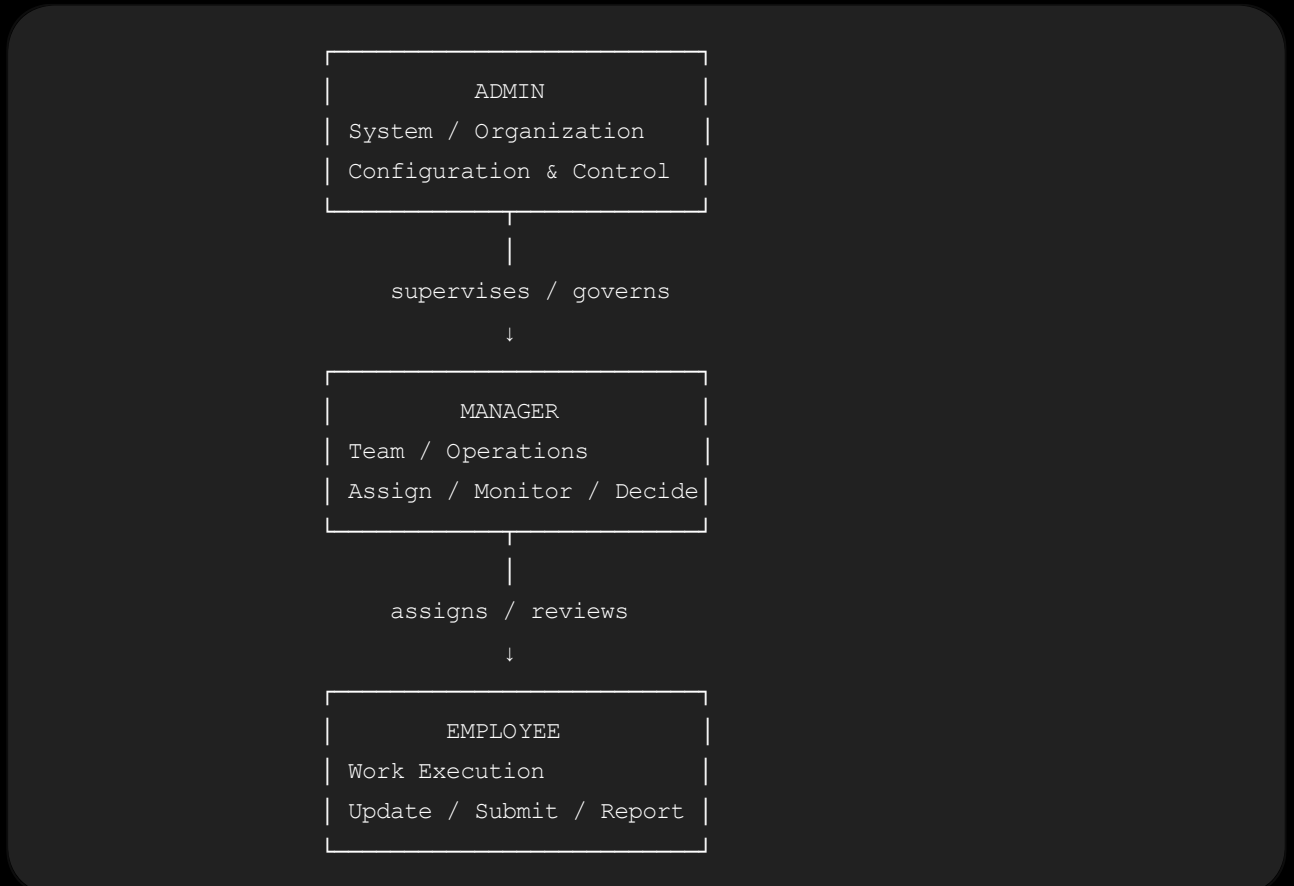
The complete Employee architecture I'd use

For the final chairperson-facing diagram, I'd reduce the detailed internals into these **six major modules**:





And the three-role boundary becomes very clear



That hierarchy is what I would aim for in the final system.

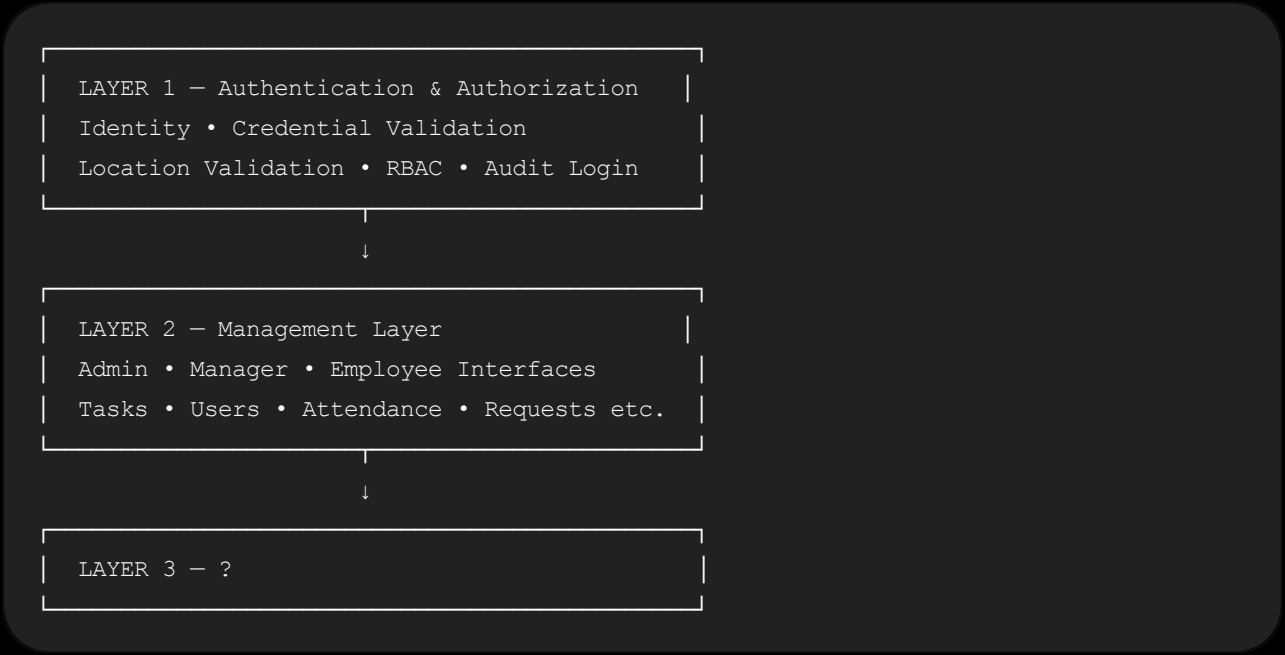
Admin controls → Manager manages → Employee executes → information flows upward → decisions flow downward.

That makes the whole system feel like an actual **organizational management platform**, rather than three separate dashboards.

You know the project that we are dealing with right, apart from authentication and authorization layer, and management layer, what is the final layer we can come up with?

Yes, mate. Based on the architecture we've been building, I think there is a **very natural third layer** that sits above the individual Admin/Manager/Employee operations.

So far we have:



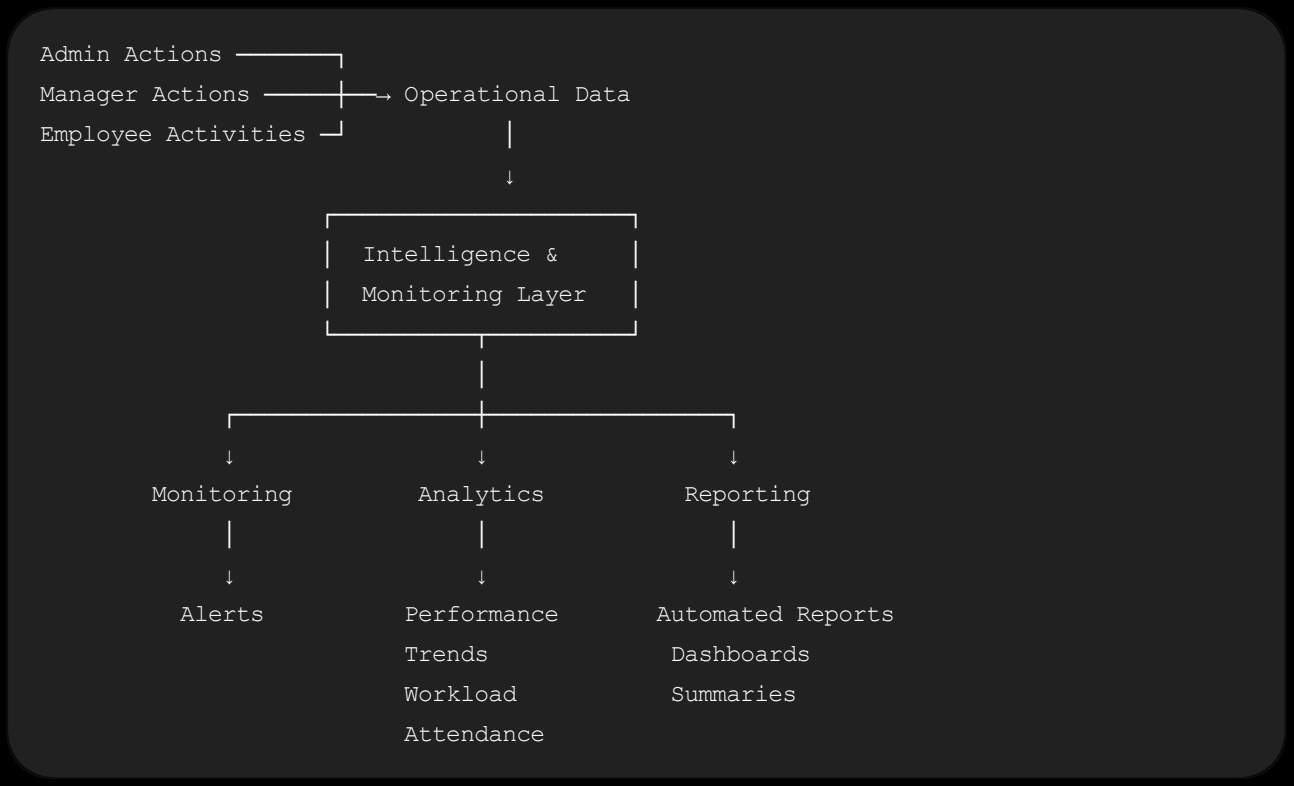
For **your particular project**, I would make Layer 3:

Intelligence, Monitoring & Reporting Layer

This is the layer where the system takes everything happening inside the Management Layer and turns it into **monitoring, analytics, reports, alerts, and organizational insight**.

It is what makes the project feel like a complete management platform rather than simply a collection of role-based dashboards.

What Layer 3 actually does



I would divide the layer into around 5 major components.

1. Operational Monitoring

The system continuously observes what is happening.

```
Operational Monitoring
        ↓
Task Activity
Attendance Activity
User Activity
Request Activity
System Activity
        ↓
Real-Time / Periodic Monitoring
```

For example:

- tasks becoming overdue
- unusual attendance patterns
- pending requests
- workload imbalance
- inactive assignments
- repeated task reassignment

This is different from the Manager simply **viewing their team**.

The Manager operates the team.

This layer **observes the whole system**.

---

2. Analytics & Performance Intelligence

This is where raw records become meaningful information.

```
Analytics Engine
      ↓
Task Analytics
Employee Analytics
Attendance Analytics
Workload Analytics
Management Analytics
      ↓
KPIs / Trends / Patterns
```

For example:

```
Task Data
   ↓
Completion Rate
On-Time Rate
Overdue Rate
Reassignment Rate
Average Completion Time
```

and:

```
Attendance Data
   ↓
Attendance Trends
Absence Patterns
Late Arrival Trends
```

This gives you a strong **decision-support** aspect.

---

3. Reporting & Dashboard Generation

You already have **Automated Reporting** appearing in your architecture, so this is where I would place it.

```
Reporting Engine
      ↓
Collect System Data
```

```
      ↓
Process / Aggregate
      ↓
Generate Reports
      ↓
Role-Based Reports
```

For example:

```
Admin → Organization-wide reports
Manager → Team reports
Employee → Personal work summary
```

This preserves your role boundaries.

---

4. Alert & Escalation Engine

This is particularly important because of the **Overdue Task** logic we just designed.

```
Event / Condition
       ↓
Rule Evaluation
       ↓
Threshold Violated?
      /       \
    No         Yes
    |           ↓
    |        Generate Alert
    |           ↓
    |      Notify Responsible Role
    |           ↓
    |      Escalation if unresolved
    |
    └──────────→ Continue Monitoring
```

Examples:

```
Task overdue
      ↓
Manager alert
      ↓
Still unresolved
      ↓
Escalation
```

or:

```
Repeated attendance issue
      ↓
Manager notification
      ↓
Report / escalation
```

Now your **Manager → Escalation Management** has a system-level engine supporting it.

---

5. Audit & Compliance Monitoring

You already have:

Admin → Security & Audit Viewer

I would keep the **viewer/control interface** in the Management Layer, but put the underlying **monitoring and analysis capability** in Layer 3.

Something like:

```
Activity / Audit Logs
        ↓
Audit Processing
        ↓
Detect / Classify Events
        ↓
Compliance Monitoring
        ↓
Reports / Alerts
```

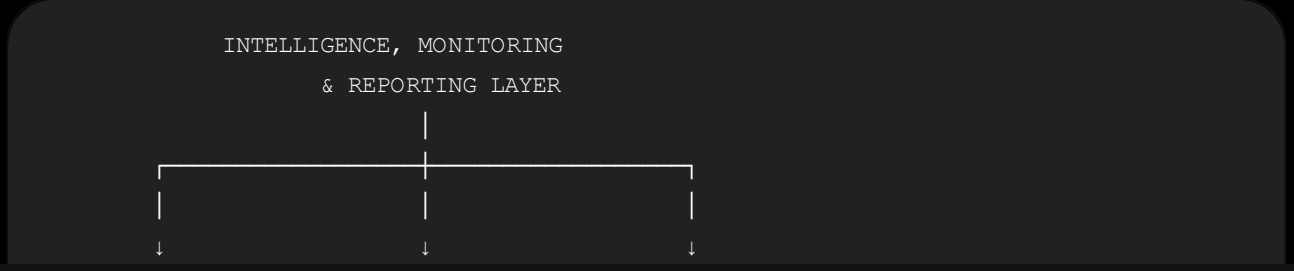
So there is a nice separation:

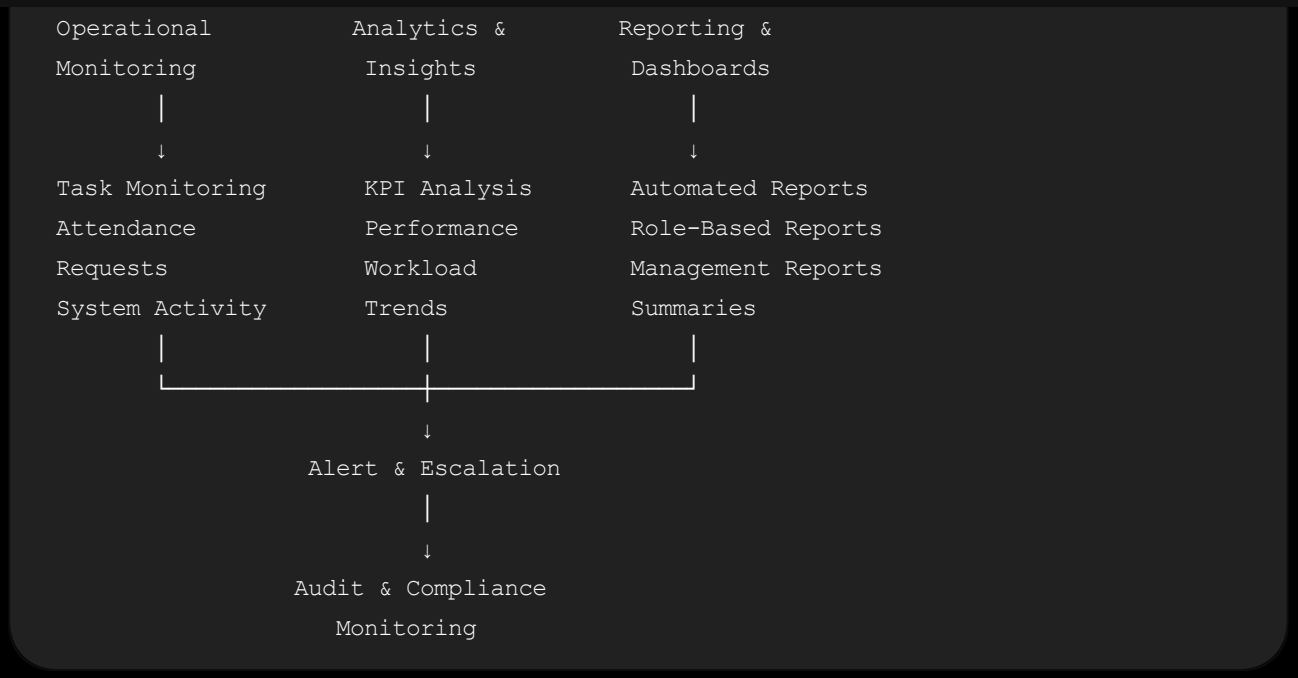
Layer 2: Admin views and acts on audit information.

Layer 3: System processes and monitors that information.

The complete third layer

I'd represent it like this:





And now your whole architecture makes sense

I'd structure the complete system as:



```
                    | REPORTING LAYER              |
                    |                              |
                    | Monitoring                   |
                    | Analytics                    |
                    | KPIs / Insights              |
                    | Automated Reporting          |
                    | Alerts & Escalation          |
                    | Audit & Compliance           |
                    └──────────────────────────────┘
```

Why I like this for your project

It gives each layer a very clear responsibility:

Layer 1 asks:

> **"Who are you, and what are you allowed to access?"**

Layer 2 asks:

> **"What work and management activities are you performing?"**

Layer 3 asks:

> **"What is happening across the organization, what does the data tell us, and what requires attention?"**

That last question is what elevates the architecture.

And there is a very clean data flow:

```
Authentication
      ↓
Authorized User
      ↓
Management Operations
      ↓
Operational Data
      ↓
Monitoring & Analytics
      ↓
Reports / Alerts / Insights
      ↓
Management Decision
      ↓
New Action
```

So your system becomes a **closed management loop**, rather than a one-way workflow.

I would actually use **“Intelligence, Monitoring & Reporting Layer”** as the formal title in your project diagram. It is broad enough to contain your existing **Automated Reporting**, **alerts**, **performance analytics**, **monitoring**, and **escalation** without making the architecture look artificially complicated.

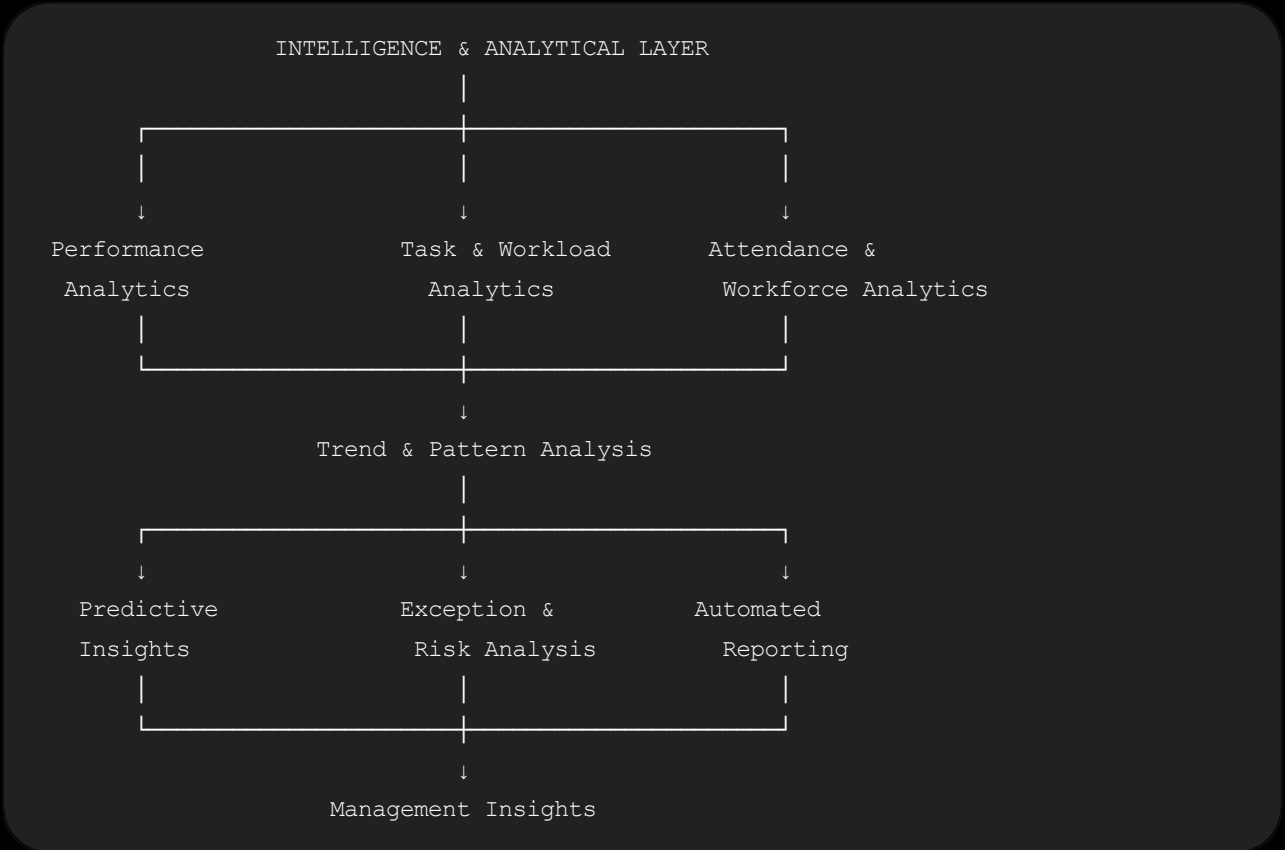
Let's name it as Intelligence and Analytical layer okay? so based on the term, what are the functionalities are you going to suggest

Yes, mate. **“Intelligence and Analytical Layer”** is a better name for what we're building. It sounds purposeful rather than just a generic reporting layer.

The important thing is that the layer should **not become another dashboard containing duplicated Manager/Admin functions**. It should take operational data generated by Layers 1 and 2, analyze it, identify patterns/risks, and present useful intelligence.

I would structure it around **7 core functionalities**.

Intelligence and Analytical Layer



1. Performance Analytics

This should analyze **how work is being performed**, rather than merely displaying completed tasks.

```
Performance Analytics
        ↓
Task Completion Rate
On-Time Completion
Average Completion Time
Productivity Trends
Manager Feedback Trends
Employee Performance Trends
Team Performance Trends
```

For example, the system could derive:

> “Team A's on-time completion rate has declined over the last four weeks.”

That's intelligence; simply showing a list of completed tasks is not.

---

2. Task & Workload Analytics

This connects directly to the Task Management system we've designed.

```
Task & Workload Analytics
          ↓
Task Distribution
Pending Tasks
Overdue Tasks
Completion Rate
Reassignment Frequency
Task Priority Distribution
Employee Workload
Team Workload
```

The interesting part is **workload analysis**:

```
Employee A → 12 active tasks
Employee B → 4 active tasks
Employee C → 2 active tasks
```

The system can identify:

> **Potential workload imbalance**

That information can then help the Manager decide whether reassignment is necessary.

---

3. Attendance & Workforce Analytics

Your attendance system shouldn't stop at storing attendance records.

```
Attendance Analytics
        ↓
Attendance Rate
Absence Trends
Late Arrival Trends
Working Hours
Availability Patterns
Leave Utilization
Attendance Exceptions
```

Then the analytical layer can identify patterns such as:

```
Attendance Data
      ↓
Pattern Analysis
      ↓
Recurring Absence / Late Pattern
      ↓
Manager Insight
```

This is considerably more useful than just displaying an attendance table.

---

4. Trend & Pattern Analysis

This is where the word **“Intelligence”** starts becoming justified.

Instead of looking at individual records, the system looks at changes over time.

```
Trend & Pattern Analysis
          ↓
Historical Data
          ↓
Compare Time Periods
          ↓
Detect Trends
          ↓
Identify Patterns
          ↓
Generate Insight
```

Possible trends:

- increasing overdue tasks
- increasing reassignment
- declining completion rates

- rising workload
- attendance changes
- increasing task delays
- recurring operational bottlenecks

You can even show:

```
Current Period
      ↕
Previous Period
      ↓
Change Detection
      ↓
Positive / Negative / Stable
```

---

5. Exception & Risk Analysis

I would **definitely** include this.

Your system already has events such as:

Overdue → Manager Review → Intervention

The Intelligence layer can identify those exceptions across the organization.

```
Exception & Risk Analysis
          ↓
Overdue Task Detection
Workload Risk
Repeated Reassignment
Attendance Exceptions
Pending Requests
Unresolved Blockers
Performance Anomalies
          ↓
Risk Classification
          ↓
Low / Medium / High
```

For example:

```
Task repeatedly overdue
          +
Task repeatedly reassigned
          ↓
Potential Operational Risk
```

The system doesn't necessarily make the managerial decision; it **brings the situation to attention**.

---

6. Predictive Insights

This is where you can make the architecture more advanced, but I'd keep it at the **"predictive insights"** level rather than claiming full AI unless your actual implementation supports it.

```
Predictive Insights
        ↓
Historical Data
        ↓
Analytical Model
        ↓
Forecast
        ↓
Potential Outcome
```

Examples:

```
Task History
    ↓
Expected Completion Pattern
    ↓
Possible Delay Risk
```

or:

```
Workload History
    ↓
Current Workload
    ↓
Projected Workload
    ↓
Potential Overload
```

So the system could surface:

> **"Task X has a high probability of missing its deadline."**

That gives the Manager an opportunity to intervene **before** the task actually becomes overdue.

---

7. Automated Reporting

This should be the output layer of your analytics.

```
Automated Reporting
        ↓
Collect Analytical Results
        ↓
Generate Report
        ↓
Role-Based Report
```

You could have:

```
Admin
 ↓
Organization Analytics

Manager
 ↓
Team Analytics

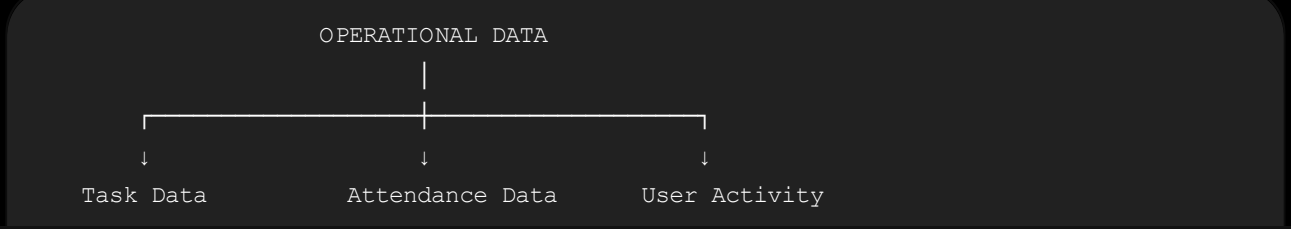
Employee
 ↓
Personal Work Analytics
```

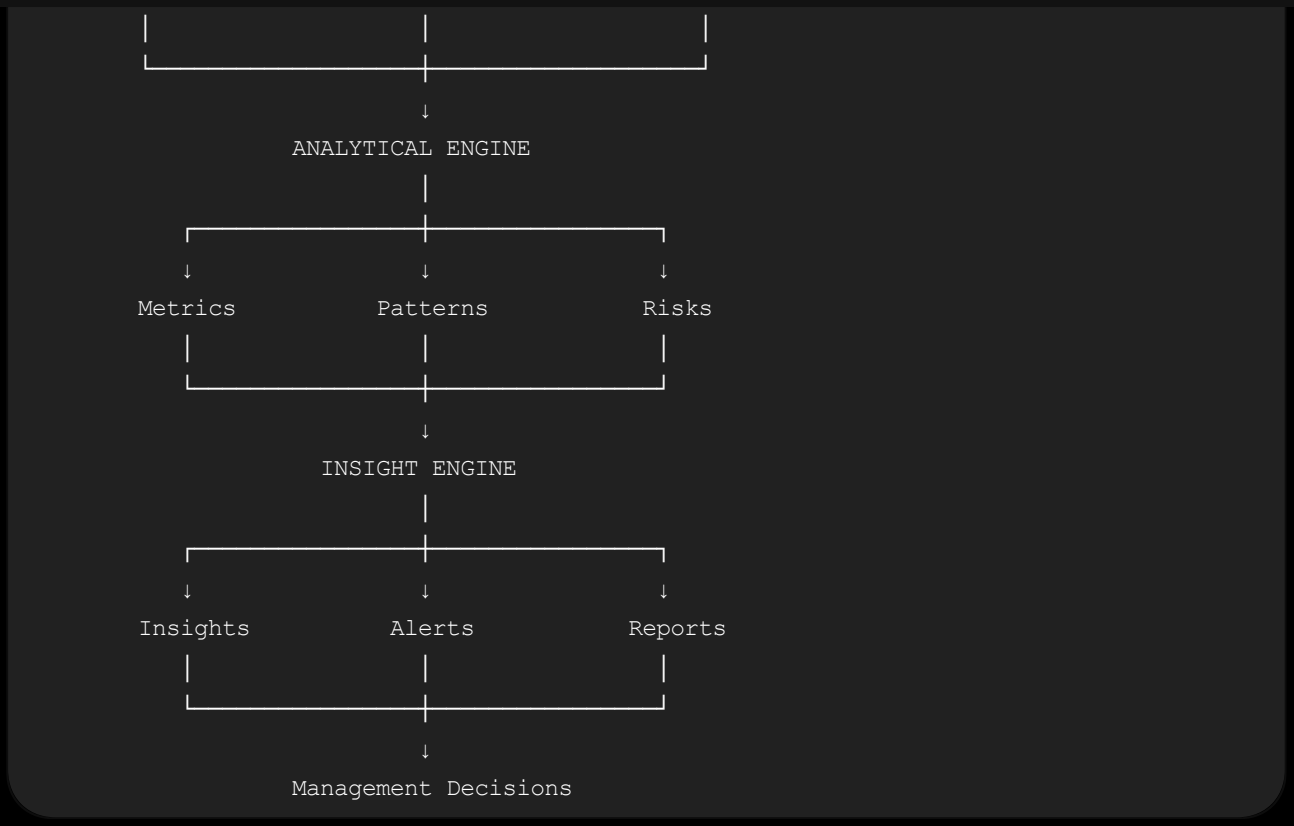
Reports could include:

- Task performance
- Employee/team performance
- Workload distribution
- Attendance statistics
- Overdue analysis
- Reassignment analysis
- Productivity trends
- Exception summaries

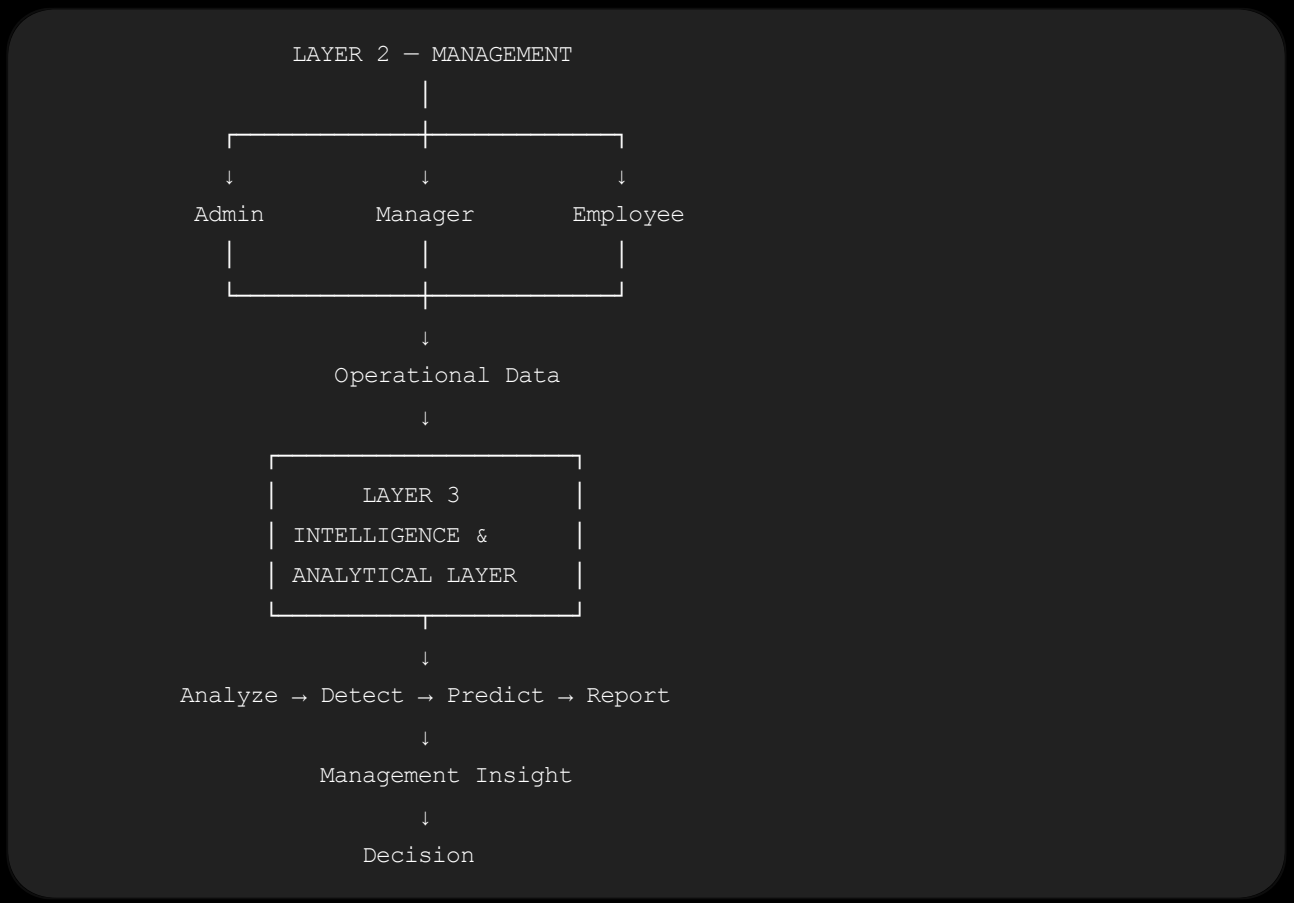
And I'd add a central "Insight Engine"

This is the part I would use to tie the entire layer together.





This gives us a really nice relationship with the previous layer:



The final functionality set I'd recommend

For the actual **high-level architecture diagram**, I wouldn't put 15–20 small functions. I'd use these **seven major modules**:

Module	Main purpose
Performance Analytics	Measure work and team performance
Task & Workload Analytics	Analyze task distribution, delays and workload
Attendance & Workforce Analytics	Analyze attendance, availability and workforce patterns
Trend & Pattern Analysis	Detect changes and recurring patterns
Exception & Risk Analysis	Identify abnormal conditions and operational risks
Predictive Insights	Forecast potential delays, overloads and risks
Automated Reporting	Convert analysis into role-based reports and summaries

And conceptually:

> **Management Layer records what happened.**
> **Intelligence & Analytical Layer determines what it means.**

That distinction is, in my opinion, the strongest way to justify having **three layers** in your project.

Availability status of the employee involves which set of options list them out

For your **Employee → Attendance & Availability** section, I would keep the availability states focused on the employee's **current work capacity**, rather than mixing them with attendance or leave statuses.

Recommended Availability Status options

Status	Meaning
Available	Employee is currently available for assigned work
Busy	Employee is working on existing tasks and has limited capacity
On Break	Temporarily unavailable but still on duty

Status	Meaning
In Meeting	Temporarily occupied with a meeting
In Training	Occupied with training or learning activities
Do Not Disturb	Working but should not be interrupted except for important matters
Working Remotely	Available for work but operating remotely
Offline	Not currently connected/active in the system
On Leave	Officially unavailable due to approved leave
Absent	Expected to work but currently unavailable
Unavailable	General temporary unavailability
Out of Office	Away from normal work responsibilities

For your actual project, I'd simplify it

Having all 12 as selectable statuses may be overkill. A clean industrial implementation could use:

```
Availability Status
        │
        ├── Available
        ├── Busy
        ├── On Break
        ├── In Meeting
        ├── In Training
        ├── Do Not Disturb
        ├── Working Remotely
        ├── On Leave
        ├── Absent
        └── Offline
```

There is also an important distinction for your architecture:

Attendance status tells you *whether the employee is present/working according to attendance records.*

Availability status tells you *whether the employee can currently take work.*

For example:

```
Attendance: Present
Availability: Busy
```

means the employee is at work but shouldn't necessarily be assigned another task.

And:

```
Attendance: Present
Availability: Available
```

means they're currently available for work.

That distinction will be very useful later in your **Workload Management → Reassignment** logic.

Under Employee's Work Management we discusssed about Record-Work-Activity right?
So based on that, what do you mean when coming to progressive update, work notes, and supporting documents, okay? apart from that for the other things:
1.1 Task dashboard
1.2 Get task details
1.3 Task Execution {
In this the task update progress, holds stages that a work undergoes, In-Progress, blocked, waiting for review,

Show more

Yes mate. Now that you have defined **1.3 Task Execution** and **1.4 Task Submission** much more precisely, **1.5 Record Work Activity** should not duplicate those functions.

The clean way to think about it is:

> **Task Execution tells us the current state of the work.**
> **Record Work Activity tells us the history of what the employee did to reach that state.**

That distinction is very important for your audit architecture.

1.5 Record Work Activity

I would structure it as:

```
1.5 RECORD WORK ACTIVITY
        |
        ├── 1.5.1 Progress Update
        ├── 1.5.2 Work Notes
        ├── 1.5.3 Time-Spent / Effort Entry
        └── 1.5.4 Supporting Documents / Evidence
```

These four are not four different stages of the task.

They are **four categories of activity records generated while the employee performs the task**.

1.5.1 Progress Update

What does it mean?

A **Progress Update** records the employee's current advancement toward completion.

It answers:

> **"How much of the assigned work has been completed, and what has changed since the previous update?"**

For example:

```
Task: Develop Login Module

Progress Update #1
→ UI completed
→ Backend authentication pending
→ Progress: 40%

Progress Update #2
→ Backend authentication completed
→ Testing in progress
→ Progress: 75%

Progress Update #3
→ Testing completed
→ Ready for submission
→ Progress: 100%
```

What it can contain

```
Progress Update
   |
   ├── Progress %
   ├── Current work status
   ├── Work completed since last update
   ├── Work remaining
   └── Update timestamp
```

You don't necessarily need to show all of these on your main flowchart.

How does it assist?

It helps the Manager answer:

- Is the employee actually progressing?
- Is the task stagnant?
- Is the task likely to miss its deadline?
- How much work remains?
- Did progress stop after a particular point?

How does it help auditing?

Every update becomes a **timestamped historical record**:

```
Employee
   ↓
Progress Update
   ↓
Timestamp
   ↓
Previous Value → New Value
   ↓
Audit Record
```

For example:

```
Progress: 40% → 75%
Updated by: Employee
Date/Time: 10:42 AM
```

So the system can reconstruct the task's progress over time.

1.5.2 Work Notes

This is different from Progress Update.

Progress Update = structured progress information

Work Notes = contextual explanation

Work Notes answer:

> **"What happened while performing the work?"**

For example:

```
Work Note:

"Authentication API was failing because the token
validation service was returning an invalid response.
Updated validation handling and retested."
```

Another example:

```
"Waiting for database credentials from the infrastructure team."
```

That is not really a progress percentage. It is **context**.

Work Notes can capture

```
Work Notes
   │
   ├── Actions performed
   ├── Observations
   ├── Decisions
   ├── Problems encountered
   ├── Dependencies
   └── Resolution details
```

How does it assist?

This gives the Manager context that a status like:

> `In Progress – 65%`

cannot provide.

Suppose the Manager sees:

```
Task Status: Blocked
Progress: 65%
```

The work note may explain:

> "Blocked because API credentials have not been provided."

Now the Manager knows **why** the task is blocked.

Audit role

Work notes provide the **narrative trail**.

They help establish:

> Who reported what, when it was reported, and what explanation was given.

This becomes especially valuable when there is a dispute or escalation.

1.5.3 Time-Spent / Effort Entry

This is slightly different from attendance.

This is **task-level effort**, not employee attendance.

That's an important distinction.

Attendance asks:

> "How long was the employee working?"

Time/Effort Entry asks:

> "How much effort was spent on this particular task?"

For example:

```
Task: Develop Login Module

09:15 – 10:00
Development
45 minutes

10:30 – 11:20
Testing
50 minutes

14:00 – 15:15
Bug fixing
75 minutes
```

Structure

```
Time / Effort Entry
       |
       ├── Start / End
       ├── Duration
       ├── Activity type
       └── Entry timestamp
```

You can optionally describe effort in hours rather than requiring start/end times.

How does it assist?

It can help management analyze:

- estimated vs actual effort
- which tasks consume excessive time
- workload distribution
- productivity trends
- future task estimation

For your **Intelligence & Analytical Layer**, this becomes useful data.

For example:

```
Estimated effort → 4 hours
Actual effort    → 9 hours
                     ↓
              Effort variance
```

That could indicate:

> difficult task, underestimated scope, employee dependency, or process issue.

Audit role

The audit trail can record:

```
Employee
Task
Effort Entry
Duration
Timestamp
Modification history
```

If an employee changes:

```
2 hrs → 4 hrs
```

the system can preserve:

```
Old Value: 2 hrs
New Value: 4 hrs
Changed By: Employee
Changed At: [timestamp]
```

That is much stronger than simply storing the latest number.

1.5.4 Supporting Documents / Evidence

This is not just a generic file-upload feature.

Its role is to provide **evidence associated with the work performed**.

Examples depend on your project, but conceptually:

```
Supporting Evidence
        |
        ├── Documents
        ├── Reports
        ├── Screenshots
        ├── Test Results
        ├── Design Files
        └── Other Work Artifacts
```

For example:

```
Employee completes testing
         ↓
Uploads test report
         ↓
Manager reviews submission
```

The evidence therefore supports the claim:

> **"This work was actually performed."**

How does it assist?

It helps the Manager:

- verify completed work
- review deliverables

- understand progress
- validate corrections
- make approval decisions

And it becomes particularly important in your:

Task Submission → Manager Review

stage.

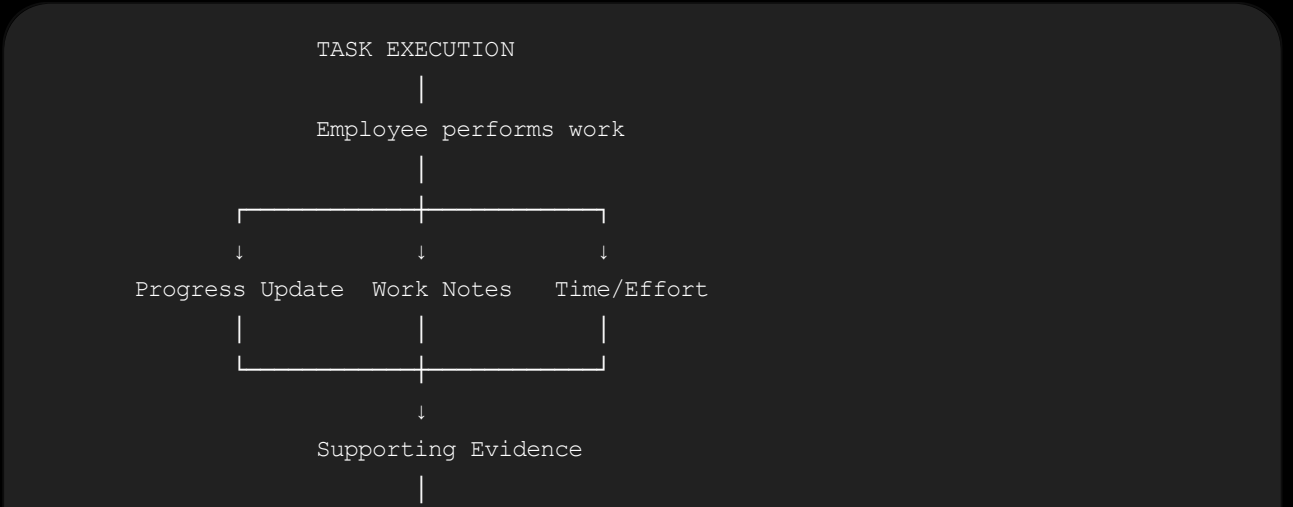
Now the critical part: how these relate to your Task Execution stages

You've defined:

```
Task Execution
      ↓
In-Progress
      ↓
Blocked
      ↓
Waiting for Review
      ↓
Approval
      ↓
Issue Resolved
      ↓
Done / Closed
```

I would **not make Progress Update, Work Notes, Time Entry and Supporting Documents separate task statuses**.

Instead:



```
                    ↓
              Current Task State
```

So the employee can generate multiple work-activity records while the task remains in one state.

For example:

```
IN-PROGRESS
     │
     ├── Progress Update → 35%
     ├── Work Note → "API integration started"
     ├── Effort Entry → 2.5 hours
     └── Document → API test report

     ↓

IN-PROGRESS
     │
     ├── Progress Update → 70%
     ├── Work Note → "Integration completed"
     └── Effort Entry → +3 hours

     ↓

WAITING FOR REVIEW
```

That is a much more realistic model.

How all of this feeds your Audit Trail

This is where your architecture becomes strong.

You effectively have **two different things being audited**:

A. State Changes

These are changes to the **task lifecycle**.

```
Assigned
   ↓
In-Progress
   ↓
Blocked
   ↓
In-Progress
   ↓
Waiting for Review
   ↓
```

```
Changes Requested
   ↓
In-Progress
   ↓
Waiting for Review
   ↓
Approved
   ↓
Closed
```

Audit records:

```
Previous State
New State
Changed By
Timestamp
Reason
```

B. Work Activity Changes

These are the **actions/data recorded during those states**.

```
Progress Update
Work Note
Effort Entry
Document Upload
```

Audit records can therefore look conceptually like:

```
TASK AUDIT HISTORY
_____________________________________

10:02  Employee
       Status: Assigned → In-Progress

10:25  Employee
       Progress: 0% → 20%

10:40  Employee
       Work Note Added

11:30  Employee
       Effort Entry: +1.5 hrs

12:10  Employee
       Supporting Document Uploaded

14:05  Employee
```

```
       Status: In-Progress → Blocked

14:12  Employee
       Work Note:
       "Waiting for infrastructure access"

15:30  Manager
       Blocker Resolved

15:31  System
       Status: Blocked → In-Progress

17:10  Employee
       Progress: 20% → 100%

17:15  Employee
       Status: In-Progress → Waiting for Review

17:45  Manager
       Status: Waiting for Review → Approved

17:46  System
       Task Closed
```

Now you have a **complete chronological reconstruction of the task**.

And there is one more distinction I'd strongly recommend

Don't call the entire thing simply **"Audit Changes"** in the Employee section.

Because:

Record Work Activity ≠ Audit Log

Instead:

```
Employee
   ↓
Record Work Activity
   ↓
Operational Records
   │
   ├── Progress Updates
   ├── Work Notes
   ├── Effort Entries
   └── Supporting Evidence
   ↓
```

```
Audit / History Mechanism
   ↓
Immutable Historical Record
```

The employee **creates the work activity**.

The system **audits the activity and changes**.

The Manager **reviews the resulting history**.

The Admin **can inspect the broader audit trail**.

That separation fits beautifully with the three-role architecture you've already established.

So I'd define 1.5 like this in your documentation

> **1.5 Record Work Activity:** Enables employees to maintain a traceable record of task execution through progress updates, work notes, time/effort entries, and supporting evidence. These records provide contextual and quantitative information about work performed and contribute to the task's chronological audit history.

And the four components have very clean meanings:

Component	Answers	Primary value	Audit contribution
Progress Update	*How much has progressed?*	Progress tracking	Records progress changes over time
Work Notes	*What happened / why?*	Context & explanation	Provides narrative history
Time / Effort Entry	*How much effort was spent?*	Effort analysis	Records effort changes/entries
Supporting Documents	*What evidence proves the work?*	Verification	Records uploaded/modified artifacts

The beautiful part is that these four then feed your **Intelligence & Analytical Layer** later:

```
Work Activity
     ↓
Progress + Notes + Effort + Evidence
     ↓
Operational History
```

```
      ↓
Intelligence & Analytical Layer
      ↓
Performance
Workload
Task Efficiency
Delay Analysis
Trend Analysis
Risk Detection
Reports
```

So **1.5 isn't a random collection of employee features** — it becomes the data-generating foundation for your analytics and audit architecture.

So based on the entire above description prepare a task-given to me, like what is it aim, why it is needed that's all, not in layer detail, in mark-down format, from initial set of instructions from the actor chairman perspective on designing about flowchart

Task: Design the Complete Management System Flowchart

Aim

Design a **comprehensive, logically connected, and professionally structured flowchart** for the proposed management system that clearly represents how users enter, operate within, and interact with the system from authentication through organizational operations and analytical outcomes.

The flowchart should be designed from the **Chairman's perspective**: it must allow a non-technical stakeholder to understand **how the organization operates through the system, who is responsible for each action, how decisions are made, and how information moves through the organization**.

Why This Flowchart Is Needed

The flowchart is required to provide a **single visual representation of the complete system workflow** rather than showing isolated features or screens.

It should demonstrate:

- How users are **authenticated and authorized** before accessing the system.
- How responsibilities are separated among **Admin, Manager, and Employee**.

- How organizational activities such as **users, tasks, attendance, leave, requests, communication, approvals, and work execution** are managed.
- How tasks move through their lifecycle, including **assignment, execution, progress tracking, blockers, overdue situations, reassignment, review, correction, approval, and closure**.
- How employee work is recorded through **progress updates, work notes, effort/time entries, and supporting evidence**.
- How exceptional situations are handled through **manager intervention, escalation, and traceable changes**.
- How operational information is transformed into **performance measurements, trends, risks, insights, and reports**.
- How activities and important changes remain **traceable for accountability and auditing**.

Chairman-Level Design Objective

The final flowchart should answer four fundamental questions at a glance:

> **Who is accessing the system?**
> **What can they do?**
> **What happens when something changes or goes wrong?**
> **What information does management ultimately receive for decision-making?**

Expected Design Characteristics

The flowchart should be:

Comprehensive enough to represent the important operational processes, but **simple enough that a Chairman or senior management member can understand it without technical knowledge**.

It should maintain:

- Clear hierarchy
- Distinct responsibilities for each role
- Logical decision points
- Meaningful process flows
- Exception and escalation paths
- Traceability and audit awareness
- Clear movement of information from operational activities toward management insights

The objective is **not to display every software function**, but to visually communicate the **complete organizational workflow, responsibility structure, decision-making**

process, and information flow of the system.

Why my claude=code is not responding as based on the following json{ "env": { "ANTHROPIC_BASE_URL": "https://openrouter.ai/api", "ANTHROPIC_AUTH_TOKEN": "", "ANTHROPIC_API_KEY": "", "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1" } }, "model": "nvidia/nemotron-3.5-lightning:free", "enabledPlugins": { "mcp-server-dev@claude-plugins-official": true }, "effortLevel": "high", "theme": "dark-daltonized", "extraKnownMarketplaces": { "impeccable": { "source": { "source": "github", "repo": "pbakaus/impeccable" } } }

The main problem is that your JSON is mixing **Claude Code configuration** with **OpenRouter model routing**, but the configuration you showed has a couple of likely blockers.

The biggest one is here:

JSON

```
"ANTHROPIC_AUTH_TOKEN": "",
"ANTHROPIC_API_KEY": ""
```

You're telling Claude Code to send requests through OpenRouter, but you're giving it **no authentication token**. OpenRouter expects an API key, and OpenRouter's current Claude Code setup specifically recommends putting the OpenRouter key in `ANTHROPIC_AUTH_TOKEN` while explicitly setting `ANTHROPIC_API_KEY` to an empty string. OpenRouter +1

So the important part should look like:

JSON

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "sk-or-v1-xxxxxxxxxxxxxxx",
    "ANTHROPIC_API_KEY": "",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  }
}
```

The second issue: your model

You're forcing:

```
JSON

"model": "nvidia/nemotron-3.5-lightning:free"
```

This is a **third-party NVIDIA model**, not an Anthropic Claude model.

Claude Code is designed around Anthropic's API semantics, and OpenRouter's own Claude Code guidance recommends Anthropic models for compatibility. OpenRou…

So even though OpenRouter supports hundreds of models, that does **not** mean every model is a drop-in replacement for everything Claude Code expects.

For troubleshooting, use an Anthropic model first, for example:

```
JSON

"model": "anthropic/claude-sonnet-4"
```

or a current Claude model exposed by OpenRouter.

OpenRouter's current model catalog uses the `provider/model` form, and Anthropic models are available through its Anthropic integration. OpenRouter +1

Third issue: `:free`

This:

```
nvidia/nemotron-3.5-lightning:free
```

means you're specifically requesting a **free variant**. Free models are subject to their own availability/rate limits, so even if the model itself is reachable, it may not behave like a reliable long-running coding-agent backend. OpenRouter documents separate free-tier limitations. OpenRou…

For debugging Claude Code, I'd first get the setup working with a known-compatible Anthropic model and only then experiment with other models.

Fourth issue: cached Claude authentication

There is another easy-to-miss issue.

If Claude Code was previously logged into an Anthropic account, the cached authentication can interfere with the environment-variable configuration. OpenRouter's

current Claude Code instructions specifically recommend logging out and relaunching when switching over. OpenRou...

So the sequence should be roughly:

```
1. Configure OpenRouter key
2. /logout in Claude Code
3. Restart Claude Code
4. Check /status
5. Test a simple prompt
```

`/status` should show that authentication is coming from `ANTHROPIC_AUTH_TOKEN` and that the base URL is OpenRouter. OpenRou...

I'd simplify your config while diagnosing it

Your current config has:

JSON

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "",
    "ANTHROPIC_API_KEY": "",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  },
  "model": "nvidia/nemotron-3.5-lightning:free",
  "enabledPlugins": {
    "mcp-server-dev@claude-plugins-official": true
  },
  "effortLevel": "high",
  "theme": "dark-daltonized",
  "extraKnownMarketplaces": {
    "impeccable": {
      "source": {
        "source": "github",
        "repo": "pbakaus/impeccable"
      }
    }
  }
}
```

I'd temporarily reduce it to:

JSON

```
{
  "env": {
```

```
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "YOUR_OPENROUTER_KEY",
    "ANTHROPIC_API_KEY": "",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  },
  "model": "anthropic/claude-sonnet-4"
}
```

Get **that** working first. Then add your plugins, theme, marketplace, and experiment with alternate models one at a time.

One more thing

Don't paste your actual OpenRouter key into chat. Replace it with something like:

```
sk-or-v1-*******
```

When you're testing locally, the key must actually be present in your Claude Code configuration/environment. OpenRouter's Claude Code setup explicitly uses the OpenRouter `sk-or-...` key as `ANTHROPIC_AUTH_TOKEN`. OpenRou...

If you give me the **exact error/output Claude Code is showing** (for example `401`, `404`, `model not found`, `BadRequest`, or it simply hangs), I can pinpoint which part of this configuration is failing.

I changed it in past mate, like the sonnet to a particular model that are actually free on openrouter, so based on them I am working on it.. i got free models on openrouter so that this help my claude code to work freely

Ahh, yes mate — **I understand what you're doing now.** You're not trying to use Claude's paid Sonnet subscription. You're using **Claude Code as the client**, with **OpenRouter as the backend**, and selecting a `$0` OpenRouter model so your coding sessions don't consume paid Anthropic usage.

That approach is reasonable. OpenRouter currently does have free models, and its `openrouter/free` router can automatically select among available free models. OpenRouter +1

The important catch is **Claude Code compatibility**, not whether the OpenRouter model itself is free.

OpenRouter's current Claude Code documentation explicitly says the native Claude Code integration is built around **Anthropic request semantics** and is only guaranteed with Anthropic's first-party provider. It specifically says non-Anthropic models aren't supported through that native integration. OpenRou...

So your idea is:

```
Claude Code
     ↓
OpenRouter
     ↓
Free model
     ↓
No Anthropic subscription usage
```

That's conceptually fine, but this:

```
JSON

"model": "nvidia/nemotron-3.5-lightning:free"
```

doesn't guarantee that Claude Code will function correctly with it.

Why it may appear to "not respond"

Claude Code isn't simply a normal chat client. It performs things such as:

```
Prompt
 ↓
Tool calls
 ↓
File inspection
 ↓
Command execution
 ↓
Tool results
 ↓
More model reasoning
 ↓
Another tool call
 ↓
Final response
```

So a model can successfully answer ordinary prompts through OpenRouter but still fail when Claude Code expects particular **tool-calling / Anthropic message semantics**.

And this is especially relevant because OpenRouter's free router explicitly filters models based on capabilities such as **tool calling, structured outputs, image understanding,**

and extended reasoning. OpenRou…

So I'd diagnose your setup in this order

Don't change everything at once.

First verify whether your OpenRouter authentication itself works:

```json
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "YOUR_OPENROUTER_KEY",
    "ANTHROPIC_API_KEY": "",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  }
}
```

Then test your chosen model.

If **ordinary responses work but Claude Code's tools don't**, the problem is very likely the model's compatibility with Claude Code's agent/tool protocol rather than your API key.

Also, your free-model idea is valid in principle: OpenRouter currently documents free models as `$0` inference, with a separate `openrouter/free` router that selects from available free models. OpenRouter +1

One correction to my previous answer, though: **I shouldn't have assumed that you needed to switch back to Sonnet simply because you're using Claude Code.** Your goal is specifically to use Claude Code's interface/workflow while routing inference through a free OpenRouter model. The real question is **which free model can satisfy the parts of Claude Code you actually use**.

Send me the **exact symptom/error from Claude Code** — even just a screenshot or the terminal text — and I can tell you whether you're dealing with **authentication, endpoint, model compatibility, tool calling, or rate limiting**.

what should i put in auth_token

Put your **OpenRouter API key** there — the key that OpenRouter gives you, typically starting with `sk-or-`. OpenRouter's current Claude Code setup uses that key as

`ANTHROPIC_AUTH_TOKEN`. OpenRou...

So your config should be:

```
JSON

{
  "env": {
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "sk-or-v1-YOUR_OPENROUTER_KEY",
    "ANTHROPIC_API_KEY": "",
    "CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY": "1"
  },
  "model": "your-model-id"
}
```

Important: don't put your key into this chat. Keep the actual `sk-or-...` value only in your local Claude Code configuration.

OpenRouter's current documentation also says `ANTHROPIC_API_KEY` should explicitly remain empty when using this setup, so Claude Code doesn't fall back to Anthropic authentication. OpenRou...

To obtain the key, use OpenRouter's **API Keys** section and create/copy your key. OpenRou...

f"I need each location\'s three character codes mentioned here {locations} in a python list, just be python list returned no greetings or anyother bluff just return me the python list or set that\'s it I need it for processing"

locations = set(['INDORE', 'RAJKOT', 'PUNE', 'NASHIK', 'BHUBANESWAR', 'BHOPAL', 'VADODARA', 'GOA', 'AHMEDABAD', 'JAIPUR', 'UDAIPUR', 'NAGPUR', 'GUWAHATI', 'GURUGRAM', 'MOHALI', 'KOLKATA', 'HYDERABAD', 'TRIVANDRUM', 'AJMER', 'RANCHI', 'MUMBAI', 'SHAMBAJINAGAR', 'RAIPUR', 'COIMBATORE', 'THANE', 'SILIGURI', 'KOCHI', 'SURAT', 'CHENNAI', 'BENGALURU', 'DELHI']);

['IDR', 'RAJ', 'PNQ', 'ISK', 'BBI', 'BHO', 'BDQ', 'GOI', 'AMD', 'JAI', 'UDR', 'NAG', 'GAU', 'GUR', 'MOH', 'CCU', 'HYD', 'TRV', 'KQH', 'IXR', 'BOM', 'IXU', 'RPR', 'CJB', 'THN', 'SIL', 'COK', 'STV', 'MAA', 'BLR', 'DEL']

Let's focus on the monitoring portal as it was renamed as internal operations tracking system. So the things is right now I am focusing on deciding access-levels okay.. I am saying the access levels like:
1. Higher level access
2. Limited access Regarding the entire system.

The following is the roles that I have:
1. Chairman,
2. Senior Manager
3. General Manager

Show more

Worked for 21s

Yes, mate. Now that you have renamed the platform to the **Internal Operations Tracking System**, I would treat this as an **access-control and responsibility design problem**, not merely a list of job titles.

The existing project design already establishes that authentication determines the user's role and routes them into role-specific management functions; the Management Layer then separates Admin, Manager, and Employee responsibilities, with task assignment, attendance, work execution, audit history, and reporting forming the operational core. gemini-chattings +1

Internal Operations Tracking System

Role, Access-Level & Responsibility Model

1. Purpose of the Access-Level Model

The purpose of the access model is to ensure that every person can access the information and operations required for their job **without receiving unnecessary authority**.

A proper RBAC design associates permissions with roles rather than individual users, and role hierarchies can be used to provide broader permissions to higher-level roles. NIST describes RBAC this way and emphasizes that privileges should correspond to the responsibilities of the role. NI... +1

The system should therefore answer three separate questions:

> **What level of access does this role have?**
> **What information can this role see?**
> **What actions can this role perform?**

This distinction is important because **organizational seniority and technical privilege are not the same thing**.

For example, the Chairman may have organization-wide visibility and decision authority but should not necessarily have every technical configuration capability. Conversely, an IT System Analyst may require privileged technical access without receiving authority to make business decisions.

That separation follows the principles of least privilege and separation of duties recommended in modern access-control practice. OWASP Cheat ... +1

---

2. Your Two Primary Access Levels

Your proposed two-level model can work well as the **top-level classification**:

Higher-Level Access

Intended for roles that require broad organizational visibility, significant decision-making authority, or privileged platform control.

Limited Access

Intended for roles whose authority is restricted to a defined operational, managerial, technical, team, or personal scope.

However, I would **not interpret "Limited" as "weak."**

A Lead Manager can have substantial managerial authority while still having limited scope.

Likewise, an IT System Analyst can have technically powerful capabilities while having limited business-decision authority.

So I recommend:

```
ACCESS LEVEL
     |
     ├── Higher-Level Access
     |
     └── Limited Access
```

```
Then separately:

DATA SCOPE
     |
     ├── Organization
     ├── Department
     ├── Team
     └── Own

And:

ACTION AUTHORITY
     |
     ├── View
     ├── Create
     ├── Edit
     ├── Approve
     ├── Reassign
     ├── Configure
     └── Override
```

This is much closer to an actual RBAC design. Your earlier project material also arrived at the same idea of defining permissions by **data scope, actions, workflow rights, and administrative rights**. gemini-chattings

---

3. Proposed Role Classification

Role	Access Level	Primary Scope
Chairman	Higher-Level	Organization-wide oversight
Senior Manager	Higher-Level	Broad operational/departmental oversight
General Manager	Higher-Level	Business/branch-wide management
Lead Manager	Limited	Assigned team / operational unit
Executive	Limited	Own work and assigned responsibilities
IT System Analyst	Higher-Level*	Technical/system operations
Business Development Executive	Limited	Own assignments, targets and work

Role	Access Level	Primary Scope
Social Media Manager	Limited	Own/team content operations
Graphic Designer	Limited	Assigned creative work
Admin	Limited	Routine administrative operations
Super Admin	Higher-Level	Platform-wide administration

*The IT System Analyst is technically privileged, but that should **not** mean unrestricted business authority.

---

4. Chairman

Access Level

Higher-Level Access

Purpose

The Chairman represents the highest **business oversight** level in the system.

The Chairman should primarily use the platform to understand the organization's operational condition and make high-level decisions rather than perform routine administrative work.

Core responsibilities

Organizational oversight

- View organization-wide operational status.
- Monitor major active work.
- Review overall task progress.
- View departmental/team performance.
- Observe workload and overdue activity.
- Review important exceptions and escalations.

Decision-making

- Approve or intervene in matters requiring executive authority.
- Resolve exceptional situations beyond normal managerial authority.
- Review major operational changes.
- Authorize selected high-impact decisions.

Strategic visibility

- View analytical summaries.
- Review performance trends.
- Review operational reports.
- Monitor organizational KPIs.

Your existing design already gives higher-level administration the ability to monitor global work, directly intervene when necessary, and review system activity. gemini-chattings

Chairman should generally NOT perform routinely

- password resets
- everyday employee task assignment
- editing every attendance record
- routine user maintenance
- database-level operations
- modifying system internals

The Chairman has **high visibility and decision authority**, not necessarily the highest technical privilege.

---

5. Senior Manager

Access Level

Higher-Level Access

Purpose

The Senior Manager should operate at a broader management level than a Lead Manager.

Responsibilities

- Monitor multiple teams or operational functions.
- Review Manager performance.
- Review team workload.
- Monitor task completion and SLA adherence.
- Review overdue and escalated work.
- Approve significant operational requests.
- Review performance analytics.
- Intervene when an issue exceeds a Lead Manager's authority.
- Review cross-team dependencies.
- Monitor organizational targets relevant to their area.

Authority

The Senior Manager is primarily a **management decision layer**, not a system-administration layer.

For example:

```
Lead Manager
     ↓
Unable to resolve issue
     ↓
Senior Manager
     ↓
Decision / Intervention
```

---

6. General Manager

Access Level

Higher-Level Access

Purpose

The General Manager should have broad visibility over the business operations under their responsibility.

Responsibilities

- Monitor overall operational performance.
- Review department/branch productivity.
- Monitor targets and KPIs.
- Review workload distribution.
- Monitor major overdue activities.
- Review managerial performance.
- Approve high-level operational requests.
- Review analytical reports.
- Coordinate across operational divisions.
- Handle escalations beyond individual teams.

Main difference from Senior Manager

This depends on your company's actual hierarchy, so I would **not hard-code the hierarchy yet**.

For the system design, the cleaner distinction is:

```
Lead Manager → Team-level operations
Senior Manager → Multi-team / departmental oversight
General Manager → Broad business / branch operations
Chairman → Executive organizational oversight
```

If your real organization places Senior Manager and General Manager differently, the same permission architecture can be rearranged without redesigning the system.

---

7. Lead Manager

Access Level

Limited Access

Scope

Team / assigned operational group

This is the manager who interacts most directly with employee work.

Your existing system already defines the Manager as the party responsible for creating tasks, setting priority and due dates, assigning employees, notifying them, handling reassignment, and reviewing completed work. gemini-chattings

Responsibilities

Task management

- Create tasks.
- Define priority.
- Define due date/SLA.
- Assign employees.
- Reassign tasks.
- Review submitted work.
- Approve or request corrections.
- Handle overdue tasks.

Team management

- View team members.
- Monitor availability.
- Review workload.
- Review attendance.
- Manage leave requests where authorized.

Operational intervention

- Resolve employee blockers.
- Review issue tickets.
- Intervene in overdue tasks.
- Escalate unresolved issues.

Performance

- Monitor team KPIs.
- Review task completion.
- Review work activity.
- Provide feedback.

This corresponds well with the project's existing Manager architecture around task assignment, reassignment, leave/roster management, blocker resolution, and live target monitoring. gemini-chattings +1

---

8. Executive

Access Level

Limited Access

Scope

Own assignments and authorized operational information

The Executive should not be interpreted as an administrator simply because the designation sounds senior.

Responsibilities

- View assigned work.
- Update assigned tasks.
- Record work activity.
- Upload supporting evidence.
- Submit completed work.
- Raise issues/blockers.
- Request assistance or escalation.
- View personal attendance.
- Submit leave/request forms.
- Receive notifications.
- View personal performance information.

Restrictions

- Cannot assign work to others.
- Cannot approve another employee's work.
- Cannot alter system configuration.
- Cannot access organization-wide reports unless specifically authorized.

---

9. IT System Analyst

This role needs special treatment.

Access Level

Higher-Level Technical Access

But **not unrestricted business authority**.

Technical responsibilities

- Monitor system availability.
- Diagnose application issues.
- Investigate technical errors.
- Monitor integrations/APIs.
- Support authentication and access problems.
- Investigate system incidents.
- Monitor system logs where authorized.
- Assist with technical configuration.
- Support data/system integrity.
- Participate in deployment and maintenance activities.

Your earlier project discussion already associated the IT Analyst role with deployment, issue-resolution and technical KPIs. gemini-chattings

Critical security boundary

I would **not formally make IT System Analyst = Super Admin**.

They can share some technical administration capabilities, but the Super Admin should retain authority over things such as:

- role hierarchy
- permission definitions
- platform-wide configuration
- privileged overrides
- emergency administrative actions

NIST explicitly recommends separation of duties, including avoiding concentration of access-control administration and audit responsibilities in the same role. NIST Publicati...

So a better model is:



rather than:

```
Super Admin = IT Analyst = Admin
```

---

10. Business Development Executive

Access Level

Limited Access

Scope

Own assigned operational work and authorized BDE information

Responsibilities

- View assigned tasks.
- Execute assigned business-development activities.
- Record activity/progress.
- Record work notes.
- Record effort/time.
- Upload evidence.
- Submit completed work.
- Raise blockers.
- View assigned targets/KPIs.
- View personal performance summaries.
- Manage authorized operational records relevant to the role.
- Receive managerial instructions and notifications.

The earlier project design specifically identified BDE performance around activity volume and measurable output against deadlines. gemini-chattings

11. Social Media Manager

Access Level

Limited Access

Scope

Assigned social-media work and authorized content operations

Responsibilities

- View assigned campaigns/tasks.
- Manage assigned content work.
- Update progress.
- Record work notes.
- Upload drafts and supporting material.
- Submit content for review.
- Respond to correction requests.
- Track revision cycles.
- Record effort/time.
- View personal/team content-related KPIs where authorized.

For your analytical layer, useful measures could include:

```
Assignment
   ↓
First Draft
   ↓
Revision
   ↓
Approval
   ↓
Publication/Completion
```

The project discussion already identified delivery time and revision loops as meaningful metrics for design/social-media work. gemini-chattings

---

12. Graphic Designer

Access Level

Limited Access

Scope

Assigned creative work

Responsibilities

- View design assignments.
- Access task instructions.
- Execute assigned design work.
- Upload drafts.
- Record progress.
- Add work notes.
- Upload supporting files.
- Submit work for review.
- Receive feedback.
- Make requested corrections.
- Resubmit work.
- Track own work history.

A designer therefore follows:

```
Assigned
   ↓
Work
   ↓
Draft
   ↓
Review
   ↓
Changes Requested?
   ├── Yes → Correct → Resubmit
   └── No → Approved → Completed
```

This fits directly with your already established task-execution and submission workflow.

---

13. Admin

Access Level

Limited Administrative Access

This is where your distinction between **Admin and Super Admin** becomes especially important.

Your project already defines Admin-side functions such as user management, location/attendance control, global task management, system settings, security/audit viewing and announcements. gemini-chattings

The Admin can perform **routine administrative operations**, but should not control the platform's fundamental security policy.

Admin responsibilities

User administration

- Create authorized user accounts.
- Update user information.
- Activate/deactivate accounts.
- Assign permitted roles.
- Handle routine access requests.
- Reset credentials according to policy.

Operational administration

- Maintain location information.
- Manage attendance-related operational records.
- Monitor organizational tasks.
- Publish authorized announcements.

Audit visibility

- View relevant activity logs.
- Review modification history.
- Investigate routine administrative events.

Administrative controls

- Manage approved notification templates.
- Manage approved reporting schedules.
- Maintain routine system information.

Admin restrictions

The Admin should not independently:

- redefine the entire role hierarchy
- create unrestricted privileges
- modify core security policies
- disable critical audit controls
- perform unrestricted platform-wide overrides
- remove evidence of administrative activity

That is where the Super Admin begins.

14. Super Admin

Access Level

Higher-Level Access

Scope

Entire platform

This is your **highest technical authority**.

Responsibilities

Identity and authorization

- Define system roles.
- Define permissions.
- Modify permission structures.
- Configure role hierarchy.
- Manage privileged access.

System administration

- Configure platform-wide settings.
- Manage critical integrations.
- Manage system-level configuration.
- Perform emergency administrative actions.
- Control platform-wide operational policies.

Security

- Access comprehensive audit information.
- Investigate security-related events.
- Manage privileged access controls.
- Review administrative changes.
- Enforce security policies.

Global operations

- Perform authorized platform-wide interventions.
- Override restricted workflows under controlled conditions.
- Resolve exceptional administrative situations.

Crucial principle

Every privileged action taken by the Super Admin should itself be **auditable**.

That gives you:

```
Super Admin
     ↓
Privileged Action
     ↓
Security Check
     ↓
Action Executed
     ↓
Audit Record
```

The system should never create the idea of:

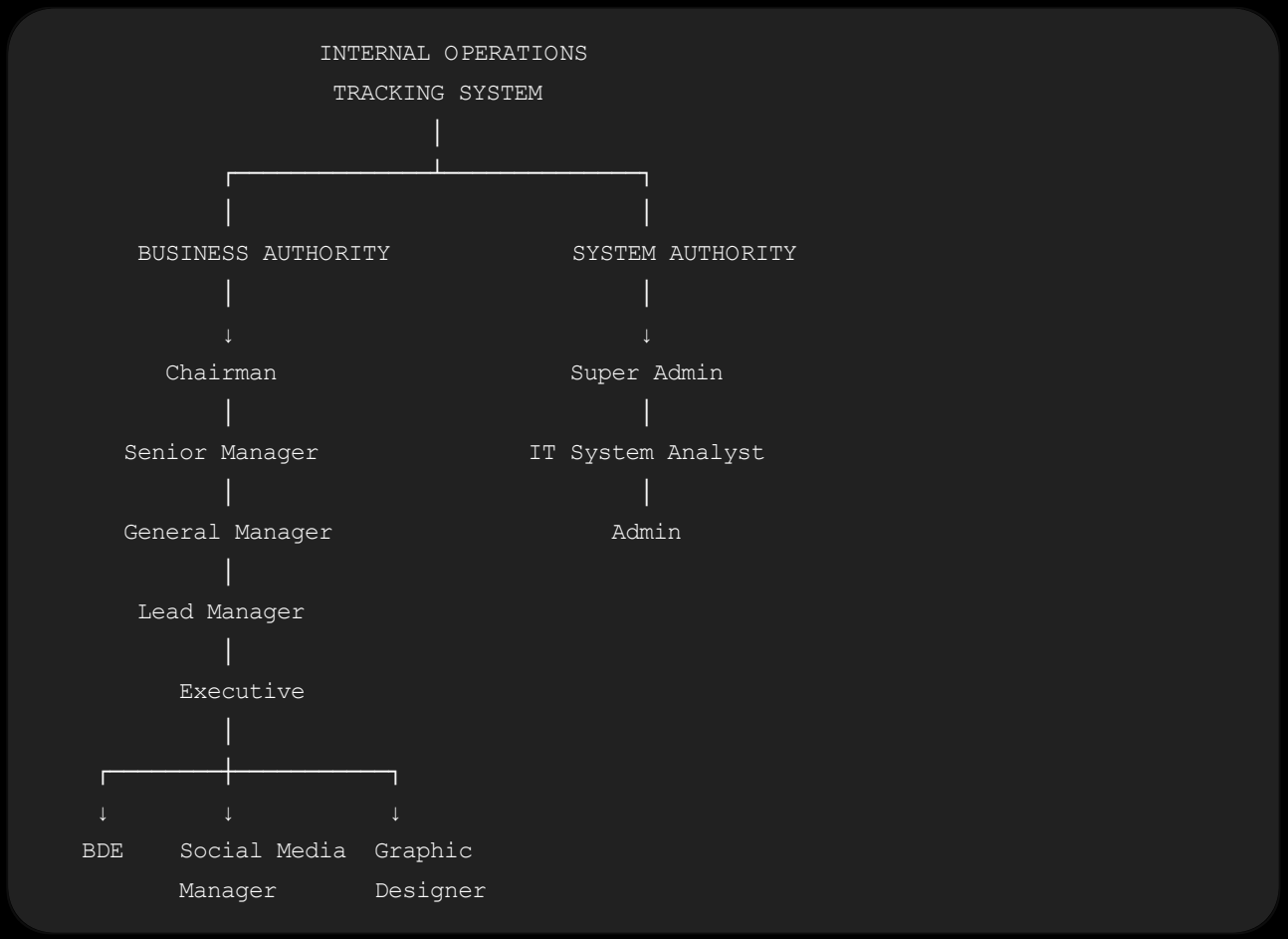
> "Super Admin can do anything and nobody can see it."

That conflicts with the accountability objective of your system and with separation-of-duty principles. NIST Publicati...

---

15. The Role Model in One View

I would ultimately describe your system like this:



But **this is not the permission hierarchy**.

It is the **responsibility hierarchy**.

The authorization engine should separately determine what each role can actually access.

---

16. Recommended Technical RBAC Structure

For your actual system specification, I would model authorization with four dimensions:

Role

```
Chairman
Senior Manager
General Manager
Lead Manager
Executive
IT System Analyst
BDE
Social Media Manager
Graphic Designer
Admin
Super Admin
```

Data Scope

```
OWN
TEAM
DEPARTMENT
BRANCH
ORGANIZATION
PLATFORM
```

Action

```
VIEW
CREATE
EDIT
SUBMIT
APPROVE
REJECT
REASSIGN
EXPORT
CONFIGURE
DELETE
OVERRIDE
```

Context

```
Business Operations
Technical Operations
Security
Administration
Analytics
```

This is substantially more robust than storing something simplistic such as:

```
user.role = "manager"
```

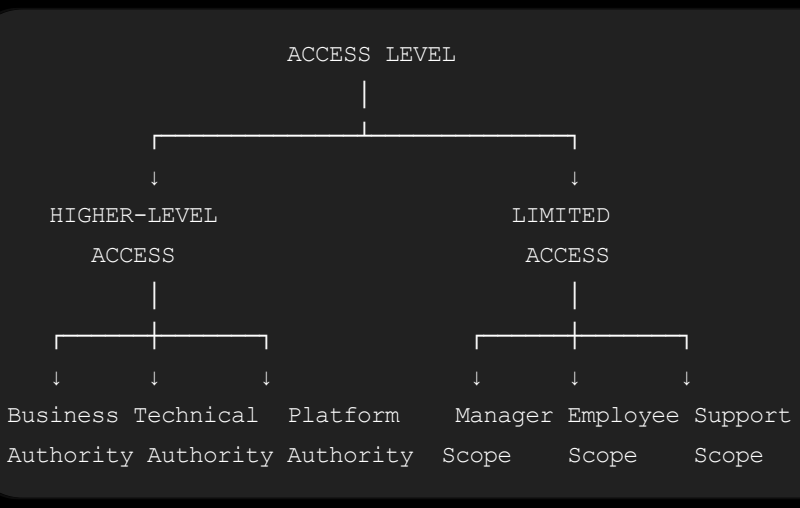
and then assuming every manager has identical permissions.

OWASP also recommends considering authorization beyond simple RBAC where necessary, including attributes and relationships, because access can depend on the user, resource and context. OWASP Cheat…

---

17. The most important correction to your two-level idea

I would keep your **two access levels**, but represent them as **bands**, not as the entire authorization model:



That lets you say something technically strong in your presentation:

> **"The system uses role-based access with hierarchical authority and scoped permissions. Access level determines the broad authority band, while role, data scope and permitted actions determine what the user can actually do."**

That is far more defensible than simply saying **"Chairman and Super Admin have high access, everyone else has low access."**

The guiding security principle

The system should grant **only the privileges necessary for a person's assigned responsibilities**, with stricter controls for privileged operations. That follows the principle of least privilege, while separation of duties prevents a single role from accumulating excessive authority. OWASP Cheat ... +1

For your project, that gives us a very clean conceptual chain:

```
AUTHENTICATION
      ↓
ROLE IDENTIFICATION
      ↓
ACCESS LEVEL
      ↓
ROLE PERMISSIONS
      ↓
DATA SCOPE
      ↓
ACTION AUTHORIZATION
      ↓
OPERATION
      ↓
AUDIT RECORD
```

And **that**, mate, is the access-control architecture I would build the Internal Operations Tracking System around.

can you give me detailed prompts for generating detail descriptions for google's stitch to create a detailed best ui designs for every dashboard for role specific from login itself lets say, login page is same for every one lets also say that there is no registration page, as a manual account is getting created in the system itself and those credentials are primarily used to create the profiles of others, okay? so based on these details and both pdf, detail and over our previous chatting details that include, authe-authorize layer, management layer, and the last one intelligence-analytical layer right? so based on those details on our above conversations.. generate detailed prompts..

Worked for 37s

Yes, mate. At this point I would **not give Stitch one giant prompt and hope it figures out the whole product**. That usually produces inconsistent screens.

The better approach is:

1 master product/design prompt → 1 common login prompt → 11 role-specific dashboard prompts → refinement prompts for important workflows.

I'm basing this on the project PDF and the flowchart/reference images available in this conversation. The project material establishes the secure-login → location validation → audit → role routing sequence, the three major architectural layers, manager-led task operations, employee work execution, escalation, and audit/reporting. gemini-chattings +1

Google's current Stitch supports high-fidelity UI generation from natural-language and image context, iterative design/prototyping, and reusable design rules through `DESIGN.md`, so this staged approach fits how Stitch is designed to be used. Google Blog +1

0. MASTER PROMPT — Give This to Stitch First

Paste this as the **project foundation** before generating individual screens.

```
Design a complete enterprise-grade internal web application called:

INTERNAL OPERATIONS TRACKING SYSTEM

This is an internal organizational operations platform, NOT a public SaaS product,

The product is used by an organization to securely authenticate users, control acce

The system is organized conceptually into three major layers:

1. AUTHENTICATION & AUTHORIZATION LAYER
2. MANAGEMENT LAYER
3. INTELLIGENCE & ANALYTICAL LAYER

================================================
1. AUTHENTICATION & AUTHORIZATION
================================================

There is NO public registration page.

All user accounts are created manually inside the system by authorized administrati

The account creation process creates an internal user profile and associates:
- employee identity
- designation
- organizational location
- team/department where applicable
```

- access level
- system role
- account status
- permitted permissions

The login screen is COMMON to every user.

Do NOT create separate login pages for Chairman, Manager, Employee, Admin, etc.

The login process visually represents:

User opens system
→ enters credentials
→ credentials are validated
→ location/session context is validated
→ login activity is recorded
→ system determines effective role and permissions
→ user is routed to the appropriate role-specific dashboard

Do NOT ask the user to choose their role on the login page.

The system determines role and authorization from the user's account configuration.

If a user has multiple authorized roles/designations, provide a subtle post-login v

Unknown or unusual login locations should be represented as an operational securit

==========
2. ACCESS MODEL
==========

Use two broad access bands:

HIGHER-LEVEL ACCESS
LIMITED ACCESS

However, do not visually imply that Limited Access means "unimportant".

Actual authority depends on:
- role
- data scope
- permitted actions
- organizational responsibility
- authorization rules

Use role-based access control concepts.

The interface should hide or disable controls that the current role is not authoriz

==========
3. ORGANIZATIONAL ROLES
==========

Design role-specific interfaces for:

1. Chairman
2. Senior Manager
3. General Manager
4. Lead Manager
5. Executive
6. IT System Analyst
7. Business Development Executive
8. Social Media Manager
9. Graphic Designer
10. Admin
11. Super Admin

Do not make these eleven dashboards look like eleven unrelated products.

Use one consistent application shell and design system, while changing:
- navigation
- widgets
- metrics
- actions
- data visibility
- approval capabilities
- management capabilities
- technical controls

according to role.

==========
4. MANAGEMENT MODEL
==========

Management responsibilities are separated from employee execution.

Managers manage:
- tasks
- priorities
- deadlines
- employee workload
- team availability
- leave/roster
- performance
- blockers
- escalations
- quality review
- approvals
- operational targets

Employees execute assigned work, update progress, record work activity, submit work

Administrative users manage organizational/system-level operations within their aut

Do not turn every role into an administrator.

==
5. TASK LIFECYCLE
==

Tasks follow a controlled operational lifecycle.

Core reporting states include:

ASSIGNED
PENDING
OVERDUE
REASSIGNED
COMPLETED

Do not treat REASSIGNED as a final state.

Task execution may contain operational micro-states such as:
- In Progress
- Blocked
- Waiting for Review
- Approval / Review
- Issue Resolved
- Done / Closed

The interface must distinguish:
- task status
- task execution state
- work activity
- review outcome
- audit history

==
6. EMPLOYEE WORK EXECUTION
==

Employee task execution should support:

- task acknowledgement
- starting work
- progress updates
- work notes
- time/effort entries
- supporting documents/evidence
- blocker reporting
- issue tickets
- manager intervention
- escalation when unresolved
- submission for review
- correction cycle

- resubmission
- final approval
- closure

Manager rejection must not simply terminate the task.

If changes are requested:
Manager feedback
→ Employee returns to task execution
→ Work is corrected
→ Work is resubmitted
→ Manager reviews again

==========
7. OVERDUE LOGIC
==========

Overdue is an exception state.

Do NOT make:

Overdue
→ Continue Working

the only path.

Instead visually support:

Overdue
→ Manager reviews overdue task
→ Manager intervention / resolution
→ Continue / extend deadline / reassign / escalate
→ Task returns to the operational workflow

Use appropriate warning/attention states without making the interface visually alar

==========
8. RECORD WORK ACTIVITY
==========

The employee can record:

Progress Update
Work Notes
Time-Spent / Effort Entry
Supporting Documents / Evidence

These are activity records, not independent task statuses.

The system should visually show that multiple work-activity records can exist durir

Activity history should be chronological.

```
==================================================
9. AUDITABILITY
==================================================

Important operational actions should be traceable.

Provide a reusable audit/history component capable of showing:

- actor
- timestamp
- action
- previous value
- new value
- reason/comment where applicable
- related task/user/request
- system-generated vs user-generated event

Audit history is primarily presented as read-only evidence.

Do not make audit logs look like ordinary editable data tables.

==================================================
10. INTELLIGENCE & ANALYTICAL LAYER
==================================================

The third layer is called exactly:

INTELLIGENCE & ANALYTICAL LAYER

It analyzes operational information generated by the system.

It may provide:

- performance analytics
- task analytics
- workload analytics
- attendance/workforce analytics
- trend and pattern analysis
- exception and risk analysis
- predictive insights where supported
- automated reports
- management insights

Do not turn this into meaningless decorative charts.

Every chart should answer a business question.

Examples:
- Are tasks being completed on time?
- Where are bottlenecks?
- Is workload uneven?
```

- Which work is repeatedly overdue?
- Which tasks are repeatedly reassigned?
- How is team performance changing?
- Which issues require management attention?

==
11. COMMON ENTERPRISE UI SHELL
==

Create a consistent desktop-first enterprise workspace.

Use:

- left navigation sidebar
- compact global header
- organization/system identity
- current user and role
- current branch/location
- availability indicator
- global notifications
- global search
- breadcrumbs
- contextual actions
- responsive content grid

Recommended layout:
- 12-column desktop grid
- desktop-first design around 1440px
- responsive tablet/mobile adaptation
- consistent spacing system
- accessible typography
- strong visual hierarchy
- dense but readable information presentation

Use a professional enterprise visual language:
- restrained
- precise
- modern
- data-oriented
- trustworthy
- calm
- operational

Avoid:
- marketing-style hero sections
- huge decorative illustrations
- excessive gradients
- excessive glassmorphism
- oversized empty cards
- childish dashboard widgets
- unnecessary 3D charts
- excessive rounded containers

- visually noisy neon aesthetics

Use Material/enterprise UI principles as a foundation but create a distinctive inte

Use semantic status indicators consistently for:
- normal
- pending
- warning
- overdue
- blocked
- success
- critical

Ensure status meaning is not conveyed by color alone.

==========
12. DATA-DENSE BUT EXECUTIVE-FRIENDLY
==========

The product must satisfy two audiences:

Senior management needs fast understanding.

Operational users need detailed controls.

Therefore:
- executives receive high-level summaries first
- managers receive actionable operational detail
- employees receive task-focused execution tools
- administrators receive controlled configuration tools
- technical roles receive diagnostics and system information

Use progressive disclosure.

Do not put every function on the first screen.

==========
13. INTERACTION DESIGN
==========

Use:
- drawers for quick inspection
- modals for focused confirmations
- side panels for contextual details
- tabs for related information
- expandable audit history
- filterable tables
- saved views where useful
- clear empty states
- confirmation for destructive/high-impact actions
- inline validation
- toast notifications for completed actions

```
- persistent status visibility

Tables should support:
- search
- filters
- sorting
- pagination
- column visibility
- export where authorized

================================================
14. IMPORTANT UX PRINCIPLE
================================================

The design should look "complex underneath, simple on top."

A Chairman should understand the dashboard within seconds.

A Manager should be able to operate their team without opening ten screens.

An Employee should immediately know:
- what requires attention
- what task they are working on
- what is due
- what was requested from them
```

---

1. COMMON LOGIN PAGE PROMPT

```
Using the established Internal Operations Tracking System design system, create the

Create a professional enterprise login screen.

There is NO REGISTRATION option anywhere on this screen.

The user already has an account created internally by an authorized administrator.

Include:
- system logo/name
- "Internal Operations Tracking System"
- username / employee ID field
- password field
- password visibility control
- sign-in button
- controlled password recovery/help mechanism that directs the user to the authoriz
- subtle security/privacy statement

Do NOT include:
- Create Account
- Sign Up
```

```
- Public registration
- social login
- unnecessary marketing copy

Show realistic interaction states:
1. empty login
2. invalid credentials
3. successful credentials
4. location verification in progress
5. recognized office/approved location
6. unknown or unusual location flagged for review
7. successful authorization
8. account disabled/restricted

After successful authentication, the system silently determines:
- user identity
- effective role
- access level
- location context
- permissions

Do not ask the user to choose Chairman/Manager/Employee on this page.

Make the page feel secure, serious, corporate and trustworthy.

Use the same visual language that will be used throughout the dashboards.
```

---

2. CHAIRMAN DASHBOARD

```
Create the Chairman Dashboard for the Internal Operations Tracking System.

The Chairman is a higher-level business authority.

This is an executive oversight dashboard, NOT an administration console.

Primary purpose:
Give the Chairman a fast, organization-wide understanding of operational health wit

Navigation should emphasize:

Overview
Operational Performance
Global Work
Escalations
People Overview
Analytics & Insights
Reports
Announcements
Audit Summary
```

```
Profile / Security

Top-level dashboard should show:

- organization operational health
- active work count
- completed work
- pending work
- overdue work
- critical escalations
- team/department performance
- workload distribution
- attendance/availability summary
- target achievement
- major operational exceptions
- recent management actions

Create an executive insight panel:
"Requires Attention"

Examples:
- overdue work
- repeated task reassignment
- unresolved blockers
- unusual attendance patterns
- declining performance trend
- high workload concentration

Create an "Organizational Pulse" analytical section with:
- task completion trend
- SLA/on-time trend
- workload trend
- attendance trend
- target achievement trend

Create a "Global Work Monitor" table:
Task
Owner
Team
Priority
Due Date
Status
Risk/Attention
Last Activity

Allow the Chairman to inspect work and intervene only where their authorization per

Make interventions visibly important and require confirmation.

Do NOT fill the Chairman dashboard with:
- routine password resets
- everyday task creation
```

- low-level user management
- technical diagnostics

---

3. SENIOR MANAGER DASHBOARD

Create the Senior Manager Dashboard.

The Senior Manager has higher-level operational oversight across multiple teams or

Purpose:
Monitor performance, workload, escalations and execution quality across the Manager

Primary navigation:

Overview
Teams
Operational Work
Performance
Workload
Escalations
Approvals
Analytics
Reports
Notifications

Main dashboard:

- active teams
- team productivity
- tasks at risk
- overdue work
- pending approvals
- escalated blockers
- workload imbalance
- target achievement
- team attendance/availability summary

Create a Team Comparison section showing:
team workload
completion rate
on-time completion
overdue work
reassignment frequency
open issues

Create an Escalation Center:
severity
issue

```
affected team
responsible manager
age
current action
escalation level

Create a Performance Trend panel.

Create an Approval Queue for decisions requiring Senior Manager authority.

The dashboard should be operational and analytical, not purely executive and not te

Provide drill-down from summary metrics into team-level operational records.
```

---

4. GENERAL MANAGER DASHBOARD

```
Create the General Manager Dashboard.

The General Manager requires broad business/branch operational visibility and decis

The dashboard should sit between executive oversight and day-to-day team operations

Primary navigation:

Business Overview
Teams & Departments
Targets
Operational Work
Performance
Approvals
Escalations
Analytics
Reports
Communication

Main dashboard should emphasize:

- overall operational target achievement
- department/team performance
- active work
- overdue workload
- workload distribution
- productivity trend
- attendance/availability health
- unresolved operational issues
- pending managerial actions

Create:
```

```
1. Business Performance Overview
2. Target Monitoring
3. Department Performance
4. Operational Exceptions
5. Workload Distribution
6. Escalation Summary
7. Analytical Insights

Provide drill-down paths from business metrics to the actual operational records.

Do not make this dashboard a copy of the Chairman dashboard.

Chairman:
organization-level oversight and major decisions.

General Manager:
broader operational/business management and coordination.

Make these differences visible through navigation, metrics and actions
```

---

5. LEAD MANAGER DASHBOARD

This is the most important management dashboard in your system.

Your project already established manager-side task origination, deadline/SLA definition, bandwidth checking, assignment, notification, live progress, overdue handling, QA and audit linkage. gemini-chattings

```
Create the Lead Manager Dashboard for the Internal Operations Tracking System.

This is the primary team-operations workspace.

The Lead Manager directly manages assigned employees, operational work and team per

Primary navigation:

Dashboard
Task Management
Team
Workload
Live Targets
Attendance & Leave
Blockers & Escalations
Approvals / QA
Performance
Reports
Communication
Notifications
```

The home dashboard should show:

- team availability
- active tasks
- pending work
- overdue tasks
- tasks awaiting review
- blocked tasks
- today's targets
- workload distribution
- leave requests
- pending interventions
- team performance indicators

Create a "Live Team Status" panel showing:
Employee
Role
Availability
Current Task
Task Status
Priority
Due Time
Current Workload

Availability states may include:
Available
Busy
On Break
In Meeting
In Training
Do Not Disturb
Working Remotely
On Leave
Absent
Offline

Create a "Manager Action Center":
- tasks requiring review
- overdue tasks
- blockers requiring intervention
- leave requests
- reassignment decisions
- change requests
- escalations

Task Management must support this conceptual workflow:

Create Task
→ Define task and deliverables
→ Set priority
→ Set due date / SLA
→ Assess employee bandwidth

```
→ Assign employee
→ Notify employee
→ Monitor execution
→ Handle overdue/blocker conditions
→ Review submission
→ Approve or request changes
→ Complete and close
→ audit action

Do not make overdue a passive status.

Create an Overdue Intervention section.

Reassignment must be a controlled workflow:
select task
→ inspect reason
→ choose new employee
→ record reassignment reason
→ notify new owner
→ preserve reassignment history

Create Team Workload visualization.

Create Live Target Monitoring.

Create Leave & Roster Management.

Create Blocker / Escalation Center.

Every high-impact managerial action should show a contextual audit/history drawer.

Make this dashboard the most operationally rich dashboard in the management layer.
```

---

6. EXECUTIVE DASHBOARD

Here "Executive" means the limited-access operational role, not executive management.

```
Create the Executive Dashboard.

This is a limited-access employee workspace focused on personal execution and assig

The user should immediately understand:

"What do I need to work on?"
"What is due?"
"What needs my attention?"
"What have I completed?"
"Am I blocked on anything?"

Primary navigation:
```

```
My Dashboard
My Tasks
Task Execution
Work Activity
Issues / Blockers
Requests
Attendance
Notifications
My Performance
Profile

Dashboard sections:

- today's priorities
- active tasks
- due today
- upcoming deadlines
- overdue personal tasks
- waiting for review
- requested changes
- blocked tasks
- current availability
- today's work activity
- personal performance summary

Provide a "Continue Work" action for active tasks.

Show task cards/table with:
task title
priority
due date
status
progress
last update
next action

Make the task execution interface more prominent than analytics.

Do not show team management controls.
Do not show employee reassignment controls.
Do not show administrative configuration.
```

---

7. IT SYSTEM ANALYST DASHBOARD

```
Create the IT System Analyst Dashboard.

This is a technically privileged operational workspace.
```

The IT System Analyst monitors and supports the health, reliability and technical c

Do not make this dashboard a generic employee dashboard with a few technical cards.

Primary navigation:

Technical Overview
System Health
Services / APIs
Incidents
Authentication & Sessions
Access Support
Audit & Security
Deployments / Changes
Technical Reports

Main dashboard:

- application health
- service availability
- API health
- database/connectivity status
- active incidents
- unresolved technical issues
- authentication anomalies
- recent system changes
- failed operations
- technical workload

Create a System Health section with service status:
service
state
last check
response time
issue count

Create an Incident Center:
incident ID
severity
affected service
opened
owner
status
resolution progress

Create Authentication & Session monitoring:
successful sessions
failed authentication
unusual location/session events
recent privileged events

```
Create an Audit Explorer with read-only historical investigation.

Technical controls should be visually separated from business management controls.

The IT analyst must not automatically receive business decision authority merely be

Where a system action requires higher authorization, visibly show:
"Authorization Required"
```

---

8. BUSINESS DEVELOPMENT EXECUTIVE DASHBOARD

```
Create the Business Development Executive Dashboard.

This is a limited-access operational dashboard for business-development work.

The dashboard should emphasize target execution, assigned work, activity logging ar

Primary navigation:

Dashboard
My Tasks
Targets
Lead / Activity Work
Follow-ups
Work Activity
Issues
Attendance
Notifications
Performance
Profile

Main metrics:

- today's target
- completed target
- remaining target
- active tasks
- overdue tasks
- follow-ups due
- completed activities
- personal performance trend

Create a Target Progress module:
daily target
completed
remaining
```

```
progress
deadline

Create an Activity Workspace for recording work performed.

Create a Follow-up Queue:
priority
contact/opportunity
next action
due time
status

Create blocker reporting.

Use the common task lifecycle and work-activity model:
progress updates
work notes
time/effort
supporting evidence
task submission

Do not expose team-management tools or administrative controls.
```

---

9. SOCIAL MEDIA MANAGER DASHBOARD

```
Create the Social Media Manager Dashboard.

Design this as an operational content-management workspace inside the Internal Ope

Primary navigation:

Dashboard
Content Pipeline
Campaigns
Assigned Tasks
Content Calendar
Reviews & Approvals
Work Activity
Analytics
Issues
Notifications
Profile

Main dashboard:

- content awaiting creation
- content in progress
- content awaiting review
- changes requested
```

```
- approved content
- overdue content
- upcoming publishing deadlines
- active campaigns

Create a visual content pipeline:

Brief
→ Drafting
→ Internal Review
→ Changes Requested
→ Resubmission
→ Approved
→ Completed / Published

Each item should show:
owner
campaign
platform
deadline
status
revision count
last update

Create a Content Calendar.

Create a Review Workspace where manager feedback is clearly attached to the relevar

Track work activity:
progress updates
work notes
effort
supporting files

Create analytics focused on:
delivery timeliness
revision frequency
campaign workload
content completion trend

Do not turn this into a public-facing social media dashboard.
```

---

10. GRAPHIC DESIGNER DASHBOARD

```
Create the Graphic Designer Dashboard.

This should function as a professional internal creative workbench.

Primary navigation:
```

Dashboard
My Tasks
Creative Queue
Design Briefs
Files & Versions
Reviews
Work Activity
Issues
Attendance
Performance
Notifications
Profile

Home dashboard:

- active design tasks
- due today
- upcoming deadlines
- awaiting review
- revision requested
- approved designs
- overdue designs
- current workload

Create a Creative Queue with:
priority
brief
client/project
deadline
status
revision count

Create a Design Brief view:
objective
requirements
deliverables
reference assets
deadline
assigned manager

Create version-aware file presentation:
Version 1
Version 2
Version 3

Show who uploaded each version and when.

Review workflow:
Draft
→ Submit for Review
→ Manager Feedback

```
→ Changes Requested
→ Revised Version
→ Resubmit
→ Approved

Make file evidence and version history highly visible.

Include Record Work Activity:
progress update
work note
time/effort
supporting document

Do not provide task reassignment, team management or system administration.
```

---

11. ADMIN DASHBOARD

This should remain **limited administrative control**, not the Super Admin.

Your existing project groups Admin functionality around user/access, location/attendance, global task management, settings, audit viewing and announcements. gemini-chattings

```
Create the ADMIN DASHBOARD.

This is a LIMITED ADMINISTRATIVE workspace for routine organizational and system ac

Primary navigation:

Dashboard
User Management
Location & Attendance
Global Task Management
System Settings
Security & Audit
Announcements
Reports

Dashboard summary:

- active users
- pending account actions
- attendance exceptions
- location flags
- active organizational work
- overdue organizational work
- pending administrative requests
- recent audit events

USER MANAGEMENT
```

Create a professional user-management table:
employee ID
name
designation
role
access level
location
team
account status
last activity

Provide controlled actions:
Create User
View Profile
Edit User
Activate / Deactivate
Reset Credentials where authorized

Create User should be a structured form:
identity
designation
organizational location
team
system role
access level
account status

Do NOT include public registration.

LOCATION & ATTENDANCE

Show:
office locations
attendance records
availability
location exceptions
attendance flags

GLOBAL TASK MANAGEMENT

Allow authorized administrative oversight of organization-wide work.

Show:
task
team
owner
priority
deadline
status
risk
last activity

```
Support authorized:
view
edit
cancel
intervention
approval
cleanup/archive where permitted

Every high-impact action requires confirmation and creates an audit event.

SYSTEM SETTINGS

Keep these controlled and limited.

Include:
designations
report schedules
notification templates
approved operational settings

SECURITY & AUDIT

Read-only by default.

Provide:
activity log
modification history
login history
role/permission changes
system actions

Do NOT allow normal audit records to be edited.

ANNOUNCEMENTS

Create:
draft
choose audience
preview
publish

Audiences may include:
all users
specific roles
specific teams

Make the Admin workspace powerful but clearly below the platform governance authori
```

---

12. SUPER ADMIN DASHBOARD

Create the SUPER ADMIN DASHBOARD.

This is the highest technical and platform-governance workspace.

It should look noticeably different in authority from the normal Admin dashboard, b

Primary navigation:

Platform Overview
Identity & Access
Roles & Permissions
Organization Configuration
System Configuration
Security Center
Audit & Compliance
Integrations
System Health
Global Operations
Reports

Main dashboard:

- total active users
- active roles
- privileged accounts
- authentication health
- security events
- system health
- active incidents
- configuration changes
- integration health
- audit events
- organization-wide operational health

IDENTITY & ACCESS

Provide:
user accounts
role assignment
access levels
permissions
account lifecycle
privileged users

ROLES & PERMISSIONS

Create a permission-management interface based on:

Role
→ Data Scope
→ Actions

→ Workflow Rights
→ Administrative Rights

Support matrix-style viewing and editing.

Do not make permission changes casual.

Use:
Review
→ Confirm
→ Apply
→ Audit

GLOBAL CONFIGURATION

Show:
global role definitions
system-wide settings
notification framework
reporting configuration
integration configuration
security policies

SECURITY CENTER

Show:
authentication anomalies
privileged actions
suspicious sessions
critical configuration changes
security events

AUDIT & COMPLIANCE

Provide an immutable/read-only audit explorer.

Show:
actor
action
resource
timestamp
old value
new value
reason
source

GLOBAL OPERATIONS

Provide authorized platform-wide operational intervention capabilities.

Any override or privileged action must:
- explain consequence

```
- require confirmation
- create an audit record
```

---

13. INTELLIGENCE & ANALYTICAL DASHBOARD PROMPT

I would actually give Stitch this as a **separate design prompt**, because this is your third architectural layer.

```
Create the INTELLIGENCE & ANALYTICAL CENTER for the Internal Operations Tracking Sy

This is not another operational dashboard.

Its purpose is to transform operational data into management intelligence.

Use information generated from:
- tasks
- task states
- deadlines
- work activity
- time/effort
- supporting evidence
- attendance
- availability
- requests
- blockers
- escalations
- approvals
- reassignment history
- audit activity

Organize the analytics center into:

1. Performance Analytics
2. Task & Workload Analytics
3. Attendance & Workforce Analytics
4. Trend & Pattern Analysis
5. Exception & Risk Analysis
6. Predictive Insights
7. Automated Reporting

TOP AREA

Create an "Operational Intelligence Summary":

Overall operational health
Tasks at risk
```

Overdue trend
Workload imbalance
Escalation trend
Target achievement
Attendance health

TASK ANALYTICS

Show:
completion rate
on-time rate
pending rate
overdue rate
reassignment frequency
average completion duration

WORKLOAD ANALYTICS

Show:
workload by team
workload by employee
capacity vs assigned work
concentration of active work
potential overload

PERFORMANCE ANALYTICS

Show:
performance trends
target attainment
completion quality
review cycles
revision frequency

EXCEPTION & RISK

Show:
overdue clusters
repeated reassignment
repeated blockers
unresolved escalations
abnormal operational patterns

TREND ANALYSIS

Provide time-range controls:
Today
7 Days
30 Days
Quarter

Allow comparison with a previous period.

```
PREDICTIVE INSIGHTS

Only show predictive information as an analytical signal.

Examples:
Potential deadline risk
Potential workload overload
Potential recurring bottleneck

Do not fabricate scientific certainty.

AUTOMATED REPORTS

Show:
report type
frequency
audience
last generated
next generation

Reports should be role-scoped.

Chairman:
executive summaries

Management:
team/department operational reports

Employee:
personal work summary

Use charts deliberately.

Every visualization must answer a business question.
Avoid decorative charts.
```

---

14. MANUAL ACCOUNT CREATION — ADMIN/SUPER ADMIN SCREEN

Because your system has **no registration**, this deserves its own excellent UI.

```
Design the "Create Internal User Profile" screen.

This is NOT a registration page.

It is an authorized administrative workflow used to create an employee's account ar

Create a structured multi-section form:
```

SECTION 1 — Identity
Employee ID
Full Name
Official Email
Contact Information

SECTION 2 — Organizational Information
Designation
Department / Team
Location / Branch
Reporting Manager

SECTION 3 — Authorization
System Role
Access Level
Data Scope
Permitted Actions

SECTION 4 — Account
Username / Employee ID
Temporary Password / Credential Setup
Account Status
Activation Date

SECTION 5 — Review
Show a permission summary before account creation.

Example:

User:
John Doe

Designation:
Graphic Designer

Access:
Limited

Scope:
Assigned Work / Personal Records

Allowed:
View Tasks
Update Progress
Upload Evidence
Submit Work

Not Allowed:
Assign Tasks
Approve Work
Manage Users

```
Modify System Settings

Final flow:

Enter details
→ Validate
→ Review permissions
→ Confirm account creation
→ Generate account
→ Record administrative action in audit history

Make it feel like controlled enterprise provisioning, not a simple signup form.
```

---

15. COMMON TASK DETAIL SCREEN

This screen should work across Manager and Employee contexts, but show different controls according to authorization.

```
Create a professional TASK DETAIL workspace for the Internal Operations Tracking Sy

Header:
Task ID
Task Title
Priority
Current Status
Due Date
SLA indicator
Assigned Employee
Assigned Manager

Sections:

1. Task Overview
Description
Deliverables
Instructions
Priority
Deadline

2. Current Execution State
Assigned
Pending
Overdue
Reassigned
Completed

Also show operational micro-state:
In Progress
Blocked
```

```
Waiting for Review
Approval
Issue Resolved
Done / Closed

3. Progress
Current percentage
latest update
progress history

4. Work Activity
Progress Updates
Work Notes
Time/Effort Entries
Supporting Documents

5. Issues / Blockers
Issue description
severity
reported by
date
manager action
resolution
escalation state

6. Submission / Review
submission timestamp
review status
manager feedback
requested changes
resubmission history

7. Audit History
chronological immutable activity history

Use a split layout where the task summary remains visible while the user explores d

Differentiate Employee and Manager controls using authorization:
Employee:
execute/update/submit/report issue

Manager:
assign/reassign/review/approve/request changes/intervene

Do not show unauthorized actions.
```

---

16. COMMON "AUDIT HISTORY" COMPONENT PROMPT

Design a reusable enterprise AUDIT HISTORY component.

This component will be embedded in:
- tasks
- user profiles
- permissions
- attendance
- requests
- system settings
- administrative actions

Show events chronologically.

Each event should display:

Actor
Role
Timestamp
Action
Object/resource
Previous value
New value
Reason/comment
Source:
User Action / System Action

Examples:

Employee:
Status changed
Pending → Waiting for Review

Employee:
Progress updated
65% → 80%

Manager:
Feedback added

Manager:
Task reassigned
Employee A → Employee B

Admin:
User activated

Super Admin:
Permission changed

System:
Task automatically marked Overdue

```
Make the component read-only.

Allow filters:
date
actor
action type
resource

Do not visually suggest that audit records can be edited.
```

---

17. FINAL STITCH REFINEMENT PROMPT

Once Stitch generates your screens, use this to clean them up rather than regenerating everything.

```
Review the current Internal Operations Tracking System design as a senior enterpris

Do NOT redesign everything from scratch.

Audit the current screens for:

1. Role clarity
2. Information hierarchy
3. Navigation consistency
4. Excessive visual density
5. Excessive whitespace
6. Inconsistent status terminology
7. Missing workflow states
8. Missing audit visibility
9. Incorrect privilege exposure
10. Weak management decision points
11. Poor task execution UX
12. Poor accessibility
13. Inconsistent table/filter behavior
14. Inconsistent modal/drawer behavior
15. Inconsistent typography and spacing

Preserve the existing design language.

Make the product:
- cleaner
- more enterprise-grade
- more consistent
- easier to understand
- more information-dense where operationally necessary
- more visually calm

Do not add decorative UI merely to make the screen look impressive.
```

```
Every component must have a functional purpose.

Ensure Chairman, Senior Manager, General Manager, Lead Manager, Executive, IT Syste
```

The important part: don't ask Stitch to generate all 11 simultaneously

I'd build the project in this order:

```
COMMON DESIGN SYSTEM
        ↓
COMMON LOGIN
        ↓
APP SHELL
        ↓
SUPER ADMIN
        ↓
ADMIN
        ↓
CHAIRMAN
        ↓
GENERAL MANAGER
        ↓
SENIOR MANAGER
        ↓
LEAD MANAGER
        ↓
IT SYSTEM ANALYST
        ↓
EXECUTIVE
        ↓
BDE
        ↓
SOCIAL MEDIA MANAGER
        ↓
GRAPHIC DESIGNER
        ↓
TASK DETAIL
        ↓
TASK EXECUTION
        ↓
AUDIT HISTORY
        ↓
INTELLIGENCE & ANALYTICAL CENTER
```

The reason I would start with **Super Admin → Admin → management → operational roles** is that your authorization model becomes the foundation for the rest of the UI. Your

earlier project material already frames the system around RBAC, scoped actions, workflow rights and administrative rights. gemini-chattings

And I would **import your existing flowchart/reference images into the Stitch project as visual context**, rather than explaining the entire architecture only through text; Stitch explicitly supports image/context-driven UI generation and iterative refinement. Google Blog

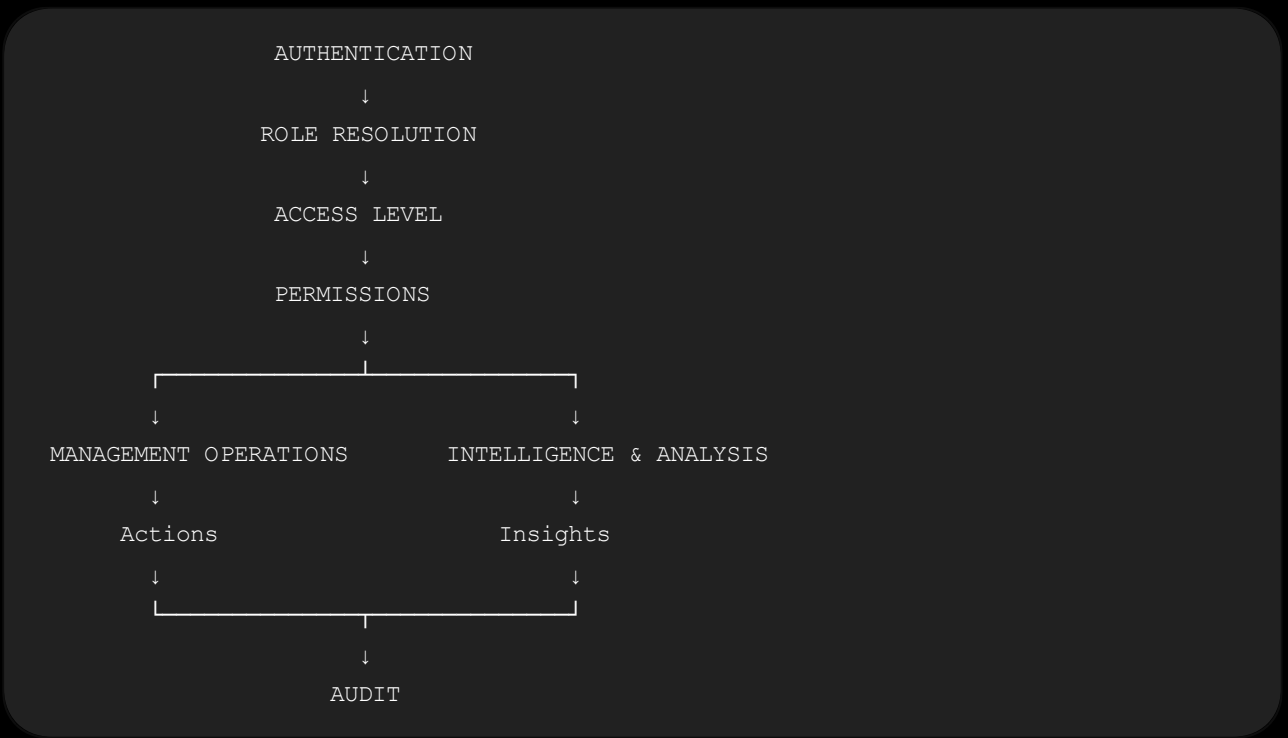
One architectural decision I would lock before generating

Keep these three concepts separate throughout every screen:

```
DESIGNATION
Chairman / Manager / BDE / Designer / IT Analyst
        ↓
AUTHORIZATION
Higher-Level / Limited
        ↓
PERMISSIONS
View / Create / Edit / Approve / Reassign / Configure / Override
```

That will prevent Stitch from making the common mistake of assuming **"higher job title = access to everything."**

Your current system is better modeled as:



That gives Stitch a **real product architecture to design against**, instead of just a list of dashboard widgets. gemini-chattings

I found **one project PDF (`gemini-chattings.pdf`) plus the uploaded flowchart/reference images** in the available files; I used those together with our

subsequent discussions for the prompts above.

given the roles like chairman, and all, lets say general-manager (here the system: divided this role into 4 chunks for managerial purposes, but these roles include : sr.manager, lead manager, executive, and the general mgr itself, so assuming general-manager is the main one), bdes, social-media manager, graphic designer are the actual employees right? so based on these roles i need to ask my manager every detailed description like what are the metrics, how they do the work, and what are the kpi, list out the detailed things that I need to ask my manager that might include in this system... do this.. apart from this.. my

Show more

Done, mate. I consolidated the **updated role model, access logic, lead-driven workflow, royalty factor, KPI/metric discovery questions, task/work-activity model, audit requirements, Intelligence & Analytical Layer, and the complete Stitch prompting framework** into one text file.

Download the complete Internal Operations Tracking System document

The important new addition is the **Lead → Conversion → Royalty → Analytics → Chairman visibility** chain, while keeping the exact royalty formula and attribution rules as questions for your Manager rather than inventing business rules. The existing project material supports the lead-driven operational context, role-based responsibilities, task/SLA/reassignment workflows, and audit/reporting concepts. gemini-chattings +2

internal_operations_tracking_system_manager_discove…
Document

Gonna use lovabale mate

Yep mate — **Lovable actually changes how I'd approach this.**

For Lovable, I'd move away from a pure **UI-generation prompt** and give it a **product + workflow + authorization specification**, because we want the generated app to have a coherent data model and working interactions, not just pretty dashboards.

The architecture we have is:

```
AUTHENTICATION & AUTHORIZATION
            ↓
      MANAGEMENT LAYER
            ↓
INTELLIGENCE & ANALYTICAL LAYER
```

And the business core is:

```
LEAD
 ↓
ASSIGNMENT / WORK
 ↓
ACTIVITY
 ↓
CONVERSION
 ↓
ROYALTY
 ↓
ANALYTICS
 ↓
MANAGEMENT DECISION
```

For Lovable, I would use the text file I prepared as the **requirements/reference document**, then give Lovable a carefully structured master prompt.

The first Lovable prompt I'd use

```
Build a production-oriented internal web application called:

INTERNAL OPERATIONS TRACKING SYSTEM

This is an internal business operations platform for a lead-driven organization.
It is not a public SaaS product and must not contain public registration.

The system has three conceptual layers:

1. Authentication & Authorization Layer
2. Management Layer
3. Intelligence & Analytical Layer
```

The application must use a consistent enterprise design system across all roles,
but the navigation, data visibility and available actions must change according
to the authenticated user's role and permissions.

==
AUTHENTICATION & ACCOUNT PROVISIONING
==

There is one common login page for all users.

Do NOT create separate login pages per role.

Do NOT create a public registration page.

User accounts are created manually by authorized administrators.

Account provisioning should support:

- employee identity
- employee ID
- designation
- department/team
- reporting manager
- branch/location
- system role
- access level
- account status
- credentials

Login workflow:

Login
→ Validate credentials
→ Validate session/location context
→ Record authentication event
→ Resolve user's effective role and permissions
→ Route to authorized workspace

The user must NOT manually select their role at login.

==
ROLES
==

Implement these roles:

Chairman
Senior Manager
General Manager
Lead Manager
Executive
IT System Analyst

Business Development Executive
Social Media Manager
Graphic Designer
Admin
Super Admin

General Manager is the main higher-level management role in the current organizational model.

Senior Manager, Lead Manager and Executive represent progressively scoped management/operational responsibility under that management structure.

Operational employees include:

- Business Development Executive
- Social Media Manager
- Graphic Designer
- IT System Analyst where applicable

Admin and Super Admin are administrative roles.

==========
ACCESS MODEL
==========

Use two broad access levels:

HIGHER-LEVEL ACCESS
LIMITED ACCESS

Do not interpret Limited Access as low importance.

Effective authorization must depend on:

- role
- access level
- organizational scope
- permitted actions

Support data scopes such as:

OWN
TEAM
DEPARTMENT
BRANCH
ORGANIZATION
PLATFORM

Support actions such as:

VIEW
CREATE

```
EDIT
SUBMIT
APPROVE
REJECT
REASSIGN
EXPORT
CONFIGURE
DELETE
OVERRIDE

Authorization must be enforced in the interface and backend.

Unauthorized actions must not simply be hidden visually;
they must also be blocked at the application/data layer.

==================================================
LAYER 2: MANAGEMENT
==================================================

Management responsibilities include:

- task management
- team management
- workload management
- attendance/availability
- leave and roster
- approvals
- performance monitoring
- blockers
- escalations
- communication
- operational targets

Employees execute assigned work and do not receive management controls.

==================================================
TASK MANAGEMENT
==================================================

Tasks support:

- task creation
- task description
- deliverables
- priority
- due date
- SLA
- assignment
- reassignment
- notification
- progress
- work activity
```

- blockers
- submission
- review
- corrections
- approval
- closure

Reporting states:

ASSIGNED
PENDING
OVERDUE
REASSIGNED
COMPLETED

Operational execution states:

IN PROGRESS
BLOCKED
WAITING FOR REVIEW
APPROVAL / REVIEW
ISSUE RESOLVED
DONE / CLOSED

REASSIGNED is a transition, not a terminal state.

OVERDUE is an exception condition.

Overdue workflow:

Overdue
→ Manager reviews
→ Manager action
→ Continue / Extend / Reassign / Escalate
→ Return to operational workflow

Manager rejection must include feedback/correction information.

Rejected work must return to employee execution and allow resubmission.

==========
EMPLOYEE WORK ACTIVITY
==========

Record Work Activity is separate from task status.

Support:

1. Progressive Update
2. Work Notes
3. Time-Spent / Effort Entry
4. Supporting Documents / Evidence

Each work activity record should retain:

- actor
- timestamp
- related task
- activity type
- content/value
- revision history where applicable

A task can contain multiple work activity records.

==========
BLOCKERS & ESCALATIONS
==========

Employees must be able to report blockers/issues.

Example:

Employee
→ Report Issue
→ Select issue type
→ Add description
→ Attach evidence
→ Notify Manager
→ Manager intervention
→ Resolved
→ Employee continues work

If the manager cannot resolve the issue:

Manager
→ Escalate
→ Higher authority
→ Resolution
→ Audit record

==========
LEAD-CENTRIC BUSINESS MODEL
==========

The organization operates around LEADS.

Implement a lead lifecycle that can support:

Lead Source
→ Lead Capture
→ Duplicate Check
→ Qualification
→ Assignment
→ Activity

→ Follow-up
→ Stage Progression
→ Conversion / Business Outcome
→ Royalty Eligibility
→ Royalty Record
→ Analytics

Do not invent the firm's royalty formula.

Royalty rules must be configurable because the actual business rules will be confirmed by management.

The system must support royalty-related fields including:

- related lead
- business outcome
- attribution
- royalty eligibility
- amount/value
- status
- recognition date
- adjustment/reversal history
- audit history

Support attribution questions such as:

- original lead owner
- current owner
- converting employee
- team attribution

These must be configurable rather than hard-coded.

==========
BDE WORKSPACE
==========

Business Development Executive should have:

- My Leads
- Lead Pipeline
- Follow-ups
- My Tasks
- Targets
- Work Activity
- Performance
- Notifications
- Profile

Display:

- assigned leads

- new leads
- pending follow-ups
- overdue follow-ups
- qualified leads
- converted leads
- target progress
- authorized royalty contribution visibility

BDEs must not gain management controls.

==========
SOCIAL MEDIA MANAGER
==========

Provide:

- campaigns
- content pipeline
- content calendar
- assigned tasks
- revisions
- reviews
- approvals
- work activity
- analytics

Support:

Brief
→ Draft
→ Review
→ Changes Requested
→ Revision
→ Resubmission
→ Approved
→ Completed/Published

==========
GRAPHIC DESIGNER
==========

Provide:

- creative queue
- design brief
- assigned tasks
- files
- versions
- reviews
- revisions
- work activity

Support version history.

==========
IT SYSTEM ANALYST
==========

Provide:

- system health
- incidents
- APIs
- authentication/session events
- technical tasks
- audit/security
- deployments/changes
- diagnostics

Technical privilege must not automatically grant business-management authority.

==========
LEAD MANAGER
==========

This is the direct team-management workspace.

Provide:

- team overview
- employee availability
- task management
- workload
- live targets
- attendance/leave
- blockers
- escalations
- approvals/QA
- performance
- reports
- communication

Manager task workflow:

Create Task
→ Define Deliverables
→ Set Priority
→ Set Due Date / SLA
→ Assess Employee Bandwidth
→ Assign
→ Notify
→ Monitor
→ Handle Blocker / Overdue
→ Review Submission

→ Approve OR Request Changes
→ Close
→ Audit

==========
SENIOR MANAGER
==========

Provide broad operational oversight across multiple teams/functions.

Include:

- teams
- performance
- workload
- tasks
- leads
- escalations
- approvals
- analytics
- royalty metrics within authorized scope

==========
GENERAL MANAGER
==========

This is the primary broad management workspace.

Include:

- business overview
- departments
- teams
- targets
- task operations
- leads
- workload
- performance
- escalations
- royalty
- reports
- analytics

==========
CHAIRMAN
==========

Create an executive operations cockpit.

The Chairman should see:

- organization operational health

- active work
- overdue work
- major escalations
- lead pipeline
- conversion
- royalty earned
- royalty trend
- team contribution
- operational risks

Royalty drilldown:

Total Royalty
→ Team
→ BDE
→ Lead
→ Business Outcome

The Chairman should primarily inspect, analyze and make authorized high-impact decisions rather than perform routine administrative operations.

==========
ADMIN
==========

Admin has limited administrative authority.

Provide:

- user management
- account lifecycle
- location/attendance administration
- authorized global task oversight
- announcements
- approved settings
- security/audit viewing

Admin should not have unrestricted platform authority.

==========
SUPER ADMIN
==========

Super Admin has platform-level technical and authorization authority.

Provide:

- users
- roles
- permissions
- access levels
- platform configuration

- security
- audit
- integrations
- system health
- authorized global operations

Privileged operations must require confirmation and create audit records.

==========
INTELLIGENCE & ANALYTICAL LAYER
==========

Create a separate analytical workspace.

Modules:

1. Performance Analytics
2. Task & Workload Analytics
3. Lead & Pipeline Analytics
4. Royalty Analytics
5. Attendance & Workforce Analytics
6. Trend & Pattern Analysis
7. Exception & Risk Analysis
8. Predictive Insights where supported
9. Automated Reporting

Every visualization must answer a business question.

Examples:

- Are tasks being completed on time?
- Where are bottlenecks?
- Is workload uneven?
- Which leads are stagnating?
- Which teams generate the most royalty?
- What is the royalty trend?
- Which tasks are repeatedly reassigned?
- Which blockers are unresolved?
- Which performance trends require attention?

Do not create meaningless decorative charts.

==========
AUDIT
==========

Create reusable audit history.

Audit important events such as:

- login
- logout

- failed login
- location exception
- account creation
- account changes
- role changes
- permission changes
- task creation
- task modification
- reassignment
- priority change
- deadline change
- progress change
- work note
- effort entry
- document upload/version
- blocker
- escalation
- submission
- review
- approval
- rejection
- closure
- lead assignment
- lead reassignment
- lead stage changes
- conversion
- royalty creation
- royalty adjustment
- configuration changes
- privileged actions

Audit entries should display:

Actor
Role
Timestamp
Action
Resource
Previous Value
New Value
Reason
Source

Make normal audit history read-only.

==========
DATA MODEL
==========

Create a sensible relational backend model around:

Users

Roles
Permissions
Teams
Locations
Sessions
Tasks
Task Assignments
Task Status History
Work Activities
Issues/Blockers
Escalations
Task Submissions
Task Reviews
Leads
Lead Activities
Lead Stage History
Lead Assignments
Conversions/Business Outcomes
Royalty Records
Attendance
Availability
Leave Requests
Notifications
Audit Logs
Reports

Use relationships rather than duplicating data.

Preserve historical records instead of overwriting important operational history.

==========
UI/UX
==========

Create a cohesive enterprise application shell.

Desktop-first.

Use:

- left navigation
- compact top header
- breadcrumbs
- global search
- notification center
- current user/role
- location/availability indicator
- data tables
- filters
- sorting
- drawers
- modals

- tabs
- contextual side panels
- audit history drawers
- clear empty states

Style:

professional
modern
calm
information-dense
enterprise
trustworthy
accessible

Avoid:

- marketing pages
- public signup
- excessive gradients
- excessive glassmorphism
- childish cards
- meaningless charts
- overly decorative UI
- huge empty hero sections

Make the system complex underneath but simple on top.

==========
IMPLEMENTATION RULE
==========

Do not generate eleven unrelated applications.

Build one coherent application shell with role-aware navigation,
authorization and reusable components.

Create shared components for:

- task tables
- lead tables
- status badges
- KPI cards
- audit history
- activity timeline
- approval panels
- notifications
- filters
- data tables
- profile headers
- confirmation dialogs

```
Role-specific dashboards should reuse the same design system.

Before implementing any royalty formula or business KPI threshold, keep the
field configurable and clearly mark it as management-defined.

Do not invent unknown business rules.

Start with:
1. application shell
2. database/data model
3. authentication
4. role/permission system
5. common task/lead components
6. Chairman dashboard
7. General Manager dashboard
8. Lead Manager dashboard
9. BDE dashboard
10. Designer/Social Media dashboards
```

Then I'd build Lovable iteratively

Don't ask it to finish the entire thing in one pass.

Use this sequence:

```
STEP 1
Foundation + database + auth + RBAC

↓

STEP 2
Common application shell

↓

STEP 3
Lead + royalty data model

↓

STEP 4
Task + work-activity engine

↓

STEP 5
Manager workspace

↓

STEP 6
Employee/BDE/Designer workspaces
```

```
↓

STEP 7
Chairman / GM / Senior Manager

↓

STEP 8
Admin + Super Admin

↓

STEP 9
Intelligence & Analytical Layer

↓

STEP 10
Audit + permissions hardening
```

That is especially important for your project because **royalty cannot safely be designed as a cosmetic dashboard metric**. Its source, attribution, trigger and calculation need to come from your Manager's answers.

Also, the project material already established the idea that permission should be based on **scope + actions + workflow rights**, rather than simply role names. gemini-chattings

So I'd use Lovable to build the **framework first**, then plug in the actual answers you get from your Manager.

And mate, **the text file I generated is still useful** — use it as your internal requirements/specification alongside these Lovable prompts, rather than pasting the entire file blindly every time.

How many roles do we have? and list them

We currently have **11 roles**:

1. **Chairman**
2. **Senior Manager**
3. **General Manager**
4. **Lead Manager**

5. **Executive**
6. **IT System Analyst**
7. **Business Development Executive (BDE)**
8. **Social Media Manager**
9. **Graphic Designer**
10. **Admin**
11. **Super Admin**

Let's just say, we are designing and internal operations tracking system for Ciril.in right?, so do a research on that company and suggest 3d animation ideas that we can actually embed on to the login page.

Worked for 39s

Absolutely, mate. I researched the current public-facing CIRIL site and its recent material before proposing the concepts. CIRIL presents itself as a commercial real-estate advisory network with services spanning **office, retail, industrial, land, investment, and logistics/warehousing**, and describes its process around client analysis, market analysis, site inspection, site selection, lease negotiation and post-leasing. CI... +2

The current CIRIL homepage also highlights a broad India-wide footprint, currently stating **45+ offices and 30+ cities**, while its H2 2025 report presents **32 cities, 41+ locations and 17 states**. CI... +1 CIRIL also publishes recurring Indian commercial-real-estate research, so **market intelligence + geography + property + business movement** are much more authentic themes for your login animation than a generic "corporate 3D animation." CI...

What I would make the login animation communicate

Your internal system is not the public CIRIL website.

So I would **not** put:

> "Luxury buildings rotating around a login form"

That would look like a property-sales website.

Instead, the animation should quietly communicate:

CIRIL → India-wide network → real-estate operations → people/work → intelligence

That also connects beautifully with your three-layer architecture:

```
                CIRIL
                  ↓
      AUTHENTICATION & ACCESS
                  ↓
       INTERNAL OPERATIONS
                  ↓
      INTELLIGENCE & ANALYTICS
```

Concept 1 — "India Network to Operations"

This is probably the most natural concept for CIRIL.

Animation sequence

```
Dark/neutral background
        ↓
A 3D India map appears
        ↓
Small location nodes illuminate
        ↓
Thin lines connect the nodes
        ↓
The network slowly forms
        ↓
Camera moves toward one location
        ↓
The map transforms into a 3D city/property grid
        ↓
Data lines begin moving through the grid
        ↓
The CIRIL identity appears
        ↓
Login panel remains clean on the side
```

The geographic concept has a direct connection to CIRIL's published multi-city network and its repeated use of India maps in its research material. CI... +1

Visual language

Use:

- low-poly 3D India map
- subtle extruded terrain
- glowing location nodes
- thin network connections
- abstract commercial buildings
- slow camera movement
- restrained particles

Don't show 30+ city labels. That would become busy.

Instead, perhaps show **5–7 representative nodes** and let the rest remain abstract.

Why it fits the portal

It immediately says:

> **"This is a nationwide organization operating through a connected network."**

That is more appropriate for an internal operations system than a random futuristic animation.

Concept 2 — "From Property to Intelligence"

This one connects directly to your **Intelligence & Analytical Layer**.

Instead of focusing primarily on CIRIL's geographic reach, the animation tells a miniature story:

```
3D property
     ↓
Real-estate information
     ↓
Data points
     ↓
Connected operational network
     ↓
Analytics
```

Animation

Start with an elegant abstract 3D commercial building.

Then:

```
Building
   ↓
Floor outlines appear
   ↓
Property attributes emerge as floating data
   ↓
Data flows into a central network
   ↓
Multiple properties connect
   ↓
Graphs / analytics surfaces appear
```

```
   ↓
Everything resolves into the CIRIL identity
```

The property should **not** be a recognizable real building. Make it an abstract modern commercial asset.

You could have six subtle data categories orbiting it:

```
OFFICE
RETAIL
INDUSTRIAL
LAND
INVESTMENT
LOGISTICS
```

Those correspond directly with CIRIL's published service categories. CI... +1

Why I like this for your internal system

Because the portal isn't merely about properties.

It's about:

operations → data → intelligence.

That makes the animation conceptually related to your third layer.

Concept 3 — "The CIRIL Operations Grid"

This would be the most directly connected to **your actual project**.

Imagine a 3D miniature city viewed from an elevated angle.

The city isn't meant to be a realistic city. It's an **abstract operational model**.

Animation

```
3D city grid
      ↓
Buildings / districts
      ↓
Activity nodes begin appearing
      ↓
Lines connect different activity points
      ↓
Tasks / leads / people represented as moving data objects
      ↓
Some nodes change state
```

```
      ↓
Analytics layer appears above the city
      ↓
Everything settles into a clean operational grid
```

You could subtly represent your system's actual information:

```
LEADS
TASKS
PEOPLE
WORK
ROYALTY
INSIGHTS
```

For example:

```
LEAD
  ↓
BDE
  ↓
TASK
  ↓
WORK
  ↓
CONVERSION
  ↓
ROYALTY
```

This would be an excellent conceptual connection to the business model you've described.

Important

Don't literally make the login page show:

> "BDE → Royalty → Chairman"

That would be too application-specific for a login screen.

Represent it abstractly as **data flowing through an operational network**.

Concept 4 — "Six Real-Estate Worlds"

This would lean harder into CIRIL's actual business sectors.

A central 3D geometric core sits in the middle.

Around it, six small abstract environments smoothly rotate/transition:

```
OFFICE
   ↓
RETAIL
   ↓
INDUSTRIAL
   ↓
LAND
   ↓
INVESTMENT
   ↓
LOGISTICS / WAREHOUSE
```

Each could be represented by a tiny 3D environment.

For example:

Office

A clean high-rise silhouette.

Retail

A storefront block with illuminated frontage.

Industrial

A larger industrial structure.

Land

Abstract terrain/topography.

Investment

Geometric financial/data structures.

Warehouse

Large warehouse + logistics paths.

The six categories are directly reflected in CIRIL's published service material. CI... +1

The transitions should be **very slow and elegant**, not like a product advertisement.

Concept 5 — "Research → Market → Decision"

This one is particularly interesting because CIRIL publishes market research and describes analytical/advisory work as part of its offering. CI... +1

Animation

```
Abstract map
      ↓
Market data points
      ↓
Property blocks
      ↓
Charts / analytical surfaces
      ↓
Connected intelligence
      ↓
Decision node
      ↓
CIRIL
```

You can make the 3D elements feel like a combination of:

GIS + real estate + business intelligence

without copying any specific product.

Imagine:

- 3D terrain
- building masses
- thin analytical grids
- floating data points
- contour lines
- subtle graphs
- connection lines

This would be particularly appropriate because your internal system eventually has its own **Intelligence & Analytical Layer**.

Concept 6 — "Aerial India → CIRIL"

This would be the most cinematic.

Start with a very dark, abstract aerial view.

Gradually:

```
Terrain
   ↓
Road network
   ↓
Cities appear
   ↓
Commercial districts emerge
   ↓
Network connections appear
   ↓
Camera pulls upward
   ↓
The whole structure resembles India
   ↓
CIRIL identity appears
```

This is inspired by CIRIL's recurring presentation of its national network through India maps. CI... +1

It would look excellent behind a login card **provided the animation remains subtle**.

Concept 7 — "Operational Data Stream"

This one is the most futuristic and the least explicitly real-estate-oriented.

A stream of tiny 3D objects flows through an abstract network:

```
People
   ↓
Tasks
   ↓
Leads
   ↓
Work
   ↓
Decisions
   ↓
Analytics
```

Visually it might look like a **3D neural/network system**, but without pretending that your product uses AI everywhere.

Objects move through nodes:

```
• ── • ── •
     |
     •
```

```
    |
• —— •
```

Nodes represent operational entities.

The system eventually forms a clean geometric CIRIL-inspired mark or abstract "C" shape.

This would make the login page feel like the **gateway into an operating system**, rather than a normal employee portal.

Concept 8 — My favorite hybrid

I'd actually combine **Concept 1 + Concept 3 + Concept 5**.

"CIRIL Intelligence Network"

The animation would be around **8–12 seconds** and loop seamlessly.

Scene 01 — Geographic presence

```
3D India silhouette
↓
network nodes illuminate
```

Scene 02 — Operational network

```
India
↓
camera zoom
↓
abstract commercial city
↓
people/data/activity nodes
```

Scene 03 — Intelligence

```
operational nodes
↓
data streams
↓
analytics surfaces
↓
pattern/network formation
```

Scene 04 — Resolution

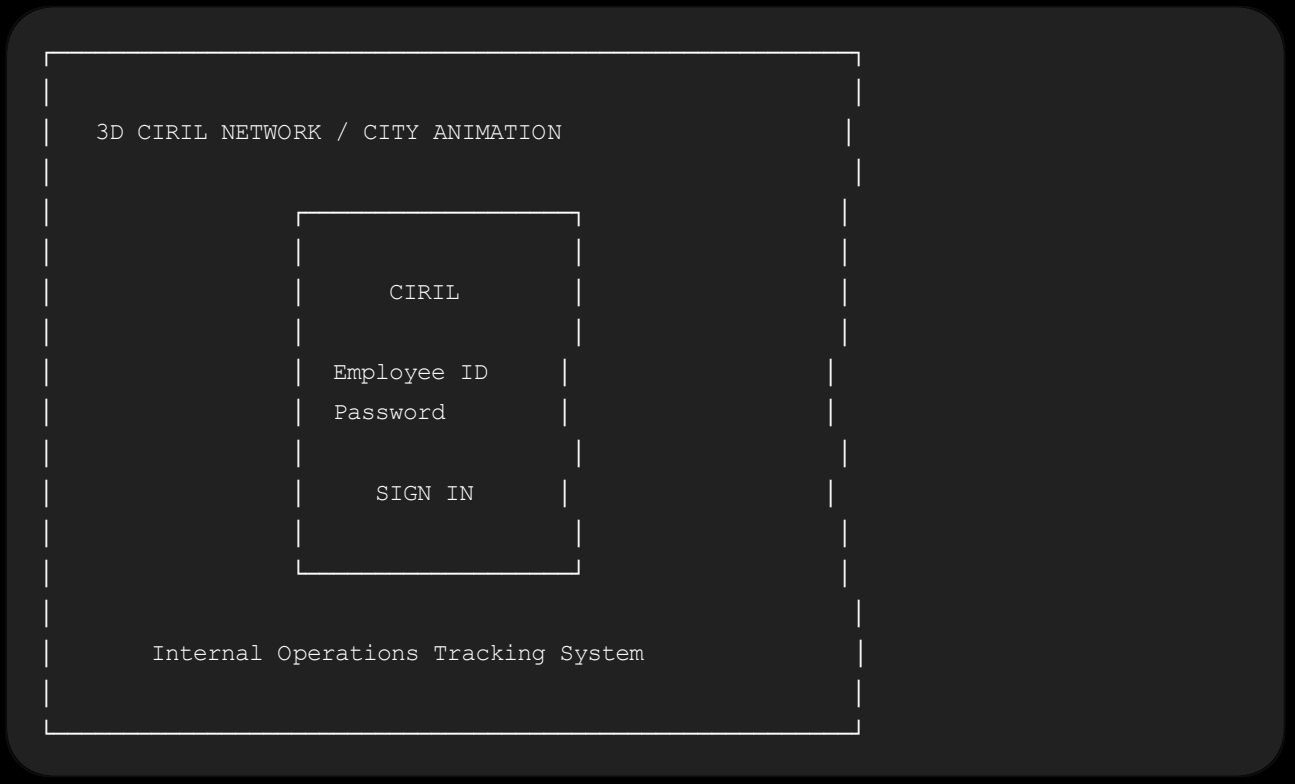
```
everything converges
↓
CIRIL identity
↓
soft fade
↓
loop
```

The login card sits over the **negative-space portion** of the animation.

How I'd actually arrange the login page

I'd avoid putting the animation behind the form at full intensity.

Something like:



Or even better:

Animation occupying 55–65% of the screen
Login panel occupying 35–45%

That lets the animation breathe.

A very CIRIL-specific visual vocabulary

Based on the company's public material, I'd build the animation around:

Geography

India / cities / network / location.

Real estate

Buildings / land / industrial / office / retail / warehouse.

Advisory

Analysis / connection / decision / flow.

Intelligence

Data / research / patterns / market insight.

CIRIL itself describes its work in terms including client analysis, market analysis, site inspection, site selection, lease negotiation and post-leasing, which gives us a surprisingly good vocabulary for a visual story. CI...

So instead of generic:

> "Cyber blue 3D cubes flying around"

I'd make it:

> **"India's real-estate network transforming into an operational intelligence system."**

That's much more specific to CIRIL.

One important thing for Lovable

Since you're using **Lovable**, I would **not put a heavy 3D render directly into the React page without considering performance**.

I'd specify the implementation conceptually as:

```
Login Page
    |
    ├── Login UI
    |
    └── Lightweight 3D Scene
            |
            ├── WebGL / Three.js
            ├── Controlled animation loop
            ├── Lazy loading
```

```
├── Reduced-motion fallback
└── Static fallback image
```

For the actual implementation, I would keep it:

GPU-light → looping → deterministic → no interaction required.

The login experience should still load quickly even if the 3D scene fails.

And I would avoid these

❌ giant rotating globe
❌ skyscraper fly-through
❌ generic stock-city animation
❌ cryptocurrency-style glowing numbers
❌ excessive neon blue
❌ spinning 3D CIRIL logo
❌ property photos flying around
❌ animated graphs everywhere

Those would make it look like a **real-estate marketing site or a flashy AI template**, rather than an internal enterprise platform.

The strongest conceptual direction

For this particular project, I'd phrase the animation brief as:

> **"A restrained 3D visualization of CIRIL's nationwide real-estate network transforming into a connected operational intelligence system."**

That single sentence gives your designer/developer a very clear creative direction while tying together **CIRIL's actual business, its national network, your lead/task operations, and the Intelligence & Analytical Layer**. CI... +1

Wed, Sep 23 at 1:13 PM

1_updated_react.html
File

chatting_2026-09-23_1...
Document

unslop_1_updated_rea...
File

This is anti-gravity generated code using gemini pro model. it is looking more verbose, and design need to reassessed so give me a new looking page code, like remove the extra functionalities that would make the page loading complex, so based on these factors study the uploaded files and give me the best UI login page, using paths of the image as they were, and just give me updated UI versions based on the uploaded files. 1_update_react.html is my personally designed version unslop_1_update_react.html is agy version with several skills related to the frontend okay.. so

Show more

Worked for 10m 19s

Yep mate. I went through all three uploaded files rather than just the generated version.

The biggest issue is clear: **your original** `1_updated_react.html` **was relatively focused**, while the Antigravity/unslop version accumulated a large interaction/animation stack — border beam, cursor spotlight/3D tilt, constellation canvas, floating orbs, telemetry, audio, passkey simulation, command palette, toast system, password entropy animation, multi-stage authentication, etc. chatting_2026-09-23_12-48-14 +1

I therefore rebuilt it around the **visual simplicity of your original**, but with a more contemporary enterprise layout. I also preserved your existing CIRIL image path:

```
../img/logo.png
```

with the same `./img/logo.png` fallback. 1_updated_react

What I changed

The new version removes the unnecessary login-page machinery:

- no WebGL / Three.js
- no canvas particle system
- no border-beam animation
- no cursor-driven 3D card tilt
- no audio/Web Audio
- no passkey/biometric simulation
- no command palette
- no telemetry popover
- no demo-role autofill
- no password-strength meter

- no toast framework
- no large collection of modals

Instead, it has:

CIRIL identity → subtle operational visual → focused authentication card → lightweight state transition.

The visual itself uses **CSS-only motion**, so the page still feels alive without creating another heavy rendering layer.

The authentication behavior still reflects the design direction from your conversation: one common login, internally provisioned accounts, and role/permission resolution after authentication rather than asking the user to select a role. chatting_2026-09-23_12-48-14

Updated file

Download the redesigned modern CIRIL login page

The new layout is intentionally much closer to **“professional internal enterprise system”** than **“AI-generated SaaS landing page.”**

modern_ciril_login.html
HTML

I think you got the idea of the project, that i am working on right?

Yes, mate. I’ve got the overall picture of what you’re building.

You’re developing **CIRIL’s Internal Operations Tracking System (IOTS)** — essentially an internal enterprise platform that connects **authentication, employee/manager operations, business workflows, and operational intelligence** in one system.

The structure we’ve established is:

1. Authentication & Authorization Layer
A common login for everyone, location validation, account/identity verification, permission resolution, and controlled routing into the appropriate workspace. No public registration; accounts are created internally.

2. Management Layer
This is where the actual organization operates:

- Chairman
- Senior Manager
- General Manager
- Lead Manager
- Executive
- IT System Analyst
- BDE
- Social Media Manager
- Graphic Designer
- Admin
- Super Admin

The layer handles things such as **tasks, assignments, employee work, attendance/availability, approvals, blockers, escalation, leads, royalty-related operations, and managerial oversight**.

3. Intelligence & Analytical Layer
This sits above the operational data and turns it into **metrics, patterns, exceptions, alerts, reports, and decision-support information**. The important distinction we established is:

> **Management Layer = records and manages what is happening.**
> **Intelligence & Analytical Layer = determines what the operational data means.**

And there are some important business-specific concepts tied into it, especially the **lead → business outcome → royalty** pipeline, because CIRIL's business model revolves heavily around leads and the resulting royalty.

For the employee side, we also separated **work activity from auditing**:

- Progress updates
- Work notes
- Time/effort
- Supporting documents/evidence

while the system separately maintains the **audit trail** of actions and state changes.

And for the UI, especially the login page, your direction has become very clear too: **professional internal enterprise system, modern and clean, but not an over-engineered "AI SaaS landing page."** That's why we moved away from the Antigravity

version's excessive animations, telemetry gimmicks, fake security claims, passkey simulations, etc., and back toward the cleaner character of your original design.

So yes — I understand the project not just as "a dashboard," but as an **end-to-end internal operations platform for CIRIL where authorization → operational management → analytics/intelligence form one connected system**.

Wednesday 3:44 PM

claude_version.docx
Document

gemini_version.docx
Document

both are the documents are generated by two ai models in gravity, so based on these context, give me a detailed prompt, on designing the architecture of the Internal Operations Tracking System (IOTS), in all directions.. followed by I am also going to upload the diagram-drawio file for specifying how it should be laid out, so read these meanwhile

Worked for 40s

Yes, mate. I've now read both uploaded documents — `claude_version.docx` and `gemini_version.docx` — and they are essentially two AI-generated versions of the same IOTS enterprise specification. The important part is that they converge strongly on the same business model, 3-layer structure, 11 roles, task engine, geolocation/security, commercial/royalty flow, analytics, performance, and UI/UX philosophy. claude_version gemini_version

The documents also make an important architectural distinction: **IOTS is not merely a collection of dashboards**. It is an operational system where authentication, role/context, workflows, auditability, commercial operations, analytics, and decision support are interconnected. claude_version

So before you upload the Draw.io file, I'd structure the architecture prompt around **the complete system logic first**, and keep the exact visual composition/layout for the Draw.io-specific second pass.

IOTS — Master Enterprise Architecture Design Prompt

Version 1 — Architecture, Logic & System Structure

Layout-neutral; Draw.io layout instructions to be added after the `.drawio` file is supplied

```
You are acting as a Senior Enterprise Systems Architect, Solution Architect,
Business Process Architect, Security Architect, Data Architect, and UI/UX
Systems Architect.

You are designing the complete enterprise architecture of:

CIRIL
Internal Operations Tracking System (IOTS)

Organization:
Commercial Real Estate Institutional Alliance (CIRIL)

System Purpose:
Design one coherent internal enterprise platform that connects authentication,
authorization, employee operations, managerial control, commercial operations,
task execution, attendance, geolocation, security auditing, analytics,
reporting, performance measurement, and management intelligence.

IMPORTANT:
This is NOT a generic CRM.
This is NOT a generic HRMS.
This is NOT simply a dashboard collection.
This is an internal operations-control platform designed specifically around
CIRIL's business and organizational workflow.

The architecture must therefore represent BOTH:

1. What the organization does
2. How the software enables, controls, records, and analyzes what the organization

============================================================
1. PRIMARY ARCHITECTURAL PRINCIPLE
============================================================

Use the following principle throughout the architecture:

"COMPLEX UNDERNEATH, SIMPLE ON TOP."

The underlying system must support:
- detailed permissions
- multiple organizational roles
- workflow/state transitions
- task history
- work activity history
- geolocation records
- attendance records
- financial/royalty records
- document versions
```

- audit history
- analytics
- exception detection
- automated reporting
- multi-role contexts

However, the visible user experience must remain:
- clean
- understandable
- calm
- professional
- enterprise-grade
- information-dense without becoming visually chaotic

Do not design the system as a collection of disconnected modules.

Show the dependencies and relationships between modules.

==
2. ARCHITECTURAL FOUNDATION
==

The business architecture MUST be organized into the following three foundational layers:

LAYER 1
Authentication, Geolocation & Security Layer

LAYER 2
Management, Operations & Execution Layer

LAYER 3
Intelligence, Analytics & Reward Layer

Treat these three as the top-level conceptual architecture.

However, do not confuse these business layers with conventional software layers.

Underneath the three conceptual layers, show the technical architecture required to make them operate.

The technical architecture may include, where appropriate:

- Presentation / UI Layer
- Application / Domain Layer
- Workflow & Business Rules
- Authorization / Policy Enforcement
- Integration / API Services
- Background Processing
- Data Persistence
- File / Document Storage

- Analytics / Reporting
- Audit / Event Infrastructure
- Monitoring / Observability
- Security Infrastructure

Do NOT arbitrarily choose technologies merely to make the diagram look more technical.

Where implementation technology has not been formally confirmed, label the component by responsibility rather than inventing a technology.

==========
3. SYSTEM CONTEXT
==========

At the highest level, represent IOTS as the internal operational control platform for CIRIL.

The system should receive and manage information from:

- Employees
- Managers
- Chairman / Executive Leadership
- System Administrators
- Technical Administrators
- Internal departments
- Member companies
- Commercial leads / mandates
- Operational tasks
- Attendance and availability events
- Location information
- Documents and deliverables
- Commercial transactions
- Royalty information
- External services / APIs
- Reporting processes

The system produces:

- authorized workspaces
- operational task state
- employee activity records
- managerial decisions
- attendance information
- location context
- audit history
- commercial status
- royalty information
- performance metrics
- risk indicators
- reports
- management insights

- executive visibility

Represent IOTS as the central operational system through which these flows are coordinated.

==========
4. LAYER 1 — AUTHENTICATION, GEOLOCATION & SECURITY
==========

Design Layer 1 as the secure entry and identity-control boundary.

The common login flow must be:

User
→ Credentials Submitted
→ Credential Validation
→ Location Verification
→ Session / Security Context Creation
→ Audit Recording
→ Role / Permission Resolution
→ Workspace Routing

There is:

- NO public self-registration
- ONE common corporate login
- internally provisioned accounts
- automatic role determination
- support for users with multiple authorized roles

The user MUST NOT select their job role manually during login.

The system determines the user's authorized role context.

4.1 Identity & Account Management

Represent:

- user identity
- employee profile
- employee ID
- department
- designation
- office/location
- reporting manager
- account state
- assigned system role(s)

Accounts are created internally by authorized administrators.

4.2 Authentication

Represent:

- credential validation
- session creation
- authentication result
- failed authentication handling
- session lifecycle
- logout/check-out
- security event logging

Do not invent specific cryptographic algorithms unless explicitly marked as implementation decisions.

4.3 Geolocation

Represent the geographic verification workflow:

Credential Validation
→ Browser/Device Location Request
→ Latitude/Longitude
→ Reverse Geolocation
→ Assigned Office Comparison
→ Office / On-Premise OR Remote / Off-Site
→ Record Location Context
→ Continue Workspace Routing

The architecture must distinguish:

1. Identity verification
2. Location verification
3. Location classification
4. Audit recording

An off-site location is not automatically treated as an authentication failure.

The system should support:
- office/on-premise
- remote/off-site
- unknown/unverified context
- location exceptions

4.4 Attendance

Show attendance as related to authentication but as a distinct subsystem.

Represent:

- check-in
- check-out
- break periods
- work hours
- attendance status
- punctuality
- leave
- remote-work status
- attendance exceptions

Do not merge Attendance and Availability into the same data object.

Attendance answers:
"Was the employee present / working?"

Availability answers:
"Can the employee currently accept or handle work?"

4.5 Availability

Support the documented availability states:

- Available
- Busy
- In Meeting
- On Break
- In Training
- Do Not Disturb
- Working Remotely
- On Leave
- Absent
- Offline

Represent this as a live operational status mechanism used by managers
during workload allocation.

4.6 Security & Anomaly Detection

Represent the security monitoring path for:

- abnormal login patterns
- location anomalies
- concurrent sessions
- suspicious location changes
- remote-location clusters

- session anomalies
- other security events

Do NOT present every anomaly as an automatic account suspension unless the architecture explicitly identifies the condition and response.

Separate:

Detection
→ Classification
→ Alert
→ Review
→ Intervention
→ Audit

==========
5. LAYER 2 — MANAGEMENT, OPERATIONS & EXECUTION
==========

This is the operational core of IOTS.

The purpose of Layer 2 is to transform managerial intent into controlled, trackable work and transform employee activity into structured operational records.

It must contain:

A. People / Organization
B. Task Management
C. Work Execution
D. Blockers & Escalations
E. Attendance / Availability
F. Requests / Approvals
G. Commercial Operations
H. Lead / Deal Operations
I. Documents & Deliverables
J. Communication / Notifications

==========
6. ORGANIZATIONAL ROLE ARCHITECTURE
==========

The system contains 11 organizational roles:

1. Chairman
2. Super Admin
3. Admin
4. IT System Analyst
5. General Manager
6. Senior Manager
7. Lead Manager
8. Executive

9. Business Development Executive (BDE)
10. Social Media Manager
11. Graphic Designer

Do not represent these merely as a conventional hierarchy.

The architecture should show that these roles belong to different responsibility domains.

Use these conceptual groupings:

EXECUTIVE OVERSIGHT
- Chairman

PLATFORM GOVERNANCE
- Super Admin
- IT System Analyst

ADMINISTRATIVE CONTROL
- Admin

MANAGEMENT
- General Manager
- Senior Manager
- Lead Manager

EXECUTION / SPECIALIST WORK
- Executive
- BDE
- Social Media Manager
- Graphic Designer

IMPORTANT:
Organizational hierarchy and technical access hierarchy are NOT automatically the same thing.

==========
7. MULTI-ROLE ARCHITECTURE
==========

The system must support users with multiple authorized roles.

Flow:

User Identity
→ Authorized Role Set
→ Primary Role
→ Active Workspace Context
→ Permission Scope
→ Action
→ Audit Record containing active role context

A multi-role employee must NOT require separate accounts.

The architecture must record:

- employee identity
- authorized roles
- active role context
- permissions available within that role
- action performed under that active context

This is a critical architectural requirement.

==========
8. RBAC / AUTHORIZATION ARCHITECTURE
==========

Do not design authorization as simply:

Role = Access

Instead model authorization using multiple dimensions.

At minimum:

1. ROLE
2. DATA SCOPE
3. ACTION
4. WORKFLOW RIGHT
5. ADMINISTRATIVE RIGHT
6. CONTEXT

Data scope may conceptually include:

- Own
- Team
- Department
- Branch / Office
- Organization
- Platform

Actions may include:

- View
- Create
- Edit
- Submit
- Approve
- Reject / Request Changes
- Reassign
- Export
- Configure
- Delete

- Override

Workflow rights may include:

- Change State
- Approve
- Escalate
- Resolve
- Reopen
- Override Routing

The architecture must place authorization enforcement BEFORE protected business actions.

Conceptual path:

User Action
→ Authentication Context
→ Active Role
→ Authorization / Policy Evaluation
→ Allowed?
→ Business Workflow
→ Data Change
→ Audit Event

9. TASK MANAGEMENT ARCHITECTURE

Task Management is one of the central components of the system.

A task should be treated as a controlled operational object, not merely a row in a task list.

The task lifecycle must support:

Create
→ Define Deliverables
→ Set Priority
→ Set Due Date / SLA
→ Assess Capacity
→ Assign
→ Notify
→ Execute
→ Monitor
→ Review
→ Correct / Resubmit OR Approve
→ Complete / Close

9.1 Task Macro States

The architecture must distinguish the documented macro reporting states:

- ASSIGNED
- PENDING
- OVERDUE
- REASSIGNED
- COMPLETED

Important:

REASSIGNED is a transition/event, not a terminal state.

OVERDUE is an exception state requiring action.

COMPLETED is the terminal approved state.

9.2 Task Micro States

Support granular operational states:

- In Progress
- Blocked
- Waiting for Review
- Under Review / QA
- Changes Requested
- Issue Resolved
- Done / Closed

Do not treat these as a flat list.

Represent them as a state machine with permitted transitions.

==========
10. TASK STATE MACHINE
==========

The architecture must show:

Task Created
→ Assigned
→ Acknowledged / Active
→ In Progress

From In Progress:

→ Blocked
→ Continue
→ Waiting for Review
→ Under Review / QA

From Review:

→ Approved
→ Completed / Closed

OR

→ Changes Requested
→ Revision Cycle
→ In Progress
→ Resubmit
→ Review

Blocked flow:

In Progress
→ Blocked
→ Issue Ticket
→ Manager Intervention
→ Issue Resolved
→ In Progress

Overdue flow:

Active Task
→ Deadline Reached
→ OVERDUE
→ Manager Intervention

Manager intervention options include:

- Deadline Extension
- Reassignment
- Escalation
- Scope Reduction / Guidance

Every intervention must create an auditable event.

==========
11. REASSIGNMENT ARCHITECTURE
==========

Reassignment must be represented as a controlled managerial workflow.

Flow:

Task
→ Reassign Request
→ Team Capacity Inspection
→ Reassignment Reason
→ Handover Context

→ New Owner
→ Notification
→ Audit Record
→ Task Continues

The architecture must preserve:

- previous owner
- new owner
- reason
- handover note
- timestamp
- approving / initiating actor
- task state history

Never make reassignment look like deletion and recreation of the task.

==========
12. WORK ACTIVITY ARCHITECTURE
==========

Keep TASK STATE and WORK ACTIVITY as separate concepts.

State answers:

"Where is the task in the workflow?"

Activity answers:

"What did the employee actually do?"

Structured Work Activity consists of:

1. Progress Update
2. Work Notes
3. Time / Effort Entry
4. Supporting Evidence / Deliverables

An employee may create multiple activities during one task state.

The architecture must therefore model:

Task
→ Task State History

and separately:

Task
→ Work Activity Stream

These streams may be displayed together in the UI but must remain
conceptually distinct.

```
============================================================
13. DOCUMENT & VERSION ARCHITECTURE
============================================================

Supporting files and deliverables must have their own document/version model.

Represent:

Task / Brief
→ Deliverable
→ File
→ Version
→ Revision
→ Submission
→ Review
→ Approval

For creative work especially, preserve:
- v1.0
- v1.1
- v2.0
etc.

Never overwrite an important historical deliverable version.

============================================================
14. BLOCKER / ISSUE / ESCALATION ARCHITECTURE
============================================================

A blocker is not simply a comment.

Model it as an operational object.

Flow:

Employee
→ Raise Blocker / Issue
→ Task Paused
→ Manager Notification
→ Manager Review
→ Resolution OR Escalation
→ Issue Resolved
→ Task Resumes

Escalation path may conceptually move:

Employee
→ Lead Manager
→ Senior Manager
→ General Manager
→ Chairman
```

depending on scope and business rules.

Do not assume every issue reaches the Chairman.

==
15. MANAGEMENT CONTROL ARCHITECTURE
==

Management functions must include:

- Team Management
- Task Assignment
- Task Reassignment
- Workload Management
- Live Availability
- Attendance / Leave
- Approval & QA
- Blocker Management
- Escalation
- Team Performance
- Commercial / Royalty Oversight
- Operational Reporting

The architecture should demonstrate that managers do not merely "view dashboards".

They actively operate the workflow.

==
16. COMMERCIAL OPERATIONS ARCHITECTURE
==

Represent the CIRIL commercial syndication model as a distinct business domain.

Geographic structure:

Country
→ Zone
→ State
→ City
→ Member Company
→ Advisory Associate

The commercial lifecycle must support:

1. Mandate Origination
2. Central Clearance / Destination Handoff
3. Sourcing & Institutional Proposals
4. Inspections & Negotiation
5. Deal Conclusion / Financial Lock

6. Fee Apportionment & Royalty Settlement

==
17. LEAD GIVER / LEAD RECEIVER MODEL
==

The architecture must explicitly distinguish:

Lead Giver (LG)
Lead Receiver (LR)

Conceptual flow:

Mandate / Lead
→ Originating Member / Lead Giver
→ CIRIL Coordination
→ Destination Member / Lead Receiver
→ Property Search / Sourcing
→ Inspections
→ Proposal
→ Negotiation
→ Agreement
→ Deal Closure
→ Financial Processing

Preserve the business relationship between origin and destination parties.

==
18. ROYALTY ENGINE
==

The specification documents define:

Lead Giver Share = 30%
Lead Receiver Share = 70%
CIRIL Corporate Royalty = 7.5% of Total Gross Brokerage

Represent the royalty process as:

Deal Conclusion
→ Financial Metrics Confirmed
→ Gross Brokerage Calculation
→ 30/70 Member Split
→ CIRIL Royalty Calculation
→ Settlement / Remittance Tracking
→ Royalty Reporting
→ Executive Visibility

Do not hard-wire the architecture to one calculation formula if business policy may change.

Represent calculation rules as configurable business rules where appropriate,

while preserving the currently documented 7.5%, 30%, and 70% model as the current business configuration.

==========
19. COMMERCIAL DATA MODEL
==========

At a conceptual level the architecture should account for entities such as:

- Client / Mandate
- Lead
- Member Company
- Lead Giver
- Lead Receiver
- Property / Inventory
- Listing
- Site Visit
- Proposal
- Negotiation
- Deal / Transaction
- Financial Milestone
- Brokerage
- Royalty Settlement

Connect these to the operational task engine where work must be performed by CIRIL employees.

==========
20. ATTENDANCE + OPERATIONS RELATIONSHIP
==========

Do not isolate attendance from operational work.

Show that attendance / availability data can influence:

- task assignment
- workload balancing
- remote status
- leave management
- team availability
- management decisions
- performance analytics

But do not allow attendance data to automatically become performance judgment without an explicit business rule.

==========
21. LAYER 3 — INTELLIGENCE & ANALYTICAL ARCHITECTURE
==========

The analytical layer is not just a collection of charts.

It must be shown as a pipeline:

Operational Data
→ Data Preparation / Aggregation
→ Metrics
→ Pattern Detection
→ Exception / Risk Detection
→ Insights
→ Alerts / Reports
→ Management Decision

The system must answer actual business questions.

Examples:

- Are tasks being completed on time?
- Which teams are overloaded?
- Which tasks are repeatedly reassigned?
- Where are blockers accumulating?
- Are attendance patterns changing?
- How is the lead pipeline progressing?
- What is the current royalty position?
- Are operational delays becoming systemic?

==========
22. ANALYTICS CENTRES
==========

Represent the documented analytics areas:

1. Performance & Velocity Analytics
2. Task & Workload Distribution Analytics
3. Attendance & Workforce Health Analytics
4. Trend & Historical Pattern Analysis
5. Exception & Risk Analysis
6. Predictive Operational Signals

The architecture should show where each receives data from.

Example:

Task Engine
→ Completion Metrics
→ SLA Metrics
→ Revision Metrics
→ Performance Analytics

Attendance Engine
→ Attendance Metrics
→ Punctuality
→ Remote/On-Site Trends
→ Workforce Analytics

```
Commercial Engine
→ Deal Data
→ Brokerage
→ Royalty
→ Commercial Analytics

============================================================
23. RISK & EXCEPTION ENGINE
============================================================

Show an exception processing path:

Operational Event
→ Rule Evaluation
→ Risk Classification
→ Alert
→ Responsible Role
→ Intervention
→ Resolution
→ Audit

Documented examples include:

- deadline risk
- overdue work
- repeated reassignment
- overload
- unresolved blockers
- attendance exceptions
- security anomalies

Do not label the system "AI" merely because it performs threshold or rule
analysis.

Clearly distinguish:

Deterministic Rules
versus
Predictive / Statistical Analysis

============================================================
24. REPORTING ARCHITECTURE
============================================================

Support the three reporting rhythms:

DAILY
- operational digest

WEEKLY
- departmental / team operational review
```

MONTHLY
- executive / strategic scorecards

The architecture should include:

Data Collection
→ Metrics
→ Report Assembly
→ Role-Based Filtering
→ Report Generation
→ Distribution
→ Archive

Reports must be scoped according to role.

Examples:

Chairman
→ Executive Radar

GM / Senior Management
→ Cross-Team / SLA / Operational Reports

Lead Manager
→ Daily Action-Oriented Reports

Employees
→ Personal Performance / Activity Reports

==
25. PERFORMANCE & REWARD ARCHITECTURE
==

Represent employee performance as a downstream consumer of verified
operational records.

Inputs may include:

- task delivery
- SLA adherence
- revision history
- attendance / punctuality
- blocker resolution
- role-specific KPIs

Flow:

Operational Records
→ KPI Engine
→ Performance Calculation
→ Performance Profile

→ Monthly Evaluation
→ Recognition Workflow
→ Quarterly Recognition Review

Do not allow subjective scores to be represented as if they were system facts.

Manager-entered assessment must remain distinguishable from system-derived metrics.

==========
26. AUDIT ARCHITECTURE
==========

Audit is a cross-cutting capability.

Do not make "Audit" just another dashboard module.

Represent audit as a system-wide append-only event mechanism connected to important domains.

Audit sources include:

- authentication
- sessions
- location
- user accounts
- role changes
- permission changes
- task changes
- task reassignment
- task review
- approvals
- leave
- geofence changes
- royalty settlements
- configuration changes
- security events

Every important event should conceptually capture:

- Actor
- Active Role
- Action
- Object
- Timestamp
- Previous Value
- New Value
- Reason / Comment
- Source
- Relevant security / location context

Do not imply that a database is automatically "immutable" merely because
the UI calls it immutable.

The architecture must show the control mechanism that protects audit history.

==
27. NOTIFICATION ARCHITECTURE
==

Create a centralized notification/event mechanism.

Events may originate from:

- new task
- reassignment
- blocker
- overdue task
- review request
- changes requested
- approval
- leave request
- escalation
- security anomaly
- report publication

Flow:

Business Event
→ Event / Notification Service
→ Audience Resolution
→ Delivery Channel
→ Delivery Status
→ Audit

==
28. SEARCH & GLOBAL ACCESS
==

The common application shell should support role-aware:

- navigation
- global search
- notifications
- user identity
- active workspace context
- office/location context
- availability
- profile/security

Search results must obey authorization scope.

A user must never receive data merely because the search engine found it.

==
29. API / INTEGRATION ARCHITECTURE
==

Represent external service boundaries separately from internal domain logic.

Potential integration categories mentioned in the specifications include:

- geolocation / reverse geocoding
- email / notifications
- file storage
- reporting
- commercial portals
- external operational data sources

Use:

Internal Domain
↔ Integration / API Boundary
↔ External Service

Do not tightly couple the core workflow directly to third-party services.

External service failures must have a controlled failure path.

==
30. DATA ARCHITECTURE
==

Design the conceptual data architecture around major domains rather than
one giant table model.

Suggested domain groupings:

IDENTITY DOMAIN
- users
- roles
- role assignments
- departments
- reporting relationships
- sessions

SECURITY / LOCATION DOMAIN
- login events
- location events
- geofences
- security events

OPERATIONS DOMAIN
- tasks
- task states

- task assignments
- work activities
- blockers
- escalations
- approvals

DOCUMENT DOMAIN
- documents
- files
- versions
- submissions

PEOPLE DOMAIN
- attendance
- availability
- leave
- requests

COMMERCIAL DOMAIN
- leads
- mandates
- member companies
- properties
- proposals
- negotiations
- deals
- brokerage
- royalty settlements

ANALYTICS DOMAIN
- operational metrics
- KPI snapshots
- trend aggregates
- risk events
- generated reports

AUDIT DOMAIN
- append-only audit events

Show relationships between domains.

==
31. DATA FLOW ARCHITECTURE
==

Demonstrate the difference between:

TRANSACTIONAL DATA
and
ANALYTICAL DATA

Transactional systems record operational truth.

Analytical systems consume those records for:
- metrics
- trends
- risk detection
- reports
- insights

Do not allow analytics to silently modify source transactional records.

==========
32. TECHNICAL SEPARATION OF CONCERNS
==========

The architecture must distinguish:

IDENTITY
from
AUTHORIZATION

AUTHORIZATION
from
WORKFLOW

WORKFLOW
from
AUDIT

AUDIT
from
ANALYTICS

TRANSACTIONAL DATA
from
REPORTING DATA

ATTENDANCE
from
AVAILABILITY

TASK STATE
from
WORK ACTIVITY

ROLE
from
PERMISSION

BUSINESS AUTHORITY
from
TECHNICAL AUTHORITY

These separations are architecturally important.

==
33. PLATFORM GOVERNANCE
==

Represent separate responsibilities for:

CHAIRMAN
- enterprise oversight

SUPER ADMIN
- platform governance / master configuration

IT SYSTEM ANALYST
- technical reliability / diagnostics

ADMIN
- routine administrative operations

MANAGEMENT
- operational control

EXECUTION ROLES
- work execution

Do not treat "highest technical privilege" and "highest business authority"
as the same concept.

==
34. SECURITY ARCHITECTURE
==

Include security controls conceptually across the entire system:

- authentication
- authorization
- session security
- role context
- location verification
- auditability
- privileged actions
- approval safeguards
- secure document access
- API boundary protection
- anomaly detection
- administrative accountability

Security must be cross-cutting, not a single box at the beginning.

==
35. FAILURE / EXCEPTION ARCHITECTURE

```
============================================================

Every major subsystem should have a failure path.

Examples:

Authentication Failure
→ Inform User
→ Log Security Event

Location Failure
→ Location Unavailable / Unknown
→ Controlled Handling
→ Record Event

External API Failure
→ Retry / Fallback
→ Mark Dependency Failure
→ Notify Technical Role

Task Overdue
→ Exception
→ Manager Intervention

Blocked Task
→ Issue
→ Resolution / Escalation

Review Rejected
→ Changes Requested
→ Revision Cycle

Royalty Discrepancy
→ Financial Exception
→ Review / Correction
→ Audit

Do not design only the happy path.

============================================================
36. CROSS-SYSTEM EVENT MODEL
============================================================

Where appropriate, represent important operational events as reusable events.

Examples:

TaskCreated
TaskAssigned
TaskReassigned
TaskSubmitted
TaskChangesRequested
```

```
TaskApproved
TaskCompleted
TaskOverdue
BlockerRaised
BlockerResolved
LeaveRequested
LeaveApproved
LoginSucceeded
LoginFailed
LocationFlagged
RoleContextChanged
RoyaltyCalculated
RoyaltySettled
SecurityAnomalyDetected

These events can feed:
- notifications
- audit
- analytics
- reporting
- escalation

============================================================
37. COMMON APPLICATION SHELL
============================================================

All authenticated roles should operate inside a shared enterprise shell.

Include conceptually:

- CIRIL identity
- current user
- active role/workspace
- location context
- availability
- notifications
- search
- navigation
- profile/security
- main role-specific content

The architecture should show:

Shared Shell
+
Role-Specific Workspace
+
Shared Services

rather than 11 completely separate applications.

============================================================
```

38. ROLE-SPECIFIC WORKSPACES
==

Represent the workspace architecture for:

Chairman
Super Admin
Admin
IT System Analyst
General Manager
Senior Manager
Lead Manager
Executive
BDE
Social Media Manager
Graphic Designer

Each workspace should consume:

Shared Identity
+
Shared Authorization
+
Shared Notification
+
Shared Audit
+
Shared Data Services

but expose different operational capabilities based on role and context.

==
39. UI / UX ARCHITECTURE PRINCIPLES
==

The system should be:

- desktop-first
- enterprise-oriented
- information-dense
- calm
- structured
- low-noise
- consistent

Avoid:
- excessive animation
- decorative dashboards
- fake technical telemetry
- meaningless 3D effects
- unnecessary glassmorphism
- consumer-SaaS gimmicks

- decorative charts with no business meaning

Use progressive disclosure.

Show complexity when the user needs it.

Hide complexity when the user does not.

==========
40. EXECUTIVE VIEW
==========

The Chairman's experience should be treated as an executive control surface,
not a task-management interface.

It should aggregate:

- enterprise operational health
- commercial activity
- royalty health
- critical escalations
- workforce visibility
- major anomalies
- trend information
- performance / recognition
- significant exceptions

Do not place routine employee actions in the Chairman's primary workflow.

==========
41. MANAGERIAL VIEW
==========

Managers need operational control rather than decorative analytics.

The Lead Manager in particular must be able to:

- see team availability
- see workload
- assign work
- reassign work
- monitor task progress
- review submissions
- resolve blockers
- intervene on overdue tasks
- approve / request changes
- manage leave / approvals
- observe team performance

Senior Manager and General Manager should operate on broader scopes.

==========

42. EMPLOYEE VIEW
==

Execution workers should have a task-centered experience:

My Workspace
→ Current Work
→ Task Detail
→ Work Activity
→ Blocker
→ Submission
→ Review
→ Revision
→ Completion

The employee should not be exposed to unnecessary administrative complexity.

==
43. ANALYTICAL FEEDBACK LOOP
==

The architecture must demonstrate a feedback loop:

Operational Execution
→ Data
→ Analysis
→ Insight
→ Management Attention
→ Intervention
→ New Operational Outcome
→ New Data

This is what makes IOTS an operational intelligence system rather than just
a tracking application.

==
44. ARCHITECTURAL RELATIONSHIPS THAT MUST BE VISIBLE
==

The final architecture must make these relationships obvious:

Authentication
→ Authorization
→ Workspace

Workspace
→ Operational Actions

Operational Actions
→ Workflow

Workflow

→ Data Changes

Data Changes
→ Audit Events

Operational Data
→ Analytics

Analytics
→ Alerts / Reports / Insights

Insights
→ Management Decisions

Management Decisions
→ Operational Actions

Commercial Workflow
→ Financial / Royalty Data

Financial / Royalty Data
→ Executive Analytics

Attendance / Availability
→ Workload Management

Task State
→ Performance Metrics

Work Activity
→ Operational History

Blockers
→ Escalation

Everything important
→ Audit

==========
45. ARCHITECTURE QUALITY RULES
==========

The final architecture must:

1. Avoid spaghetti connections.
2. Avoid duplicate components.
3. Clearly distinguish domains.
4. Clearly distinguish users from system services.
5. Clearly distinguish business processes from technical infrastructure.
6. Show direction of information flow.
7. Show major decision points.
8. Show system boundaries.

9. Show trust/security boundaries where meaningful.
10. Show data ownership / responsibility.
11. Show which systems generate events.
12. Show which systems consume events.
13. Show which data is transactional.
14. Show which data is analytical.
15. Show where audit is written.
16. Show where authorization is enforced.
17. Show where workflow rules are enforced.
18. Show how exceptions return into the workflow.
19. Show how analytics feeds management.
20. Never make the architecture look more complex merely to appear technical.

==========
46. FINAL ARCHITECTURAL OUTPUT
==========

Produce the architecture as a coherent enterprise system, not a collection of unrelated diagrams.

The architecture should be capable of being decomposed into multiple views:

VIEW 1 — SYSTEM CONTEXT
Who and what interacts with IOTS?

VIEW 2 — 3-LAYER BUSINESS ARCHITECTURE
Authentication/Security
→ Management/Operations
→ Intelligence/Analytics

VIEW 3 — APPLICATION ARCHITECTURE
UI
→ Application Services
→ Domain Services
→ Supporting Services
→ Data

VIEW 4 — IDENTITY / RBAC ARCHITECTURE

VIEW 5 — TASK WORKFLOW / STATE MACHINE

VIEW 6 — COMMERCIAL LEAD → DEAL → ROYALTY FLOW

VIEW 7 — DATA / DOMAIN ARCHITECTURE

VIEW 8 — AUDIT / SECURITY ARCHITECTURE

VIEW 9 — ANALYTICS / REPORTING PIPELINE

VIEW 10 — DEPLOYMENT / INFRASTRUCTURE VIEW

VIEW 11 — ROLE / WORKSPACE ARCHITECTURE

```
VIEW 12 — CROSS-SYSTEM EVENT / NOTIFICATION ARCHITECTURE

These views must agree with one another.

Do not create contradictions between diagrams.

============================================================
47. IMPORTANT: DO NOT INVENT MISSING BUSINESS RULES
============================================================

Where the provided CIRIL documentation defines a rule, preserve it.

Where the documentation does NOT define a rule:
- mark it as configurable
- mark it as pending business confirmation
- or explicitly identify it as an architectural recommendation

Do not silently invent:
- new roles
- new approval authorities
- financial rules
- employee policies
- legal policies
- exact infrastructure vendors
- exact databases
- exact cloud providers
- exact API providers
- exact security algorithms

The architecture must distinguish:

DOCUMENTED REQUIREMENT
vs.
ARCHITECTURAL RECOMMENDATION
vs.
IMPLEMENTATION DECISION

============================================================
48. FINAL DESIGN OBJECTIVE
============================================================

The finished IOTS architecture should communicate three things simultaneously:

1. TO MANAGEMENT:
```

What I deliberately kept out of this first version

I **did not yet prescribe the exact Draw.io arrangement** — where the boxes go, whether the principal flow is left-to-right or top-to-bottom, where the three layers sit spatially, where

the role lanes go, where the databases sit, how connectors return, how the hexagon/router should be positioned, etc.

That should come from your `.drawio` file because your previous architecture work already established that **visual grammar matters almost as much as the logical grammar**. The earlier design discussion specifically emphasized separating a process architecture from a mere organizational hierarchy and using clean swimlanes, standard process/decision/data shapes, and orthogonal routing rather than spaghetti connectors. gemini-chattings

Also, both current documents strongly reinforce the core system mechanics around the **dual-level task state model, work activities, overdue intervention, reassignment, correction cycles, and immutable audit history**. claude_version gemini_version

And the commercial side is not an afterthought: the documents explicitly connect **mandate → destination handoff → sourcing → negotiation → closure → brokerage split → CIRIL royalty**, with the current documented model of 30% / 70% and 7.5%. claude_version

So at this point I have the **architecture content and system logic** from both AI versions. Upload the **Draw.io** `.drawio` **file**, and I'll use its actual structure to produce **Version 2 of this prompt with exact visual/layout instructions**, including what goes where, how the flows should be routed, which components should be grouped, what should be represented as layers vs domains vs services, and how to keep the final architecture readable at enterprise scale.

Employee Dashboard:

Work Management: {My Task Workspace, Task Queue, Every/Specific Task Details, Task Priority, Due Date Tracking}

Attendance & Availability: {Check-In (at login time), Check-Out (at logoff time), Attendance History, Work Hours (Compute), Availability Status {Busy, OnBreak, Offline, Absent/OnLeave, out of office etc..,.}}

Show more

Worked for 32s

Yes, mate — **that is exactly the earlier structure**. I went back through the older discussion rather than reconstructing it from memory, and I found the three-character model we built **before we expanded the system into the current 11 roles**. gemini-chattings

The important thing is that your current **Employee Dashboard** list is actually a much more mature version of that original Employee branch. The earlier model was simpler at the role level, while your current model has evolved into a proper workspace/module architecture.

The earlier 3-character Management Layer

At that stage, Layer 2 was deliberately represented using:

Admin → Manager → Employee

The idea was:

```
Authentication & Authorization Layer
            ↓
       Decide User Role
        ↙      ↓      ↘
     Admin   Manager   Employee
```

The three paths were intentionally designed as **parallel role-specific interfaces**, rather than trying to model every organizational designation individually. gemini-chattings

---

1. ADMIN — earlier structure

The Admin side was the most system-control-oriented.

We eventually organized it into **6 major branches**:

A. User Management

```
User Management
    ↓
Create a NEW User
    ↓
Set User Role & Permissions
    ↓
Activate / Deactivate Account
    ↓
Reset Password
    ↓
Record Change in Audit Log
```

The original discussion explicitly described this as the **employee/account lifecycle**: create, update, disable, and record the change. gemini-chattings

B. Location & Attendance Control

```
Location & Attendance
    ↓
Set Office Location Coordinates
    ↓
Define Standard Working Hours
    ↓
Monitor Daily Attendance
    ↓
Review Off-site / Missed Login
    ↓
Update / Record the Result
```

This was primarily about the Admin **controlling the rules and reviewing exceptions**, while Layer 1 generated the location information. gemini-chattings

C. Global Task Management

```
Global Task Management
    ↓
Monitor All Tasks
    ↓
Select Specific Task for Intervention
    ↓
Edit Details / Reassign / Cancel
    ↓
Override / Force Approve Where Authorized
    ↓
Record Change in Audit Log
```

This was intended as a **global oversight capability**, not the normal day-to-day task assignment path.

D. Master System Settings

```
System Settings
    ↓
Access System Registry
    ↓
Manage Roles / Designations
    ↓
Configure Automated Report Settings
    ↓
Update Notification Templates
```

```
    ↓
Save Changes
```

E. Security & Audit Viewer

This one was deliberately different because it was largely read-oriented:

```
Security & Audit Viewer
    ↓
Access Security Logs
    ↓
Filter by Date / User / Action
    ↓
Review Login & User Activity
    ↓
Review Task Modification History
    ↓
Export Security Reports
```

The earlier design explicitly described this as a **read-only/watchtower-style flow**.

gemini-chattings

F. Global Announcements

```
Global Announcements
    ↓
Create New Notice
    ↓
Select Target Audience
    ↓
Set Display Duration
    ↓
Publish to Dashboards
    ↓
Record Action
```

So the **original Admin architecture** was effectively:

```
ADMIN DASHBOARD
│
├── User Management
├── Location & Attendance
├── Global Task Management
├── System Settings
├── Security & Audit Viewer
└── Global Announcements
```

That six-branch structure is directly reflected in the earlier Draw.io-oriented work.

gemini-chattings

2. MANAGER — earlier structure

This is where our thinking evolved quite a bit.

Initially, we made it purely about:

Task Origination + Task Monitoring/QA

But then you correctly questioned that:

> a real manager doesn't only assign tasks.

That caused us to redesign the Manager section around **actual day-to-day managerial responsibility**.

The three main branches we settled on were:

A. Lead Allocation & Operations

Later, because you said to forget the lead concept for this monitoring portal, this became more generally:

Task / Work Allocation & Operations

The earlier concept was:

```
Manager
    ↓
Initiate Work / Assignment
    ↓
Define Task / Deliverables
    ↓
Set Priority & SLA / Deadline
    ↓
Assess Employee Bandwidth
    ↓
Assign Work
    ↓
Monitor Progress
```

The especially important idea was that the system should consider **employee workload before assignment**, rather than a manager blindly assigning work. gemini-chattings

B. Team Performance & Attendance

```
Team Performance & Attendance
    ↓
View Team Live Status
    ↓
Review Attendance / Work Hours
    ↓
Review Availability
    ↓
Handle Leave / Permission Requests
    ↓
Monitor Team Productivity
```

This was the point where we separated the **Admin's global attendance control** from the **Manager's team-level operational use of attendance**.

Admin establishes/maintains the broader rules.

Manager uses attendance and availability to actually manage the team.

C. QA & Issue Escalation

```
QA & Issue Escalation
    ↓
Monitor Active Work
    ↓
Detect Blocked / Delayed Work
    ↓
Intervene / Resolve Issue
    ↓
Review Deliverable
    ↓
Approve
      OR
Request Changes
    ↓
Close Work
```

The **employee → manager notification/help-needed mechanism** also came from this phase.

The employee doesn't have to physically walk over and ask the manager for help. The system can create a visible intervention request for the manager. gemini-chattings

---

Then the Manager task flow became more advanced

This was probably the most important thing we developed in the old three-role model.

Instead of:

```
Create
→ Assign
→ Work
→ Review
```

we evolved it toward:

```
Manager Dashboard
      ↓
Initiate Assignment
      ↓
Define Deliverables
      ↓
Set Priority / Deadline
      ↓
Assess Employee Bandwidth
      ↓
Assign Employee
      ↓
Notify Employee
      ↓
Monitor Execution
      ↓
Blocker / Overdue Handling
      ↓
Receive Deliverable
      ↓
QA Review
      ↓
Approved?
   ↙       ↘
 YES        NO
  ↓          ↓
Close     Feedback
            ↓
        Revision
            ↓
        Resubmit
```

That is where the **task state controller**, overdue intervention, reassignment, and revision loop later emerged. gemini-chattings

---

3. EMPLOYEE — earlier structure

And this is the part you are now showing me in the much more evolved form.

The original Employee side was essentially:

Work / Task Execution

```
Employee Dashboard
    ↓
Receive Assigned Task
    ↓
View Task Details
    ↓
Start / Continue Work
    ↓
Update Progress
    ↓
Record Work Activity
    ↓
Upload Supporting Work
    ↓
Submit for Review
```

Then:

```
Manager Reviews
     ↓
Approved?
  ↙       ↘
YES       NO
 ↓         ↓
Close    Feedback
           ↓
       Correction
           ↓
       Resubmit
```

The later system specification made this considerably richer by explicitly separating **task state** from **work activity**. Work Activity became:

- progress updates
- work notes
- time/effort
- supporting evidence

while the task itself carries its workflow state. claude_version

---

And this is where your current Employee Dashboard comes from

Your current structure:

```
Employee Dashboard

1. Work Management
   - My Task Workspace
   - Task Queue
   - Task Details
   - Priority
   - Due Date Tracking

2. Attendance & Availability
   - Check-In
   - Check-Out
   - Attendance History
   - Work Hours
   - Availability Status

3. Request & Self-Service
   - Leave Requests
   - Permission Requests
   - Request Status
   - Request Logs
   - Manager Approval History
   - Scheduled Requests
   - Task Clarification
   - Resource Requests

4. Notification & Communication
   - New Task
   - Reassigned Task
   - Overdue Task
   - Deadline Alerts
   - Manager Feedback
   - Announcements
   - Escalation Alerts

5. Performance & Activity
   - Task Activity Summary
   - Completion Metrics
   - Attendance Summary
   - Activity History
   - Feedback History

6. Profile & Security / Usage
   - My Profile
   - Account Details
   - Edit Requests
   - Password Change
   - Active Sessions
```

```
7. Intelligence & Analytical Layer
   - Daily
   - Weekly
   - Monthly
   - Role-Based Reports
   - Team Analytics
   - Performance Analytics
   - Workload Analytics
   - Attendance Analytics
   - Exception / Risk Analysis
   - etc.
```

That is **not a different concept from the original Employee interface**.

It is the **expanded, mature version** of it.

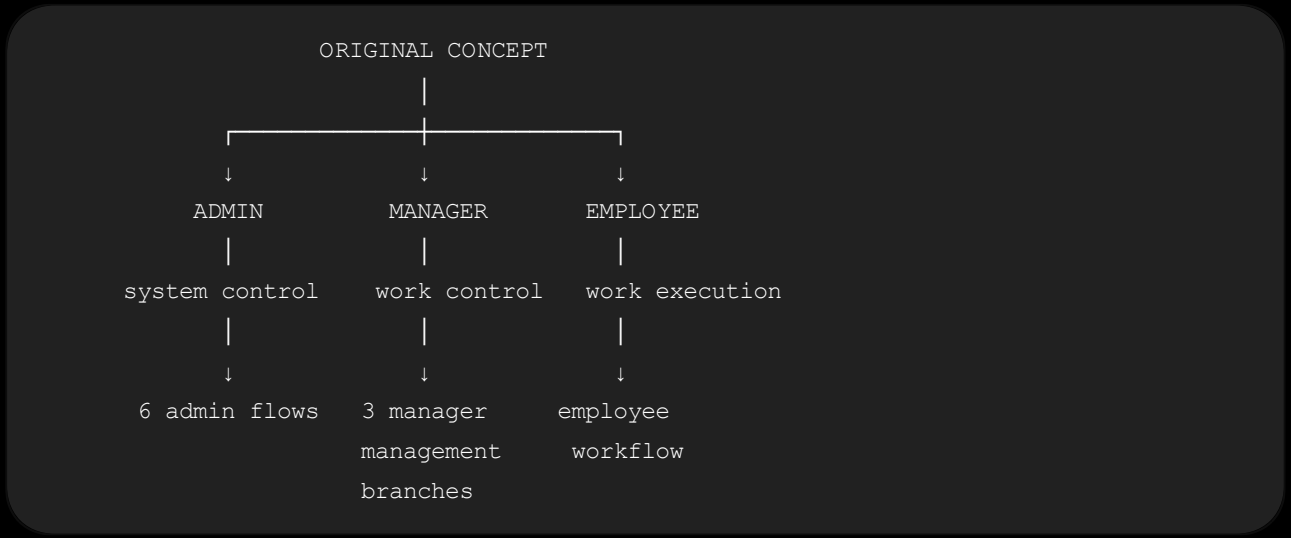
The current enterprise documents reinforce the same foundation: Employee workspace, Active Tasks, Work Activity, Blockers, Requests, Attendance, and My Performance.

claude_version

The evolution is actually important

This is the part I think you were remembering.

We went roughly like this:



Then the model matured:

```
ADMIN
 ↓
User / Location / Tasks / Settings / Audit / Announcements

MANAGER
 ↓
Task Management
```

```
Team Management
Attendance / Availability
Performance
Approvals
Blockers / Escalation
Reports

EMPLOYEE
 ↓
Work Management
Attendance
Requests
Communication
Performance
Profile
Analytics
```

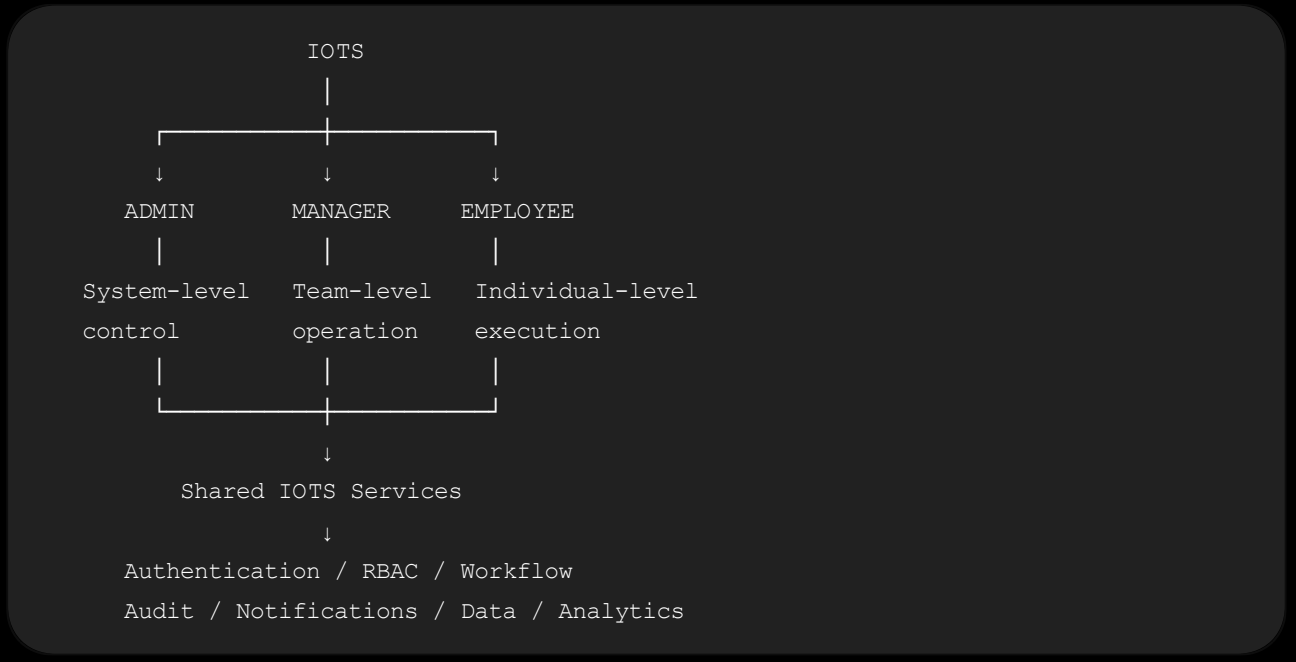
And **only after that** did we expand the organizational model into the current 11 roles.

The current architecture documents explicitly describe the 11 roles and their separate workspaces, while retaining the common application shell and the shared task/workflow infrastructure. claude_version +1

So yes — your memory is correct.

The **Admin / Manager / Employee model was our architectural abstraction layer**, not the final organizational-role model.

And that distinction is actually useful for the architecture you're about to draw:



Then **inside those three conceptual characters**, the 11 actual roles are mapped.

That gives us a much cleaner architecture than trying to put all 11 roles directly into the first high-level diagram. The earlier discussion was already moving in this direction.

gemini-chattings

And yes, I'll use **this historical progression** when interpreting the `.drawio` file you said you're going to upload.

Actual-FlowChart(1).dr…
File

Pasted markdown(1).md
File

Based on this file now refactor your response, I attached your response as text too

Worked for 38s

Yes, mate. Now that I've inspected the actual `Actual-FlowChart(1).drawio` together with the markdown containing our earlier architecture response, I would **not** use the previous prompt as-is.

The `.drawio` shows the architectural language you have already committed to: a **top Authentication & Authorization section**, followed by a large **Management Layer**, with **Admin on the left, Manager in the center, Employee on the right**, while the Manager flow continues downward into the task-state/workflow engine. The file also has a separate **"Basic Overview"** page that acts as the cleaner reference version of that same structure. Your attached markdown confirms that this three-character abstraction was deliberately established before the later 11-role expansion. Pasted markdown

So the prompt should now tell the AI **to architect around your existing diagram rather than invent another architecture**.

Refactored Master Prompt — IOTS Architecture

Based on the Actual Draw.io Structure

Copy this as the new master prompt:

```
You are acting as a Senior Enterprise Systems Architect, Solution Architect,
Business Process Architect, Data Architect, Security Architect, Workflow
Architect, and UI/UX Systems Architect.
```

You are designing the architecture of:

CIRIL
INTERNAL OPERATIONS TRACKING SYSTEM (IOTS)

Organization:
Commercial Real Estate Institutional Alliance (CIRIL)

==
IMPORTANT SOURCE MATERIAL
==

You have been provided with:

1. A detailed IOTS enterprise specification generated from previous AI-assisted
   architecture work.

2. A Draw.io file named:
   Actual-FlowChart(1).drawio

3. A separate "Basic Overview" view inside the Draw.io file.

4. A previous architecture markdown document describing the evolution of the
   Admin → Manager → Employee model.

Treat these sources as the primary design basis.

DO NOT replace the established architecture with a generic CRM, HRMS,
ERP, SaaS dashboard, or generic software architecture.

DO NOT independently redesign the visual structure before understanding the
existing Draw.io organization.

The objective is:

REFINE + COMPLETE + ARCHITECT
the existing IOTS design.

==
1. EXISTING ARCHITECTURAL VISUAL LANGUAGE
==

The provided Draw.io already establishes the primary visual architecture.

Preserve this conceptual layout.

The architecture has two major visible sections:

SECTION 1
Authentication & Authorization Layer

SECTION 2
Management Layer

Within the Management Layer, the existing high-level role abstraction is:

ADMIN
MANAGER
EMPLOYEE

These are conceptual operational lanes.

IMPORTANT:

This three-character model is the visual/system abstraction already established in the current architecture.

It existed before the system was expanded into the detailed organizational roles.

Therefore:

Do NOT immediately replace Admin / Manager / Employee with 11 separate role lanes.

Instead:

Admin / Manager / Employee remain the top-level operational interface categories in the architecture.

The detailed organizational roles can be represented as role mappings beneath these categories where required.

==========
2. REQUIRED HIGH-LEVEL ARCHITECTURE
==========

Preserve this fundamental sequence:

START
  ↓
USER LOGS INTO SYSTEM
  ↓
VALIDATE CREDENTIALS
  ↓
VALIDATE LOCATION
  ↓
AUDIT LOGIN + LOCATION CONTEXT
  ↓
DECIDE USER ROLE
  ↓
┌──────────┬──────────┬──────────┐
│   │   │

```
ADMIN      MANAGER       EMPLOYEE
|            |             |
|            |             |
▼            ▼             ▼
Admin      Manager       Employee
Interface  Interface      Interface
```

The diagram should visually make this routing obvious.

==========
3. DRAW.IO LAYOUT MUST BE RESPECTED
==========

The existing Draw.io composition should be treated as the layout foundation.

Use this spatial interpretation:

TOP:
Authentication & Authorization

MIDDLE / LOWER:
Management Layer

LEFT SIDE:
Admin Dashboard

CENTER:
Manager Dashboard

RIGHT SIDE:
Employee Dashboard

The Manager workflow is the central operational flow and should remain the principal visual spine of the Management Layer.

Admin functionality occupies the left operational zone.

Employee functionality occupies the right operational zone.

Do not move these three conceptual areas into unrelated locations.

Do not create a completely different architecture style.

==========
4. BASIC OVERVIEW VS SYSTEM ARCHITECTURE
==========

The Draw.io file contains two related views:

A. Basic Overview
B. System Architecture

Use them differently.

BASIC OVERVIEW:
Should communicate the overall business/system flow quickly.

It should show:

Authentication
→ Location
→ Audit
→ Role Decision
→ Admin / Manager / Employee
→ Major operational flow
→ Reporting / intelligence

This view must remain understandable to a Chairman or non-technical stakeholder.

SYSTEM ARCHITECTURE:
Contains the deeper operational detail.

This view should contain:
- Admin functional branches
- Manager operational branches
- Employee work branches
- workflow/state logic
- exception handling
- audit flow
- analytics/reporting connections
- shared services where necessary

Do not place every technical implementation detail into the Basic Overview.

==========
5. AUTHENTICATION & AUTHORIZATION LAYER
==========

Use the existing top-layer structure as the starting point.

Current flow:

START
→ User Logs into the System
→ Validate Credentials
→ Validate Location
→ Audit Login & Location Coordinates
→ Decide User Role
→ Route to appropriate interface

Credential failure path:

Validate Credentials

→ Invalid
→ Notify Invalid Credentials
→ Return to login

Location path:

Validate Location
→ Known / recognized
→ Continue

Validate Location
→ Unknown / off-site
→ Mark Session as Remote / Off-Site
→ Continue into the normal system flow

IMPORTANT:

An off-site location must NOT automatically terminate the workflow unless an explicit business rule says so.

The current system concept is location awareness and tracking, not blindly blocking valid employees.

The location event should feed:

- attendance
- security monitoring
- audit
- management visibility
- analytics

==========
6. ROLE RESOLUTION
==========

The existing diagram uses:

DECIDE USER ROLE

This must remain system-driven.

Do NOT ask the user to manually choose:
Admin
Manager
Employee

The system should determine the user's effective access context based on their account and assigned role(s).

For users with multiple authorized roles:

User Identity

→ Authorized Roles
→ Active Role Context
→ Workspace
→ Permissions

A multi-role user should not need multiple accounts.

Every important action should retain the identity of:
- user
- active role
- action

==
7. ADMIN DASHBOARD ARCHITECTURE
==

The current Draw.io layout already gives Admin six major functional areas.

Preserve these six branches:

1. User Management
2. Location & Attendance Control
3. Global Task Management
4. Announcement Dashboard
5. System Settings
6. Security & Audit Viewer

These are not merely menu labels.

Each must be represented as an operational flow.

7.1 USER MANAGEMENT

Use the established flow:

User Management
→ Create New User
→ Set User Role & Permissions
→ Activate / Deactivate User
→ Reset Password where authorized
→ Record Change in Audit Log

Support:
- employee profile creation
- role assignment
- department
- reporting manager
- account status
- account lifecycle

Do not create a public registration flow.

7.2 LOCATION & ATTENDANCE CONTROL

Use the existing branch:

Location & Attendance Control
→ Set Office Location Coordinates
→ Define Standard Working Hours
→ Monitor Daily Attendance Records
→ Review / Resolve Off-Site / Flagged Logins
→ Record the outcome

This is the administrative control view.

Do not confuse this with the Employee's own attendance interface.

Admin:
controls / reviews organizational attendance information.

Employee:
records / views personal attendance.

Manager:
uses team attendance and availability operationally.

7.3 GLOBAL TASK MANAGEMENT

This is global oversight, not ordinary task assignment.

Flow:

Global Task Management
→ Monitor All Tasks
→ Select Specific Task for Intervention
→ Edit / Reassign / Cancel where authorized
→ Force-Approve or intervene only where explicitly authorized
→ Record Change in Audit Log

Do not make this the normal employee task lifecycle.

The normal task lifecycle remains under Manager / operational workflow.

7.4 ANNOUNCEMENT DASHBOARD

Flow:

Announcement Dashboard
→ Create New Notice / Message
→ Select Audience
→ Set Display Duration / Scheduling
→ Publish to Appropriate Dashboards
→ Record Publishing Action

Audience may include:
- all users
- selected role groups
- selected departments / teams
- selected operational scopes

7.5 SYSTEM SETTINGS

Flow:

System Settings
→ Access System Registry
→ Manage Configurable Roles / Designations
→ Manage Automated Report Settings
→ Manage Notification Templates
→ Save Configuration
→ Audit Configuration Change

Do not give routine Admin access to unrestricted platform governance unless the authorization model explicitly allows it.

System-level governance must remain separated from routine administration.

7.6 SECURITY & AUDIT VIEWER

This is primarily a controlled read / inspection flow.

Flow:

Security & Audit Viewer
→ Access Secure System Logs
→ Apply Filters
→ Review User Activity / Login History
→ Review Task Modification History
→ Review Security Events
→ Export Authorized Security Reports

The audit history itself must not be edited through this interface.

==========

8. MANAGER DASHBOARD ARCHITECTURE
==========

The Manager area is the central operational control area of the diagram.

The Manager is NOT simply:

Create Task
→ Assign Task
→ Review Task

The established architecture must reflect what a real operational manager does.

The Manager area should contain these major functional domains:

1. Task Management & Workflow
2. Team Management
3. Performance Monitoring
4. Workload Management
5. Attendance Management
6. Team Communication
7. Feedback & Evaluation
8. Issues / Escalation Management

These should exist as structured functional branches around the central task workflow.

==========
9. MANAGER — TEAM MANAGEMENT
==========

Use the existing Draw.io structure as the foundation:

Team Management
→ View Team Members
→ View Employee Profiles
→ View Current Assignments
→ View Workload & Availability
→ Track Work Progress

Extend this conceptually to:

- team composition
- responsibilities
- active work
- availability
- assignments
- workload
- current blockers
- recent activity

The Manager should see only the organizational scope permitted by their role.

==========
10. MANAGER — PERFORMANCE MONITORING
==========

Flow:

Performance Monitoring
→ Monitor Employee Task Completion
→ Review Overdue Tasks
→ Track Deadlines
→ Identify Performance Issues
→ Give Feedback
→ Monitor Improvement

Metrics should be based on operational records rather than decorative judgement.

Candidate performance dimensions:

- task completion
- on-time completion
- overdue frequency
- turnaround time
- revision frequency
- work activity
- attendance summary
- role-specific target achievement

IMPORTANT

Do not make one universal KPI formula for every role.

Different operational roles may require different performance indicators.

==========
11. MANAGER — WORKLOAD MANAGEMENT
==========

Use the existing structure:

Workload Management
→ View Employee Workload
→ Identify Overloaded Employees
→ Check Employee Availability
→ Balance Task Distribution
→ Reassign Task if Required

This is one of the defining characteristics of the IOTS manager architecture.

Before assigning work:

Manager
→ Inspect current workload
→ Inspect availability
→ Determine capacity
→ Assign work

This prevents the task system from becoming a simple to-do list.

==========
12. MANAGER — ATTENDANCE MANAGEMENT
==========

Flow:

Attendance Management
→ View Employee Attendance
→ Review Location / Late / Absence Records
→ Review Requests
→ Approve / Reject Leave where authorized
→ Adjust Workload if Required

Keep this separate from administrative attendance configuration.

The distinction is:

ADMIN:
maintains rules and performs organizational review.

MANAGER:
uses attendance information to manage their team.

EMPLOYEE:
records and views their personal attendance.

==========
13. MANAGER — TEAM COMMUNICATION
==========

Flow:

Team Communication
→ Send Team Announcement
→ Send Employee-Specific Message
→ Share Task Instructions
→ Send / Trigger Deadline Reminder
→ Receive Employee Updates

Communication should interact with the task and request systems where
necessary.

Do not create a disconnected chat system unless there is a clear business requirement.

==========
14. MANAGER — FEEDBACK & EVALUATION
==========

Flow:

Feedback & Evaluation
→ Receive Employee Feedback
→ Review Employee Work
→ Provide Feedback
→ Record Evaluation
→ Monitor Improvement

Feedback must remain connected to the relevant work record.

A feedback event may relate to:
- task
- submission
- revision
- employee performance
- blocker resolution
- operational request

==========
15. MANAGER — ISSUES MANAGEMENT
==========

Use the existing concept already represented in the Draw.io file.

Flow:

Issues Management
→ Identify Issue
→ Assess Severity
→ Manager Review
→ Resolve
OR
→ Escalate to Higher Authority
→ Audit the Decision
→ Monitor Resolution / Improvement

This is the employee-to-manager intervention mechanism.

The employee should be able to say:

"Help Needed"

or

"Raise Issue"

without needing to leave the system.

==
16. CENTRAL MANAGER TASK MANAGEMENT FLOW
==

The existing Draw.io task flow is a major architectural element.

Keep it as the central workflow spine.

The architecture should be:

Manager Dashboard
→ Task Management & Workflow
→ Create Task
→ Set Priority & Due Date
→ Check Employee Workload
→ Assign Task
→ Notify Employee
→ Task Status Controller
→ Employee Works & Updates
→ Task Completion / Review
→ Approval / Corrections
→ Completion

==
17. TASK STATUS CONTROLLER
==

The existing Draw.io contains a central:

TASK STATUS

controller/router.

Keep this concept.

The controller is the central state-routing mechanism.

Do NOT represent task statuses merely as disconnected boxes.

It must control state transitions.

Macro reporting states:

- Assigned
- Pending
- Overdue
- Reassigned
- Completed

Operational execution states:

- In Progress
- Blocked
- Waiting for Review
- Under Review / QA
- Changes Requested
- Issue Resolved
- Done / Closed

Important:

Reassigned is a transition/event, not a terminal state.

Overdue is an exception requiring managerial intervention.

Completed is reached after appropriate review/approval.

==========
18. TASK EXECUTION FLOW
==========

The Employee-side task workflow should connect directly to the central Manager workflow.

Flow:

Task Assigned
→ Employee Receives Notification
→ Employee Opens Task
→ Employee Starts / Continues Work
→ Updates Progress
→ Records Work Activity
→ Uploads Evidence
→ Raises Issue if Blocked
→ Resolves Blocker / Continues
→ Submits Work
→ Manager Review
→ Approved OR Changes Requested
→ Resubmission if required
→ Final Approval
→ Complete & Close

==========
19. OVERDUE TASK ARCHITECTURE
==========

Overdue must not simply become another green or passive status.

Flow:

```
Active Task
→ Due Date Reached
→ OVERDUE
→ Manager Review / Intervention

Manager intervention may include:

- extend deadline
- reassign task
- provide guidance
- reduce or revise scope where authorized
- escalate

Any high-impact change should preserve:
- reason
- actor
- timestamp
- previous state
- new state

================================================================
20. REASSIGNMENT ARCHITECTURE
================================================================

Flow:

Task
→ Reassign
→ Review Employee Capacity
→ Select New Employee
→ Record Reason
→ Handover Context
→ Notify New Employee
→ Continue Task
→ Record Audit Event

Never represent reassignment as deletion of the old assignment.

Preserve historical ownership.

================================================================
21. TASK REVIEW / CORRECTION LOOP
================================================================

Flow:

Employee Submission
→ Manager Reviews
→ Approved?

YES
→ Complete & Close
```

NO
→ Provide Feedback / Corrections
→ Employee Continues Work
→ Update Work
→ Resubmit
→ Manager Reviews Again

The correction loop must preserve previous submissions and feedback.

Do not use a destructive "Reject and recreate" pattern.

==
22. EMPLOYEE DASHBOARD ARCHITECTURE
==

Use the employee model already developed in the previous architecture.

The Employee Dashboard should contain:

1. Work Management
2. Attendance & Availability
3. Request & Self-Service
4. Notification & Communication
5. Performance & Activity
6. Profile & Security / Usage
7. Intelligence & Analytical / Reports

22.1 WORK MANAGEMENT

Work Management:

→ My Task Workspace
→ Task Queue
→ Task Details
→ Priority
→ Due Date Tracking

Every task should expose the appropriate details without overwhelming the employee.

Task detail should conceptually contain:

- task brief
- deliverables
- priority
- due date
- current state
- progress
- work activity

- evidence
- blocker
- submission
- review
- feedback
- history

==
23. EMPLOYEE — ATTENDANCE & AVAILABILITY
==

Attendance & Availability:

→ Check-In
→ Check-Out
→ Attendance History
→ Work Hours Calculation
→ Availability Status

Availability examples:

- Available
- Busy
- On Break
- In Meeting
- In Training
- Do Not Disturb
- Working Remotely
- On Leave
- Absent
- Offline

Do not assume every listed status must remain in the final system;
treat them as the current candidate set.

Attendance answers:
"Am I present / working?"

Availability answers:
"Am I currently available for work?"

==
24. EMPLOYEE — REQUEST & SELF-SERVICE
==

Request & Self-Service:

→ Leave Request
→ Permission Request
→ Remote Work / Scheduled Request
→ Task Clarification
→ Resource Request

→ Request Status
→ Request History
→ Approval History

Requests should follow a controlled lifecycle:

Create Request
→ Submit
→ Manager Review
→ Approve / Decline / Request Clarification
→ Notify Employee
→ Record History

==========
25. EMPLOYEE — NOTIFICATION & COMMUNICATION
==========

Notification & Communication should capture system-generated events such as:

- new task assigned
- task reassigned
- task overdue
- approaching deadline
- manager feedback
- submission approved
- changes requested
- issue response
- leave decision
- request decision
- announcement
- escalation

The notification system should be event-driven and role-aware.

==========
26. EMPLOYEE — PERFORMANCE & ACTIVITY
==========

Performance & Activity:

→ Task Activity Summary
→ Completion Metrics
→ Attendance Summary
→ Activity History
→ Feedback History
→ Target / KPI Progress where applicable

Employee visibility should focus primarily on their own permitted data.

Performance data should be explainable.

A user should be able to understand where a metric originated.

==
27. EMPLOYEE — PROFILE / SECURITY / USAGE
==

Profile & Security / Usage:

→ My Profile
→ Account Details
→ Request Account Changes
→ Password Change
→ Active Sessions
→ Security History where permitted

Profile modifications that require approval should follow a request workflow
rather than silently altering authoritative employee records.

==
28. EMPLOYEE WORK ACTIVITY
==

Work Activity must remain separate from task state.

Work Activity contains:

1. Progress Update
2. Work Notes
3. Time / Effort Entry
4. Supporting Evidence / Documents

Conceptually:

TASK
→ STATE HISTORY

and:

TASK
→ WORK ACTIVITY HISTORY

These are related but not the same object.

==
29. DOCUMENT / EVIDENCE MANAGEMENT
==

The architecture must support:

Task
→ Deliverable
→ File
→ Version

→ Submission
→ Review
→ Revision
→ Resubmission
→ Approval

Important for:
- presentations
- spreadsheets
- reports
- design files
- screenshots
- supporting documents
- operational evidence

Do not overwrite important historical versions.

==
30. INTELLIGENCE & ANALYTICAL LAYER
==

The Intelligence & Analytical Layer is a SYSTEM-WIDE downstream layer.

It should consume operational data from:

- tasks
- work activity
- attendance
- availability
- requests
- blockers
- escalations
- performance
- commercial operations where applicable
- audit events

Architecture:

Operational Data
→ Data Preparation / Aggregation
→ Metrics
→ Trend Analysis
→ Exception / Risk Analysis
→ Insights
→ Reports / Alerts
→ Management Decisions

This layer should not become a decorative chart collection.

==
31. REPORTING RHYTHM
==

Reports should be organized into:

DAILY

WEEKLY

MONTHLY

and may additionally support:

- role-based reports
- team reports
- department reports
- management reports
- employee reports
- operational summaries
- exception reports
- performance reports

DAILY

Potential content:

- task status
- completed work
- overdue work
- blockers
- attendance exceptions
- important requests
- today's priorities

WEEKLY

Potential content:

- team performance
- workload
- SLA delivery
- overdue trends
- task bottlenecks
- attendance patterns
- manager actions

MONTHLY

Potential content:

- organizational performance
- role-specific KPIs
- task completion
- SLA trends
- attendance patterns
- exception trends
- commercial / royalty metrics where applicable
- management summary

Do not force every report to contain every metric.

==========
32. PERFORMANCE ANALYTICS
==========

Performance Analytics should answer:

"How well is work being delivered?"

Include, where relevant:

- task completion rate
- on-time completion
- average completion time
- target achievement
- revision frequency
- workload-adjusted performance
- team performance
- manager performance
- employee-specific performance
- role-specific KPI performance

Do not use one KPI formula for all roles.

==========
33. TASK & WORKLOAD ANALYTICS
==========

Answer:

"How is work distributed and where are bottlenecks forming?"

Include:

- task distribution
- pending tasks
- overdue tasks
- reassignment frequency
- employee workload
- team workload

- workload concentration
- completion rate
- active queue size
- blocked work
- review backlog

==========
34. ATTENDANCE ANALYTICS
==========

Include:

- late arrival trends
- absence trends
- attendance patterns
- work-hour patterns
- availability patterns
- office / remote patterns
- team workforce availability
- recurring attendance exceptions

Do not automatically interpret every attendance anomaly as poor performance.

==========
35. EXCEPTION / RISK ANALYSIS
==========

The Risk Engine should consume operational events and detect:

- overdue tasks
- tasks approaching deadline
- excessive workload
- repeated reassignment
- unresolved blockers
- pending approval queues
- unusual attendance patterns
- security anomalies
- operational delays

Risk classification:

LOW
MEDIUM
HIGH

The exact thresholds should be configurable rather than hard-coded into the architecture unless formally approved.

==========
36. ADDITIONAL INTELLIGENCE AREAS
==========

The architecture may later include:

- trend analysis
- bottleneck detection
- workload forecasting
- SLA breach warnings
- repeated issue patterns
- recurring reassignment patterns
- team capacity analysis
- operational efficiency
- management action effectiveness
- request turnaround analysis
- communication responsiveness
- quality / revision analysis

Distinguish:

DETERMINISTIC RULES
from
PREDICTIVE / STATISTICAL INSIGHTS

Do not call simple threshold rules "AI" unless actual AI/ML functionality exists.

==
37. SHARED SYSTEM SERVICES
==

The architecture beneath the three operational lanes should show shared services rather than duplicating the same functionality three times.

Shared services include:

- Authentication
- Authorization / RBAC
- Session Management
- Location / Geolocation
- Attendance
- Workflow Engine
- Notification Service
- Audit Service
- Document / File Service
- Reporting Service
- Analytics Service
- Search
- Configuration
- Data Access

Use this conceptual relationship:

ADMIN
MANAGER

```
EMPLOYEE
      ↓
SHARED IOTS SERVICES
      ↓
DATA / STORAGE
```

This is important.

Do not build three independent systems.

38. DATA ARCHITECTURE

Organize data conceptually into domains.

IDENTITY
- users
- roles
- role assignments
- departments
- managers
- sessions

LOCATION / ATTENDANCE
- locations
- geofences
- login events
- attendance records
- availability
- leave

TASK / OPERATIONS
- tasks
- assignments
- task states
- task state history
- work activities
- submissions
- reviews
- feedback
- blockers
- escalations
- reassignment history

DOCUMENTS
- files
- versions
- deliverables
- attachments

COMMUNICATION

- notifications
- announcements
- messages

COMMERCIAL
- leads / mandates
- member companies
- properties
- proposals
- deals
- brokerage
- royalty settlements

ANALYTICS
- metrics
- KPI aggregates
- trend data
- risk events
- report outputs

AUDIT
- audit events

==
39. AUDIT AS A CROSS-CUTTING ARCHITECTURE
==

Do not make Audit merely another menu item.

Audit is cross-cutting.

Important operations should generate audit events.

Examples:

- login
- failed login
- location event
- account creation
- role change
- permission change
- task creation
- task edit
- assignment
- reassignment
- deadline change
- priority change
- status change
- blocker creation
- blocker resolution
- submission
- feedback

- approval
- task closure
- request approval
- attendance modification
- geofence change
- announcement publication
- system setting change
- royalty adjustment
- report generation
- privileged intervention

Audit information should preserve:

Actor
Active Role
Action
Object
Timestamp
Previous Value
New Value
Reason
Source
Relevant Context

==
40. NOTIFICATION / EVENT ARCHITECTURE
==

Operational events should be reusable.

Examples:

TASK_CREATED
TASK_ASSIGNED
TASK_REASSIGNED
TASK_OVERDUE
TASK_SUBMITTED
TASK_CHANGES_REQUESTED
TASK_APPROVED
TASK_COMPLETED
BLOCKER_RAISED
BLOCKER_RESOLVED
LEAVE_REQUESTED
LEAVE_DECIDED
ANNOUNCEMENT_PUBLISHED
SECURITY_ANOMALY_DETECTED

Conceptual flow:

EVENT
→ EVENT PROCESSING
→ AUDIT

→ NOTIFICATION
→ ANALYTICS
→ REPORTING

The same event may be consumed by multiple services.

==
41. COMMERCIAL / ROYALTY INTEGRATION
==

The IOTS architecture may contain a commercial operations domain because CIRIL works around leads, member companies, transactions and royalty outcomes.

However, do not allow the commercial domain to overwhelm the core Internal Operations Tracking System.

The commercial workflow can be represented as:

Lead / Mandate
→ Assignment / Destination
→ Operational Work
→ Deal Progress
→ Deal Conclusion
→ Brokerage
→ Royalty
→ Reporting / Analytics

Current documented business configuration includes the 30 / 70 member distribution and 7.5% CIRIL royalty.

Treat these as configurable business rules, not hard-wired architectural constants.

==
42. MANAGEMENT → INTELLIGENCE FEEDBACK LOOP
==

This relationship must be visible.

Management Layer
→ produces operational records

Operational Records
→ feed Intelligence & Analytics

Intelligence
→ produces insights / alerts / reports

Management
→ uses insights for decisions

Decisions

→ create new operational actions

Therefore:

OPERATE
→ MEASURE
→ ANALYZE
→ IDENTIFY
→ DECIDE
→ ACT
→ MEASURE AGAIN

This closed loop is one of the central purposes of IOTS.

==
43. SECURITY ARCHITECTURE
==

Security must exist across all layers.

Include:

- authentication
- authorization
- session controls
- active role context
- location context
- privileged actions
- audit
- secure document access
- protected APIs
- anomaly detection
- controlled configuration
- approval safeguards

Do not portray security as one isolated box.

==
44. FAILURE / EXCEPTION PATHS
==

Every major workflow needs an exception path.

AUTHENTICATION
→ Invalid Credentials
→ Notify
→ Retry

LOCATION
→ Unknown / Off-site
→ Mark / Record
→ Continue under policy

```
TASK
→ Overdue
→ Manager Intervention

TASK
→ Blocked
→ Issue
→ Resolve / Escalate

TASK
→ Changes Requested
→ Revision
→ Resubmit

REQUEST
→ Declined
→ Notify Employee
→ Preserve History

SECURITY
→ Anomaly
→ Alert
→ Review
→ Intervention
→ Audit

EXTERNAL SERVICE
→ Failure
→ Retry / Fallback
→ Incident
→ Audit

============================================================
45. VISUAL ARCHITECTURE RULES
============================================================

The current Draw.io style must remain the visual reference.

Use:

- large boundary containers
- clear labels
- simple process boxes
- decision diamonds
- controlled status shapes
- orthogonal/right-angle connectors
- minimal crossing lines
- clear directional flow
- visually separated functional areas

Avoid:
```

- spaghetti connectors
- unnecessary 3D
- excessive decorative shapes
- random colors
- too many unrelated arrows
- giant text blocks
- technical jargon where a business phrase works

Shape meaning must remain consistent.

Suggested:

OVAL
→ Start / End

RECTANGLE / ROUNDED RECTANGLE
→ Process / Action

DIAMOND
→ Decision

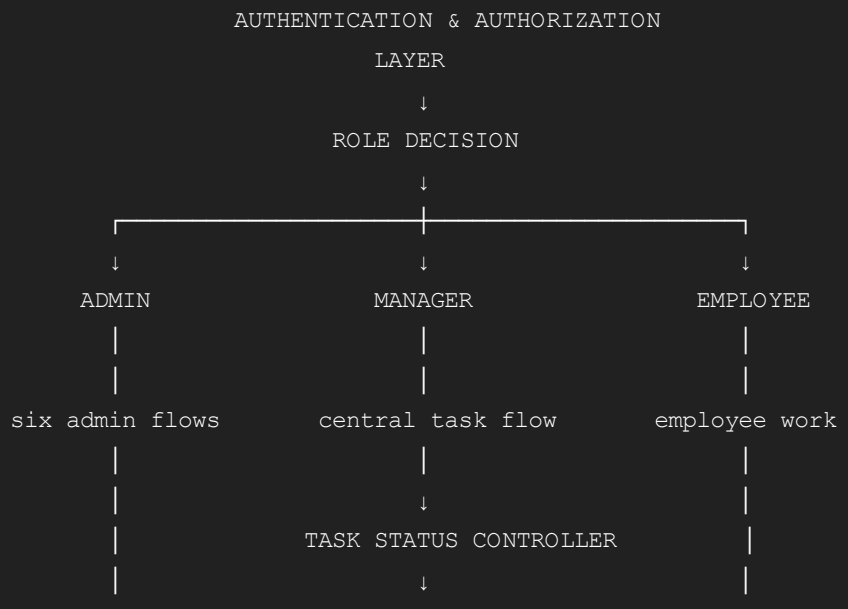
STATUS CONTROLLER / HEXAGON
→ Workflow routing / state controller

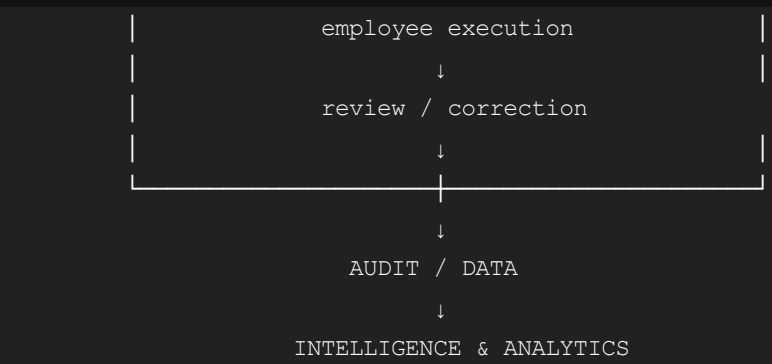
DATABASE / CYLINDER
→ Persistent data

DASHED LARGE CONTAINER
→ Architectural layer / boundary

==
46. CURRENT DRAW.IO SPATIAL STRUCTURE
==

Respect this hierarchy:





This is a conceptual layout.

Do not introduce another unrelated top-level arrangement.

==========
47. SYSTEM ARCHITECTURE VS BUSINESS FLOW
==========

Do not confuse:

BUSINESS FLOW
with
TECHNICAL ARCHITECTURE.

Business flow answers:

"What happens?"

Technical architecture answers:

"What system component makes this happen?"

Example:

BUSINESS:
Manager assigns task
→ Employee works
→ Manager reviews
→ Task closes

TECHNICAL:
Manager UI
→ Authorization
→ Task Service
→ Workflow Engine
→ Notification Service
→ Employee UI
→ Submission Service
→ Review Service
→ Audit Service
→ Database

Both should be consistent.

==========
48. WHAT THE FINAL ARCHITECTURE MUST COMMUNICATE
==========

The architecture must satisfy three audiences.

CHAIRMAN / EXECUTIVE:

"This system gives me visibility and control over the entire operation."

MANAGER:

"This system helps me manage people, workload, tasks, attendance,
exceptions and outcomes."

TECHNICAL TEAM:

"This system has properly separated identity, authorization, workflows,
data, events, audit, services, analytics and infrastructure."

==========
49. DO NOT OVER-ENGINEER THE DIAGRAM
==========

The goal is NOT to make the diagram enormous.

The goal is:

HIGH LOGICAL DENSITY
+
LOW VISUAL CONFUSION

Complexity should come from:

- relationships
- workflow rules
- state transitions
- information flow
- exception handling
- shared services

NOT from:

- excessive boxes
- unnecessary infrastructure components
- technology names everywhere
- arbitrary microservices
- decorative architecture patterns

==========

50. FINAL OUTPUT REQUIREMENTS
==========

Based on all the supplied material, produce the architecture in multiple coherent views.

VIEW A
Basic System Overview

VIEW B
Current Three-Lane Management Architecture

ADMIN
MANAGER
EMPLOYEE

VIEW C
Authentication & Authorization Flow

VIEW D
Manager Task / Workflow Engine

VIEW E
Admin Functional Architecture

VIEW F
Employee Functional Architecture

VIEW G
Shared Services Architecture

VIEW H
Data / Domain Architecture

VIEW I
Audit / Security Architecture

VIEW J
Intelligence & Analytical Architecture

VIEW K
Event / Notification Architecture

VIEW L
Commercial / Royalty Integration

VIEW M
Deployment / Infrastructure Architecture

All views must use the same terminology and underlying logic.

Do not create contradictions between diagrams.

```
================================================================
51. MOST IMPORTANT DESIGN RULE
================================================================

The provided Draw.io file is not merely an example image.

It represents the current architectural direction.

Therefore:

DO NOT START FROM A BLANK CANVAS.

FIRST:
Understand the existing layout.

SECOND:
Identify what is already structurally correct.

THIRD:
Complete missing functionality.

FOURTH:
Refine weak or ambiguous flows.

FIFTH:
Connect the layers correctly.

SIXTH:
Add deeper technical architecture underneath the established business
architecture.

SEVENTH:
Keep the final result visually compatible with the existing Draw.io
language.

================================================================
52. FINAL OBJECTIVE
================================================================

The finished IOTS architecture must demonstrate:

AUTHENTICATE
→ VERIFY CONTEXT
→ AUTHORIZE
→ ROUTE
→ OPERATE
→ EXECUTE
→ MONITOR
→ REVIEW
→ CORRECT
→ COMPLETE
```

```
→ RECORD
→ ANALYZE
→ REPORT
→ DECIDE
→ ACT

The architecture should show that CIRIL IOTS is a continuous operational
control and intelligence system.

It is not merely:

LOGIN + DASHBOARD + TASKS.

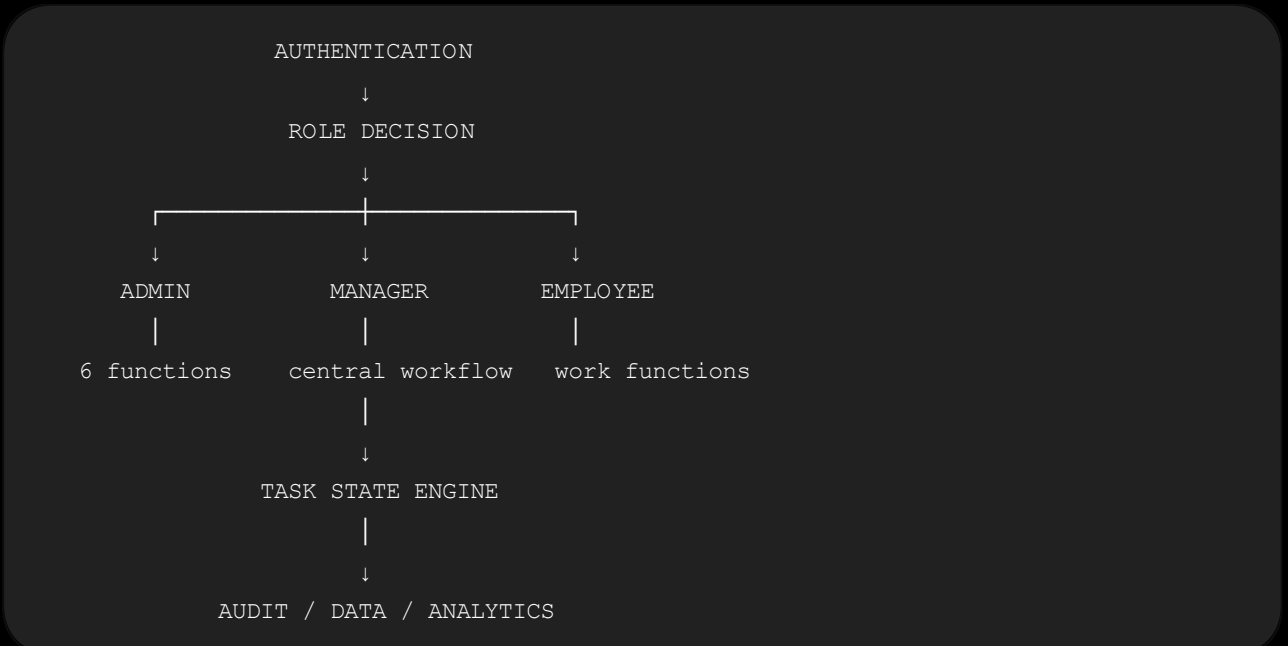
It is:

IDENTITY
+
AUTHORIZATION
+
PEOPLE
+
WORK
+
ATTENDANCE
```

The key change from my previous prompt

The biggest correction is that **I am no longer asking the AI to invent the visual architecture first**.

Your actual Draw.io already has a strong structure:



That matches the evolution documented in the attached markdown: **Admin → Manager → Employee was your original architectural abstraction**, and only afterward did the project expand into the detailed organizational roles.

Pasted markdown +1

And your current Draw.io has already moved significantly beyond the old three-branch version: the Manager area now contains **Team Management, Performance Monitoring, Workload Management, Attendance Management, Team Communication, Feedback & Evaluation, Issues Management**, alongside the central Task Management flow, while the Employee side has **Work Management, Task Submission, evidence, issue reporting, and manager intervention**. That means the architecture prompt should preserve this richer structure rather than collapsing it back to the earlier simplified model.

The other important point is that your **Employee Dashboard specification is now mature enough to be treated as a proper functional architecture**, not just a dashboard menu. Your attached response itself shows the evolution from the original task-only Employee flow into Work Management, Attendance, Requests, Notifications, Performance, Profile/Security, and Analytics. Pasted markdown

This refactored version therefore gives the AI a much stronger instruction:

“Do not redesign my architecture from scratch. Understand my existing architecture, complete it, strengthen its logic, and then build the deeper technical architecture underneath it.”

That is the direction I would use going forward.

Sources

based on the diagram, now generate: the textual content version that express the list of functionalities based on the diagram from admin, manager, employee, based on the earlier remembered response..

Worked for 1m 12s

Absolutely, mate. Based on the **current Draw.io architecture** plus the **earlier Admin → Manager → Employee structure**, here is the clean **textual functionality specification**.

The important distinction is preserved: **Admin, Manager, and Employee are the three architectural interface categories**, not the final 11 organizational roles. Pasted markdown

CIRIL IOTS — Functional Structure

```
IOTS
|
├── Authentication & Authorization Layer
|   |
|   ├── User Login
|   ├── Credential Validation
|   |   ├── Valid → Continue
|   |   └── Invalid → Notify Invalid Credentials → Return
|   |
|   ├── Location Validation
|   |   ├── Known / Valid → Continue
|   |   └── Unknown → Mark Session as Remote / Off-site
|   |
|   ├── Record Login & Location Coordinates
|   └── Decide User Role
|
└── Management Layer
    |
    ├── ADMIN
    ├── MANAGER
    └── EMPLOYEE
```

---

1. ADMIN — System-Level Management

The Admin interface is responsible primarily for **system administration, configuration, global oversight, and security control**. The earlier structure established six major Admin branches. Pasted markdown

ADMIN DASHBOARD

A. User Management

```
User Management
|
├── Create New User
├── Set User Role & Permissions
├── Activate / Deactivate User Account
├── Reset User Password
└── Record Changes in Audit Log
```

This manages the complete user/account lifecycle. Pasted markdown

B. Location & Attendance Control

```
Location & Attendance Control
|
```

```
├── Set Office Location Coordinates
├── Define Standard Working Hours
├── Monitor Daily Attendance Records
├── Review Off-site / Flagged Logins
└── Update / Record Attendance Result
```

This is the **system-level control** of attendance and location rules, while managers use that information operationally for their teams. Pasted markdown

C. Global Task Management

```
Global Task Management
│
├── Monitor All Tasks
├── Select Specific Task for Intervention
├── Edit Task Details
├── Reassign Task
├── Cancel Task
├── Force / Override Approval Where Authorized
└── Record Changes in Audit Log
```

This is a **global intervention and oversight function**, not the normal day-to-day task allocation mechanism. Pasted markdown

D. Announcement Dashboard

```
Global Announcements
│
├── Create New Notice / Message
├── Select Target Audience
│   ├── Specific User
│   ├── Role-based Audience
│   └── All Users
├── Set Message Display Duration
├── Publish to Dashboards
└── Record Action
```

Pasted markdown

E. System Settings

```
System Settings
│
├── Access System Registry
├── Create / Manage Roles & Designations
├── Manage Automated Report Settings
├── Update System Notification Templates
└── Save Configuration Changes
```

Pasted markdown

F. Security & Audit Viewer

```
Security & Audit Viewer
│
├── Access Security Logs
├── Filter Logs
│   ├── By Date
│   ├── By User
│   └── By Action / Task Status
├── Review User Activity
├── Review Login History
├── Review Task Modification History
└── Export Security Reports
```

The earlier design treated this as primarily a **monitoring/read-oriented security view**.
Pasted markdown

---

2. MANAGER — Team-Level Management

The Manager interface evolved from simple task assignment into a much broader **team operations and performance management workspace**. The earlier structure explicitly expanded beyond task origination into team performance, attendance, QA, and issue handling. Pasted markdown

MANAGER DASHBOARD

A. Task Management & Workflow

```
Task Management & Workflow
│
├── Create a Task
├── Define Task / Deliverables
├── Set Priority & Due Date
├── Check Employee Workload
├── Assign Task to Employee
├── Notify Employee
├── Monitor Task Status
├── Monitor Execution
├── Handle Overdue Tasks
├── Handle Reassignment
├── Receive Submitted Work
├── Review Task
├── Approve Work
├── Provide Feedback / Corrections
```

```
├── Request Revision
├── Review Resubmission
└── Complete & Close Task
```

The key managerial principle is **checking employee capacity before assignment**, rather than simply assigning work blindly. Pasted markdown

B. Team Management

```
Team Management
│
├── View Team Members
├── View Employee Profiles
├── View Current Assignments
├── View Employee Workload
├── View Employee Availability
└── Track Employee Work Progress
```

C. Performance Monitoring

```
Performance Monitoring
│
├── Monitor Task Completion Rate
├── Review Overdue Tasks
├── Track Deadlines
├── Identify Performance Issues
└── Give Feedback
```

D. Workload Management

```
Workload Management
│
├── View Employee Workload
├── Identify Overloaded Employees
├── Check Employee Availability
├── Balance Task Distribution
└── Reassign Tasks When Required
```

E. Attendance Management

```
Attendance Management
│
├── View Employee Attendance
├── Review Late Records
├── Review Absent Records
├── Review Unknown / Off-site Locations
├── View Leave / Late Requests
```

```
├── Approve / Reject Leave
└── Adjust Workload When Required
```

This is deliberately different from Admin attendance control: **Admin controls the system; Manager uses attendance information to manage the team.** Pasted markdown

F. Team Communication

```
Team Communication
|
├── Send Team Announcements
├── Send Employee-Specific Messages
├── Share Task Instructions
├── Send Automated Deadline Reminders
└── Receive Employee Updates
```

G. Feedback & Evaluation

```
Feedback & Evaluation
|
├── Receive Feedback from Employees
├── Review Employee Work
├── Provide Feedback
├── Audit Evaluation
└── Monitor Improvement
```

H. Issues Management

```
Issues Management
|
├── Identify Issue
├── Assess Issue Severity
├── Manager Review
├── Resolve Issue
|      OR
├── Escalate to Higher Authority
├── Audit Issue Details
└── Monitor Improvement
```

The Manager therefore becomes the **operational control point between employee execution and management oversight**, rather than merely an assignment screen.
Pasted markdown

---

3. EMPLOYEE — Individual-Level Work Execution

The Employee branch in the current diagram is more compact, but the earlier design already evolved it into the broader employee workspace. Pasted markdown

EMPLOYEE DASHBOARD

A. Work Management

```
Work Management
│
├── My Task Workspace
├── Task Dashboard / Queue
├── View Task Details
├── View Priority
├── Track Due Date
├── Start / Continue Task Execution
├── Update Task Progress
├── Record Work Activity
├── Upload Supporting Evidence / Files
├── Submit Work for Review
├── Receive Manager Feedback
├── Make Corrections
├── Resubmit Work
└── Complete Task
```

The current diagram additionally contains the issue-handling path:

```
Task Execution
      ↓
Issue?
  ├── No → Continue Work
  │
  └── Yes
       ↓
   Raise Issue Ticket
       ↓
   Assign Issue Severity
       ↓
   Manager Intervention?
       ├── Yes → Manager Resolves → Continue Work
       └── No  → Escalation / Further Action
```

The earlier model also established **Work Activity** separately from task state:

```
Work Activity
├── Progress Updates
├── Work Notes
├── Time / Effort
└── Supporting Evidence
```

Pasted markdown

B. Attendance & Availability

```
Attendance & Availability
│
├── Check-In
├── Check-Out
├── Attendance History
├── Work Hours
└── Availability Status
    ├── Available
    ├── Busy
    ├── In Meeting
    ├── On Break
    ├── In Training
    ├── Do Not Disturb
    ├── Working Remotely
    ├── On Leave
    ├── Absent
    └── Offline
```

C. Request & Self-Service

```
Request & Self-Service
│
├── Leave Requests
├── Permission Requests
├── Scheduled Requests
├── Request Status
├── Request Logs
├── Manager Approval History
├── Task Clarification / Guidance
└── Resource Requests
```

D. Notification & Communication

```
Notification & Communication
│
├── New Task Notification
├── Reassigned Task Notification
├── Overdue Task Notification
├── Deadline Alerts
├── Manager Feedback
├── Announcements
└── Escalation Alerts
```

E. Performance & Activity

```
Performance & Activity
│
├── Task Activity Summary
├── Completion Metrics
├── Attendance Summary
├── Activity History
└── Feedback History
```

F. Profile & Security / Usage

```
Profile & Security
│
├── My Profile
├── Account Details
├── Request Account Changes
├── Change Password
└── Active Sessions
```

G. Intelligence & Analytical Layer

```
Intelligence & Analytical
│
├── Daily Reports
├── Weekly Reports
├── Monthly Reports
├── Role-Based Reports
├── Performance Analytics
├── Workload Analytics
├── Attendance Analytics
├── Team Analytics where permitted
└── Exception / Risk Analysis
```

This is the matured Employee structure we developed from the original simple task-execution branch. Pasted markdown

---

4. Core Manager–Employee Task Lifecycle

The three-role architecture becomes much clearer when the task engine is expressed separately:

```
MANAGER
   ↓
Create Task
   ↓
Define Deliverables
```

```
   ↓
Set Priority & Due Date
   ↓
Check Employee Workload
   ↓
Assign Employee
   ↓
Notify Employee
   ↓
EMPLOYEE
   ↓
Start / Continue Work
   ↓
Update Progress
   ↓
Record Work Activity
   ↓
Upload Evidence
   ↓
Submit Work
   ↓
MANAGER
   ↓
Review Task
   ↓
Approved?
   ├── YES
   │    ↓
   │  Complete & Close
   │
   └── NO
        ↓
     Feedback / Corrections
        ↓
     Employee Revision
        ↓
     Resubmit
        ↓
     Manager Review
```

This is the evolved workflow that introduced the **overdue, reassignment, task-state, and revision loops**. Pasted markdown

---

5. Exception / Intervention Paths

The diagram also implies that the system is not only a straight-line workflow.

Overdue

```
Active Task
    ↓
Deadline Reached
    ↓
OVERDUE
    ↓
Manager Reviews
    ↓
Resolution?
├── Extend Deadline
├── Reassign Task
└── Escalate / Intervene
```

Reassignment

```
Task
 ↓
Reassign
 ↓
Check Employee Workload
 ↓
Choose New Employee
 ↓
Notify New Owner
 ↓
Continue Task
 ↓
Audit Change
```

Issue / Blocker

```
Employee Encounters Issue
        ↓
Raise Issue Ticket
        ↓
Assign Severity
        ↓
Manager Intervention
        ↓
Resolve?
├── YES → Employee Continues Work
└── NO  → Escalate to Higher Authority
```

Review / Correction

```
Employee Submission
        ↓
```

```
Manager Review
        ↓
Approved?
├── YES → Complete / Close
└── NO  → Feedback / Corrections
              ↓
           Resubmit
```

---

6. Cross-Cutting Functions

These are not really owned by only one of the three characters. They operate across the IOTS:

```
Shared IOTS Services
│
├── Authentication
├── Role-Based Access Control
├── Location Verification
├── Attendance
├── Task Workflow Engine
├── Notifications
├── Requests & Approvals
├── Issue / Escalation Handling
├── Audit Logging
├── Reporting
└── Intelligence & Analytics
```

The earlier architecture explicitly positioned these shared services beneath the three Admin/Manager/Employee interfaces. Pasted markdown

---

Final Functional Hierarchy

```
CIRIL IOTS
│
├── AUTHENTICATION & AUTHORIZATION
│   ├── Login
│   ├── Credential Validation
│   ├── Location Validation
│   ├── Login & Location Audit
│   └── Role Decision
│
└── MANAGEMENT LAYER
    │
    ├── ADMIN — SYSTEM CONTROL
    │   ├── User Management
    │   ├── Location & Attendance Control
```

```
│    ├── Global Task Management
│    ├── Announcement Dashboard
│    ├── System Settings
│    └── Security & Audit Viewer
│
├── MANAGER — TEAM OPERATIONS
│    ├── Task Management & Workflow
│    ├── Team Management
│    ├── Performance Monitoring
│    ├── Workload Management
│    ├── Attendance Management
│    ├── Team Communication
│    ├── Feedback & Evaluation
│    └── Issues Management
│
└── EMPLOYEE — WORK EXECUTION
     ├── Work Management
     ├── Attendance & Availability
     ├── Request & Self-Service
     ├── Notification & Communication
     ├── Performance & Activity
     ├── Profile & Security
     └── Intelligence & Analytical
```

This is the cleanest textual representation of what your current diagram is communicating, while retaining the earlier evolution: **Admin = system-level control, Manager = team-level operation, Employee = individual-level execution.** Pasted markdown

One important architectural point: **the 11 actual CIRIL roles should later be mapped inside these three functional lanes**, rather than replacing this three-character structure. Pasted markdown

Sources

Today 2:12 PM

"Intelligence and Analytical Layer:

1. Performance Analytics: {Task completion rate, on-time completion, average time completion, target based, team manager employee specific performances.}
2. Task & Workload Analytics: {Task distribution, pending tasks, overdue tasks, reassignment frequency,

particular-employee workload, and completion rate, team workload, etc.}

3. Attendance Analytics: {Late arrivals trends, absence trends, availability patterns, attendance of workforce patterns, etc.}
4. Exception / Risk Analysis: {overdue task detection, workload risk, repeated re-assignment (upon-certain-threshold), risk-classification: (low, med, high), pending requests, unresolved blockers}
5. More aspects can be included..

This layer can assist in: Work progress reports: {Daily, Weekly, Monthly, Role based reports, Team Analytics}" --> this is intelligence and analytical layer, that we discussed on our above chatting's right?, so based on this layer, elaborate each aspect with logic under the functionality.. make it completely copiable text.. elaboration here means, defining the functionality and every underlying handling logic

Show less

Yes, mate — **this is the Intelligence & Analytical Layer we established as the final analytical layer of the IOTS architecture**. Its purpose is not simply to display charts; it takes operational records from tasks, work activity, attendance, requests, blockers, reassignment, etc., converts them into measurable indicators, identifies exceptions and patterns, and produces reports for decision-making. That matches the earlier structure we established for Performance, Task/Workload, Attendance, Exception/Risk, and reporting.

Pasted markdown(1)

Below is a **fully elaborated, copy-ready functional specification**. I have kept your original five areas and expanded the underlying handling logic. Where an exact threshold or business formula had not been finalized in our earlier discussion, I have treated it as **configurable rather than inventing a fixed company rule**.

```
INTELLIGENCE AND ANALYTICAL LAYER
==================================

PURPOSE
-------
The Intelligence and Analytical Layer converts operational records generated by the
IOTS into measurable performance indicators, workload insights, attendance patterns
exceptions, risks, trends, and management reports.
```

This layer does not directly perform day-to-day work operations.
Instead, it analyzes the records created by the Management Layer and presents meaningful information to Employees, Managers, and higher-level management accordir to their authorized scope.

The layer should support:
- Historical analysis
- Current operational analysis
- Trend identification
- Performance measurement
- Workload analysis
- Attendance analysis
- Exception detection
- Risk classification
- Management reporting
- Role-based visibility
- Drill-down from summary information to underlying records

1. PERFORMANCE ANALYTICS

FUNCTION

Performance Analytics measures how effectively an individual employee, manager, team, role, or organizational unit is completing assigned work against defined expectations.

The system should not rely on a single performance value. It should combine multiple measurable indicators so that performance represents actual work outcomes rather than only task volume.

1.1 TASK COMPLETION RATE

Definition:
Measures the percentage of assigned tasks that were completed within the selected analysis period.

Logic:

Completion Rate =
(Number of tasks completed / Number of tasks that were due or assigned for the selected measurement period) × 100

Handling Logic:
- Identify all tasks belonging to the selected employee, manager, team, role, or organizational scope.
- Determine which tasks fall inside the selected reporting period.
- Count tasks whose final workflow outcome is Completed / Closed.
- Count tasks that were expected to be completed during that period.

- Exclude cancelled tasks from completion calculations where cancellation is an
  administrative outcome rather than employee non-completion.
- Reassigned tasks should retain their history and ownership information.
- Reassignment must not create duplicate task counts.
- Display the result together with the total number of tasks used in the calculatic
- Allow the user to drill from the percentage into the underlying task records.

Possible Output:
- Completion Rate: 91%
- Completed: 41
- Expected: 45
- Remaining: 4

1.2 ON-TIME COMPLETION

Definition:
Measures how many completed tasks were completed on or before their applicable
deadline.

Logic:
- Compare the actual completion/approval timestamp against the task deadline.
- A task completed on or before the deadline is classified as On-Time.
- A task completed after the deadline is classified as Late.
- Tasks whose deadlines were officially extended should use the latest authorized
  deadline for the final comparison, while retaining the original deadline in
  historical records.
- Tasks that became overdue and were later completed remain visible as late
  completions.
- Display both on-time percentage and late-completion count.

Output:
- On-Time Completion Rate
- Late Completion Rate
- Number of tasks completed on time
- Number of tasks completed late
- Average delay duration for late tasks

1.3 AVERAGE COMPLETION TIME

Definition:
Measures the average time required to move tasks from active execution to
completion.

Logic:
- Determine the defined start point for the task.
- Determine the actual completion point.
- Calculate the elapsed completion duration.
- Where the workflow records blocked/waiting periods, the reporting logic may

distinguish total elapsed time from active work time.
- Use a consistent business definition throughout the organization.
- Do not mix gross elapsed duration and active work duration without explicitly labelling them.
- Display the average together with task volume because a small sample can distort interpretation.

Output:
- Average completion time
- Median completion time where supported
- Fastest completion
- Longest completion
- Comparison with previous period

1.4 TARGET-BASED PERFORMANCE

Definition:
Measures actual achievement against a defined target.

Examples:
- Number of tasks expected
- Tasks completed
- Daily target
- Weekly target
- Monthly target
- Role-specific operational target

Logic:
- Retrieve the target applicable to the selected employee, role, team, or period.
- Determine actual achieved value.
- Compare actual achievement against target.
- Calculate achievement percentage.

Target Achievement =
(Actual achievement / Defined target) × 100

Handling:
- Targets should be period-specific.
- Targets should be role-aware where different roles have different expectations.
- Changes to targets should preserve their historical versions.
- Reports should indicate the target used for the calculation.
- A target of zero must not produce an invalid division result.
- Where no target exists, show "Target Not Configured" instead of assuming zero.

Output:
- Target
- Actual
- Achievement %
- Variance from Target
- Above / On Target / Below Target

1.5 EMPLOYEE-SPECIFIC PERFORMANCE

Definition:
Provides an individual employee performance view.

Logic:
- Filter all permitted task, attendance, activity, and performance records for the selected employee.
- Combine relevant indicators.
- Show performance over the selected period.
- Compare the employee with their own historical performance.
- Where organizational policy permits, compare against applicable team or role benchmarks.
- Do not expose information outside the viewer's authorized scope.

Possible Indicators:
- Task completion rate
- On-time completion
- Average completion time
- Target achievement
- Overdue task count
- Reassignment frequency
- Blocker frequency
- Work activity consistency
- Attendance indicators

1.6 MANAGER-SPECIFIC PERFORMANCE

Definition:
Measures managerial effectiveness based on team-level operational outcomes.

Logic:
- Aggregate performance of employees under the manager's authorized scope.
- Measure task allocation effectiveness.
- Measure overdue and blocked work.
- Measure review and approval activity.
- Measure resolution of employee issues.
- Measure team target achievement.
- Compare current team performance with previous periods.
- Do not evaluate the manager solely from employee task completion count.

Possible Indicators:
- Team completion rate
- Team on-time completion
- Overdue workload
- Review turnaround time
- Reassignment frequency

- Issue resolution time
- Team target achievement
- Team workload balance

1.7 TEAM PERFORMANCE

Definition:
Provides an aggregated performance view for a team.

Logic:
- Group employee records by team.
- Aggregate task and performance indicators.
- Show distribution rather than only a single team average.
- Identify whether poor team results originate from a few employees or are broadly distributed.
- Allow drill-down:
  Team → Employee → Task → Activity / Evidence

1.8 ROLE-BASED PERFORMANCE

Definition:
Measures performance according to the work type and expectations applicable to a specific role.

Logic:
- Identify the user's authorized role/designation.
- Apply the performance metrics configured for that role.
- Avoid forcing identical KPIs onto roles with fundamentally different work.
- Preserve a common reporting structure while allowing role-specific measures.

Example:
A BDE may be measured using lead/task outcomes,
while a Graphic Designer may be measured using creative-task completion,
revision cycles, submission quality, and deadline adherence.

2. TASK & WORKLOAD ANALYTICS
==========

FUNCTION

Task and Workload Analytics analyzes how work is distributed, how much work is pending, where bottlenecks exist, who is overloaded, and how effectively assigned work is progressing.

The layer should distinguish between:
- Task quantity
- Task status

- Workload
- Work progress
- Task ownership
- Reassignment history

2.1 TASK DISTRIBUTION

Definition:
Shows how tasks are distributed across employees, teams, managers, roles, priorities, and statuses.

Logic:
- Group tasks by the selected dimension.
- Count tasks in each group.
- Allow filtering by:
  - Date
  - Employee
  - Manager
  - Team
  - Role
  - Priority
  - Status
- Show both task count and percentage of total.
- Permit drill-down into the underlying task list.

Example:
Team A = 40%
Team B = 35%
Team C = 25%

2.2 PENDING TASK ANALYSIS

Definition:
Measures work that remains incomplete.

Logic:
- Identify tasks that are not in a final completed/closed state.
- Separate normal active work from exceptions such as overdue or blocked work.
- Group pending tasks by employee, team, priority, age, and deadline.
- Highlight long-pending tasks.
- Allow drill-down to task details.

2.3 OVERDUE TASK ANALYSIS

Definition:
Identifies tasks whose applicable deadline has passed without completion.

Logic:
- Compare current/reporting timestamp with the current authorized task deadline.
- If the task remains incomplete after the deadline, classify it as Overdue.
- Overdue tasks should not be treated merely as ordinary pending work.
- Record:
  - Overdue date
  - Duration overdue
  - Priority
  - Current owner
  - Manager
  - Whether the task was reassigned
  - Whether a manager intervention occurred
- Sort or classify overdue tasks by severity.
- Provide a direct path from analytics to manager intervention.

2.4 REASSIGNMENT FREQUENCY

Definition:
Measures how frequently tasks are reassigned.

Logic:
- Count reassignment events from the task history.
- Do not count a reassigned task as multiple tasks.
- Count each actual ownership-transfer event separately.
- Calculate:
  - Total reassignment events
  - Number of tasks reassigned
  - Reassignments per task
  - Reassignment by employee/team/manager
- Identify repeated reassignment patterns.
- Preserve the reason and history of every reassignment.

This can identify:
- Capacity problems
- Assignment problems
- Unclear ownership
- Workflow problems
- Repeated operational intervention

2.5 PARTICULAR-EMPLOYEE WORKLOAD

Definition:
Measures the amount and intensity of work assigned to a specific employee.

Logic:
- Retrieve all tasks currently assigned to the employee.
- Categorize by:

  - Active
  - Pending
  - Overdue
  - High priority
  - Blocked
  - Waiting for review
- Consider deadlines and task priority rather than simply counting tasks.
- Show current workload and historical workload.
- Compare current workload against the employee's configured/expected capacity,
  where such capacity information exists.

Possible Output:
Employee
Current Tasks
High Priority Tasks
Overdue Tasks
Blocked Tasks
Upcoming Deadlines
Workload Level

2.6 TEAM WORKLOAD

Definition:
Measures the distribution of active work across the team.

Logic:
- Aggregate employee workload records.
- Compare workload across team members.
- Identify concentration of work on individual employees.
- Identify employees with significantly lower or higher active workload.
- Show task volume alongside deadline pressure.
- Provide a manager drill-down from:
  Team → Employee → Tasks

2.7 WORKLOAD BALANCE

Definition:
Determines whether assigned work is reasonably distributed.

Logic:
- Calculate workload indicators for all applicable employees.
- Compare workload against configured capacity or expected workload.
- Classify employees as:
  - Normal
  - High
  - Overloaded
  where the actual thresholds are configurable.
- Generate management attention when workload becomes materially unbalanced.

- Do not automatically reassign work solely because an analytical threshold is reached; analytics should identify the condition, while authorized management actions perform the reassignment.

2.8 UPCOMING WORKLOAD

Definition:
Predicts near-term workload pressure from tasks whose deadlines are approaching.

Logic:
- Identify upcoming deadlines.
- Group tasks by employee/team.
- Consider task priority and due-date proximity.
- Detect concentration of deadlines within a short period.
- Flag employees likely to become overloaded.
- Present this as an operational warning rather than a confirmed failure.

3. ATTENDANCE ANALYTICS
==========

FUNCTION

Attendance Analytics analyzes attendance records, working hours, late arrivals, absences, availability, and workforce attendance patterns.

Attendance and Availability must remain separate concepts.

Attendance answers:
"Was the employee present / working?"

Availability answers:
"Can the employee currently accept or handle work?"

3.1 LATE ARRIVAL TRENDS

Definition:
Identifies patterns in late arrivals over time.

Logic:
- Retrieve attendance check-in records.
- Determine whether each arrival was on time or late using the organization's configured working schedule and applicable grace period.
- Count late events by:
  - Employee
  - Team
  - Role
  - Date

  - Week
  - Month
- Show trend over time.
- Distinguish occasional late arrivals from repeated patterns.
- Link back to attendance records for verification.

3.2 ABSENCE TRENDS

Definition:
Measures absence patterns across employees and teams.

Logic:
- Identify attendance records classified as absent.
- Group by employee, team, role, and reporting period.
- Distinguish approved leave from unexplained/flagged absence where the system contains that information.
- Show frequency and trend.
- Do not treat approved leave automatically as poor performance.

3.3 AVAILABILITY PATTERNS

Definition:
Measures how employee availability changes throughout working periods.

Logic:
- Analyze recorded availability states such as:
  Available
  Busy
  In Meeting
  On Break
  In Training
  Do Not Disturb
  Working Remotely
  On Leave
  Absent
  Offline
- Measure duration/frequency where such timestamps are available.
- Identify recurring periods of low availability.
- Compare availability with workload and task deadlines.

3.4 WORKFORCE ATTENDANCE PATTERNS

Definition:
Provides an aggregated view of workforce attendance behavior.

Logic:

- Aggregate attendance records across teams, roles, branches, or departments.
- Identify:
  - Attendance rate
  - Late-arrival patterns
  - Absence patterns
  - Remote/off-site patterns where recorded
  - Working-hour trends
- Compare current period with historical periods.
- Identify recurring organization-level patterns.

3.5 WORK HOURS ANALYSIS

Definition:
Measures recorded working duration for employees and teams.

Logic:
- Use check-in and check-out records.
- Account for recorded break periods where supported.
- Calculate work duration according to the organization's configured attendance policy.
- Show:
  - Total hours
  - Average hours
  - Daily hours
  - Team totals
- Flag incomplete or inconsistent attendance records separately instead of silently treating them as zero hours.

4. EXCEPTION / RISK ANALYSIS
==========

FUNCTION

Exception and Risk Analysis identifies operational conditions that require attention.

It converts raw operational conditions into manageable warnings.

The system should distinguish between:
- Normal operation
- Attention required
- High-risk operational condition

Risk classification should be rule-based and configurable unless a formal predictive model is implemented.

4.1 OVERDUE TASK DETECTION

Logic:
- Continuously or periodically evaluate active task deadlines.
- Detect tasks whose deadline has passed without completion.
- Calculate overdue duration.
- Consider task priority.
- Consider whether the task is blocked, awaiting review, or otherwise subject
  to a valid workflow condition.
- Create an analytical exception where appropriate.
- Notify or surface the exception to the authorized manager dashboard.
- Track whether the exception was resolved, extended, reassigned, or escalated.

4.2 WORKLOAD RISK

Definition:
Identifies situations where an employee/team may not have sufficient capacity
to handle assigned work within expected deadlines.

Logic:
- Evaluate active task count.
- Evaluate upcoming deadlines.
- Evaluate priority.
- Evaluate overdue tasks.
- Evaluate blocked work.
- Compare workload against configured capacity where available.
- Combine the conditions into a workload-risk assessment.

Example logic:
High active workload
+
multiple approaching deadlines
+
existing overdue work
=
Higher workload risk

4.3 REPEATED REASSIGNMENT RISK

Definition:
Detects when a task or employee is repeatedly involved in reassignment events.

Logic:
- Count reassignment events over a configurable period.
- Determine whether the configured threshold has been exceeded.
- Identify:
  - Task repeatedly reassigned
  - Employee frequently receiving reassigned tasks
  - Employee frequently losing assigned tasks

  - Team-level reassignment concentration
- Record the reason wherever available.
- Create a risk/attention indicator when the configured threshold is reached.
- Do not interpret reassignment automatically as employee failure; the reason
  must remain available for management review.

4.4 RISK CLASSIFICATION

The system can classify detected exceptions into:

LOW

Indicates a condition that should be monitored but does not currently require
urgent intervention.

MEDIUM

Indicates a recurring or materially important condition that should receive
manager attention.

HIGH

Indicates a condition that requires immediate managerial or authorized higher-level
attention.

Handling Logic:
- Risk level is generated from configured business rules.
- Rules may consider:
  - Severity
  - Frequency
  - Duration
  - Priority
  - Impact
  - Repeated occurrence
- Risk thresholds should be configurable.
- Every risk result should show why it was classified at that level.
- Risk should be traceable to underlying operational records.
- Risk status should be capable of moving through:
  Detected → Reviewed → Action Taken → Resolved
- Resolved risks remain in historical records for trend analysis.

4.5 PENDING REQUEST RISK

Definition:
Identifies requests that remain undecided beyond the expected handling period.

Examples:
- Leave request

- Permission request
- Resource request
- Task clarification request

Logic:
- Determine request creation time.
- Determine expected decision/response period where configured.
- Identify requests that remain pending beyond that period.
- Group by request type, requester, manager, and age.
- Surface delayed requests to the appropriate authority.
- Preserve request history and decisions.

4.6 UNRESOLVED BLOCKER ANALYSIS

Definition:
Identifies employee-reported issues/blockers that remain unresolved and may impact work completion.

Logic:
- Detect active blocker/issue records.
- Measure blocker age.
- Identify affected task.
- Identify employee and responsible manager.
- Determine severity.
- Determine whether escalation has occurred.
- Prioritize blockers based on severity, task priority, and duration.
- Surface long-running blockers as management exceptions.

4.7 COMBINED OPERATIONAL RISK

The system may combine multiple independent exceptions to identify a larger operational concern.

Example:

Employee has:
- High workload
- Multiple approaching deadlines
- Existing overdue task
- Open blocker

The analytical layer can combine those conditions into a higher-priority management attention item.

Important:
This does not automatically mean the employee is underperforming.
The system identifies the operational condition and allows management to inspect the underlying causes.

5. TREND & HISTORICAL ANALYSIS

FUNCTION

Historical analysis compares current operational performance with previous periods to identify improvement, deterioration, recurring patterns, and changes.

Logic:
- Store historical analytical snapshots or calculate from historical operational records.
- Compare:
  - Current day vs previous day
  - Current week vs previous week
  - Current month vs previous month
  - Current period vs equivalent previous period
- Show direction:
  - Improving
  - Stable
  - Declining
- Allow drill-down into the records causing the trend.

5.1 PERFORMANCE TRENDS

Analyze:
- Completion rate over time
- On-time completion over time
- Target achievement over time
- Average completion time over time

5.2 WORKLOAD TRENDS

Analyze:
- Task volume over time
- Pending workload
- Overdue workload
- Reassignment frequency
- Workload concentration

5.3 ATTENDANCE TRENDS

Analyze:
- Attendance rate
- Late-arrival frequency

- Absence frequency
- Availability patterns
- Working-hour trends

5.4 RISK TRENDS

Analyze:
- Number of detected risks
- Low / Medium / High distribution
- Repeated risk types
- Average resolution time
- Unresolved exception count
- Recurrent operational problems

6. REPORTING AND WORK PROGRESS REPORTS
==========

FUNCTION

The Intelligence and Analytical Layer converts the available analytical data into structured work-progress reports for different reporting periods and authorized roles.

6.1 DAILY REPORT

Purpose:
Provides a snapshot of current operational activity.

Should include, where applicable:
- Tasks assigned
- Tasks completed
- Pending tasks
- Overdue tasks
- Active workload
- Attendance status
- Significant blockers
- Requests pending
- Major exceptions
- Daily performance indicators

Daily reports should emphasize:
"What happened today?"
and
"What requires attention now?"

6.2 WEEKLY REPORT

Purpose:
Shows operational progress across the week and identifies patterns that may not be visible in a single day.

Should include:
- Weekly task completion
- On-time completion
- Target achievement
- Workload distribution
- Overdue work
- Reassignment frequency
- Attendance trends
- Open blockers
- Exception/risk summary
- Performance movement

Weekly reports should answer:
"How did the team perform this week?"
and
"Where are recurring issues appearing?"

6.3 MONTHLY REPORT

Purpose:
Provides a broader management-level view of operational performance.

Should include:
- Monthly completion rate
- Target achievement
- On-time performance
- Average completion time
- Workload trends
- Attendance trends
- Performance trends
- Reassignment trends
- Risk and exception summary
- Resolved vs unresolved issues
- Comparison with previous months

Monthly reporting should emphasize:
"What is the overall operational trend?"
and
"Where does management need to act?"

6.4 ROLE-BASED REPORTS

Definition:
Reports must change according to the role and authority of the viewer.

Employee:
- Personal task performance
- Personal workload
- Personal attendance
- Personal activity
- Personal feedback
- Authorized personal analytics

Manager:
- Team performance
- Team workload
- Employee-level performance within scope
- Attendance
- Overdue work
- Blockers
- Requests
- Exceptions
- Team trends

Higher Management:
- Organization / branch / department performance
- Cross-team comparisons
- Major operational exceptions
- Workload trends
- Performance trends
- High-risk conditions
- Management-level summaries

Admin:
- System-oriented operational reporting
- User/account activity where authorized
- Attendance/location/system-related reporting
- Task/system oversight
- Audit and security reporting according to permission scope

6.5 TEAM ANALYTICS

Purpose:
Provide an aggregated team-level analytical view.

Should support:
- Team task completion
- Team on-time completion
- Team target achievement
- Team workload
- Employee workload distribution
- Team attendance

- Team availability
- Overdue workload
- Reassignment frequency
- Open blockers
- Team risk indicators

Drill-down:
Team
→ Employee
→ Task
→ Work Activity
→ Evidence / History

The same hierarchy should preserve authorization at every level.

7. ANALYTICAL FILTERING AND DRILL-DOWN

The analytical layer should allow users to filter information according to their authorized scope.

Possible filters:
- Date
- Date range
- Employee
- Manager
- Team
- Role
- Department / Branch where configured
- Task priority
- Task status
- Request type
- Risk level
- Attendance status

Drill-down logic:

Summary Metric
    ↓
Category / Group
    ↓
Specific Employee / Team
    ↓
Specific Task / Request / Attendance Record
    ↓
Detailed Operational History

Example:

Team Completion Rate
    ↓

Team A
    ↓
Employee X
    ↓
Task 1045
    ↓
Task Activity / Submission / Review / Audit History

8. DATA HANDLING LOGIC

The analytical layer should read from operational records rather than manually re-entering information.

Primary analytical sources include:
- User records
- Role and authorization records
- Task records
- Task state history
- Work activity records
- Work notes
- Effort/time records
- Supporting documents/evidence
- Attendance records
- Availability records
- Requests
- Approval records
- Blockers/issues
- Escalation records
- Reassignment history
- Audit records
- Configured targets
- Configured working schedules

8.1 STATE-AWARE ANALYSIS

The analytical layer must understand the difference between task states.

Examples:
Assigned
Pending / In Progress
Overdue
Reassigned
Completed

Operational execution may additionally contain:
Blocked
Waiting for Review
Under Review

Changes Requested
Issue Resolved
Done / Closed

Reassigned should be treated as a workflow event/history rather than counting the same task as a new task.

Overdue should be treated as an exception requiring attention rather than simply another normal task category.

8.2 HISTORICAL PRESERVATION

Analytical results must be traceable to historical operational records.

When task assignments, deadlines, priorities, roles, or other operational values change:
- Current values should represent the current state.
- Historical values should remain available for analysis.
- Reports should be able to explain why historical results changed.
- Reassignment, deadline extension, review, and approval history should not be destroyed.

8.3 AUDIT TRACEABILITY

Every important analytical result should be traceable back to its source data.

Example:

Risk = HIGH
    ↓
Reason:
    - 3 overdue tasks
    - 2 high-priority deadlines approaching
    - 1 unresolved blocker
    ↓
Underlying Records:
    Task IDs
    Blocker ID
    Workload Records
    Attendance / Availability Records where relevant

This allows management to verify the analytical conclusion rather than treating the dashboard value as unexplained.

9. MANAGEMENT ATTENTION / ACTION CENTRE
==========

The analytical layer may provide an "Operational Attention" area that highlights items requiring human review.

Possible attention items:
- High-risk workload
- Overdue critical tasks
- Repeated reassignment
- Long-pending requests
- Unresolved blockers
- Attendance anomalies
- Declining performance trend
- Significant target variance

Each item should provide:
- What happened
- Who/what is affected
- Why it was flagged
- Risk level
- When it occurred
- Current status
- Authorized next action
- Link to underlying record

10. INTELLIGENCE MATURITY

The Intelligence and Analytical Layer can be implemented in progressive levels.

LEVEL 1 — DESCRIPTIVE ANALYTICS

Answers:
"What happened?"

Examples:
- 40 tasks completed
- 5 tasks overdue
- 3 requests pending
- Attendance rate = 94%

LEVEL 2 — DIAGNOSTIC ANALYTICS

Answers:
"Why is it happening?"

Examples:
- Overdue work concentrated in one employee
- Reassignment increased during the last week
- Delays are associated with unresolved blockers
- Team workload is unevenly distributed

LEVEL 3 — PREDICTIVE / ADVANCED ANALYTICS

Answers:
"What is likely to happen?"

Examples:
- Employee may become overloaded
- Deadline risk is increasing
- Repeated blockers may affect completion
- Current workload may create a future overdue condition

Predictive functionality should only be labelled as predictive/AI when an actual predictive model or appropriate analytical method is implemented.

Rule-based thresholds and condition checks should remain clearly identified as rule-based analytics.

11. ROLE-BASED ANALYTICAL VISIBILITY

EMPLOYEE

Focus:
- My performance
- My tasks
- My workload
- My attendance
- My progress
- My activity
- My authorized reports

MANAGER

Focus:
- Team performance
- Team workload
- Employee workload
- Overdue work
- Attendance
- Availability
- Blockers
- Requests
- Exceptions
- Team trends

SENIOR / GENERAL MANAGEMENT

Focus:

- Cross-team performance
- Operational workload
- Major exceptions
- Organizational trends
- Performance movement
- Targets
- Management attention areas

CHAIRMAN / EXECUTIVE VIEW

Focus:
- Overall operational health
- Key performance indicators
- Major risks
- Major escalations
- Team / organizational trends
- Strategic operational summaries

The analytical layer should not expose lower-level information merely because a person can see a dashboard. Visibility must follow authorization and data scope.

12. FINAL ANALYTICAL OUTPUT

The Intelligence and Analytical Layer should ultimately transform:

RAW OPERATIONAL DATA
↓
DATA AGGREGATION
↓
METRIC CALCULATION
↓
TREND ANALYSIS
↓
EXCEPTION DETECTION
↓
RISK CLASSIFICATION
↓
MANAGEMENT INSIGHT
↓
REPORT
↓
MANAGEMENT ACTION

CORE ANALYTICAL FUNCTIONS
---

1. Performance Analytics
   - Task completion rate

- On-time completion
- Average completion time
- Target achievement
- Employee-specific performance
- Manager-specific performance
- Team performance
- Role-based performance

2. Task & Workload Analytics
- Task distribution
- Pending task analysis
- Overdue task analysis
- Reassignment frequency
- Employee workload
- Team workload
- Workload balance
- Upcoming workload pressure

3. Attendance Analytics
- Late arrival trends
- Absence trends
- Availability patterns
- Workforce attendance patterns
- Working-hour analysis

4. Exception / Risk Analysis
- Overdue detection
- Workload risk
- Reassignment risk
- Low / Medium / High risk classification
- Pending request risk
- Unresolved blocker analysis
- Combined operational risk

5. Trend & Historical Analysis
- Performance trends
- Workload trends
- Attendance trends
- Risk trends
- Period-over-period comparison

6. Reporting
- Daily reports
- Weekly reports
- Monthly reports
- Role-based reports
- Team analytics
- Management-level reports
- Drill-down operational reports

7. Management Attention
- Identify issues

```
   - Explain why they were flagged
   - Classify severity/risk
   - Link to underlying records
   - Present authorized management actions


FINAL PRINCIPLE
---------------

The Intelligence and Analytical Layer should not merely answer:

"How many tasks are there?"
```

The core logic of the layer

The most important architectural point is that the analytical layer should sit **above the operational records**, not become another place where people manually maintain data:

```
Task / Work / Attendance / Requests / Issues / Audit
                    ↓
          Intelligence Engine
                    ↓
       Metrics + Trends + Exceptions
                    ↓
           Risk / Insight
                    ↓
       Daily / Weekly / Monthly
              Reporting
                    ↓
        Management Decision
```

And this preserves the distinction we established earlier:

Management Layer = performs and controls the work.
Intelligence & Analytical Layer = interprets the work and helps management understand what is happening. Pasted markdown

Sources

ChatGPT can make mistakes. Check important info.